A Unified Benchmark for RL Post-Training of Language Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

RL post-training of language models is now shaped by a small set of algorithms (PPO, GRPO, DPO) and a growing set of frameworks (TRL, TINKER, OpenRLHF, veRL), yet it remains unclear which empirical conclusions transfer across implementations, model families, and scales. We present TINKERRL-BENCH, a benchmark spanning 70+ runs across 7 RL libraries and 5 model families (0.6B–$\sim$671B parameters) on GSM8K, HumanEval, and synthetic tool-use tasks. Results to date cover five end-to-end stacks (TRL, TINKER, SB3, CleanRL, Tianshou); the remaining libraries are scaffolded but not yet run. The strongest evidence in the current release comes from short-horizon GSM8K training dynamics; held-out evaluation and non-math coverage remain narrower.

Three conclusions are supported conservatively. First, Zero-Variance Fraction (ZVF)—the share of GRPO groups with identical rewards—tracks when binary outcome rewards stop producing within-group learning signal. Because ZVF is mechanically coupled to reward sparsity, group size, and baseline accuracy, we use it as a descriptive diagnostic rather than a standalone causal predictor. Second, trainability in our current sweeps varies substantially with initialization and rollout regime: instruction-tuned checkpoints were generally easier to optimize than comparable base models, and intermediate group sizes often behaved better than the smallest or largest settings we tested, but we do not claim a universal optimal group size. Third, PPO/GRPO rankings and frontier-model stability are heterogeneous across model families and end-to-end stacks; our single-seed API runs should be read as case studies, not universal laws. A separate held-out GSM8K control shows that the mean Qwen3-8B-Instruct GRPO delta over the same checkpoint's pre-RL held-out accuracy is small and not statistically significant (83.3% vs. 82.0%, p=0.26), while tool-use and code results remain limited by sparse rewards and custom evaluation.

We release code, logs, and checkpoints at `https://anonymous.4open.science/r/tinker-rl-lab`.

1 Introduction

Reinforcement learning post-training now underpins much of modern language-model reasoning and alignment [**?** **?**], but empirical claims in this area are often hard to compare. PPO [**?**], GRPO [**?**], and DPO [**?**] are usually evaluated on different model families, with different framework defaults, different reward functions, and different notions of "success." What the field needs is not another leaderboard but a substrate that separates algorithmic conclusions from implementation, initialization, and task-design effects.

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

TINKERRL-BENCH is an attempt at such a substrate. It aggregates 70+ runs spanning 7 RL libraries (five with completed results), 5 model families, 32+ checkpoints, and scales from 0.6B to $\sim$671B parameters, with common reporting conventions and released artifacts. At the same time, we emphasize a central caveat: the current benchmark is *not* uniformly mature across tasks. The strongest evidence comes from short-horizon GSM8K experiments with verifiable binary rewards. Frontier API runs, tool-use experiments, and code-generation studies are useful case studies, but many remain single-seed, short-horizon, partially completed, or custom-evaluated.

The first contribution of the benchmark is therefore methodological rather than triumphalist: it makes visible how much end-to-end stack choice matters. In our results, model family, initialization, rollout regime, and framework defaults can all change the apparent algorithm ranking. Several comparisons that look like "PPO vs. GRPO" are better understood as *stack-level comparisons*, because the training backends differ in tokenizer/runtime integration, managed defaults, and rollout plumbing. This is precisely why a shared measurement substrate is needed.

Our second contribution is a descriptive diagnostic for sparse-reward GRPO: Zero-Variance Fraction (ZVF), the fraction of rollout groups with identical rewards, together with Gradient Utilization (GU = $1 - \text{ZVF}$). ZVF is useful because it directly reveals when within-group contrast disappears. However, because our main tasks use binary outcome rewards, ZVF is mechanically coupled to reward sparsity, group size, and baseline accuracy. We therefore use ZVF/GU as diagnostics of signal degeneracy, not as proof of an independent or causal failure mode beyond simpler observables such as reward mean, entropy, advantage variance, or divergence proxies.

Our third contribution is a set of targeted ablations on trainability. In the current benchmark, trainability varies substantially with initialization and rollout regime: instruction-tuned checkpoints were generally easier to optimize than comparable base models, and intermediate rollout group sizes often behaved better than the smallest or largest settings we tested. These observations support a narrower claim: whether RL fine-tuning works at all can depend materially on initialization and reward-bearing rollout structure, in addition to the nominal RL algorithm. They do *not* yet justify a universal optimal group size or a general "SFT dominates RL" law.

Finally, we explicitly separate training reward from held-out generalization. For Qwen3-8B-Instruct on GSM8K, a five-seed post-GRPO held-out evaluation yields 83.3% accuracy, but the same instruction-tuned checkpoint's pre-RL held-out accuracy under the identical greedy protocol is already 82.0%; the +1.3 percentage-point delta is not statistically significant (p=0.26). This negative control matters: strong online reward curves do not by themselves establish generalization gains. We release the benchmark, step-level traces, and checkpoints so that stronger follow-up work can test exactly these failure points.

2 Related Work

TINKERRL-BENCH sits at the intersection of five rapidly evolving lines of work: policy-gradient *algorithms* for language models, RL for *reasoning* (including process reward models), RL for *tool use* and agentic behaviour, empirical *scaling laws* for RL post-training, and the *frameworks* and *benchmarks* that instantiate these algorithms. We situate our contribution against more than fifty recent works; entries marked with † appeared at NeurIPS, ICML, or ICLR in the 2025–2026 cycle.

2.1 RL Algorithms for Large Language Models

Proximal Policy Optimization (PPO) [**?**] is the workhorse algorithm behind InstructGPT-style RLHF [**? ? ? ? ?**] and has dominated production deployments for five years. Direct Preference Optimization [**?**] reframed preference learning as contrastive classification and spawned a large family of RL-free alternatives, including IPO [**?**], KTO [**?**], SimPO [**?**], ORPO [**?**], and SLiC [**?**], all of which we consider as no-rollout baselines in our cross-library protocol.

The current wave is defined by *group-relative* policy gradient methods. GRPO was introduced with DeepSeekMath [**?**] and popularised by DeepSeek-R1 [**? ?**], the first reasoning RL method published in *Nature*, which showed that pure outcome-reward RL can elicit chain-of-thought behaviour from a base model without supervised cold-start data. A second wave of refinements dissects the biases of vanilla GRPO. Dr. GRPO [**?**] removes a response-length bias and a question-difficulty amplification bias by using a constant loss normaliser and dropping the standard-deviation denominator in the

advantage. GDPO [**?**] decouples group-relative normalisation per reward component, preventing advantage collapse when rewards are multi-valued. MC-GRPO (median-centered advantage) [**?**] replaces the group-mean baseline in the advantage with the group median to stabilise group-relative normalisation under small rollout counts. G^2RPO [**?**] forces per-prompt advantages to match $\mathcal{N}(0, 1)$ to fix reward-topology variance in multimodal settings. Wu et al. [**?**] prove that GRPO is "secretly DPO" — the group-level advantage is an implicit contrastive objective — and show that 2-GRPO retains 98.1% of the performance of 16-GRPO at 12.5% of the rollout budget.

A parallel thread focuses on *production-scale* refinements. GSPO, from the Qwen team [**?**], redefines the importance ratio at the sequence level (rather than token level) and was credited with stabilising the Qwen3 series, especially its Mixture-of-Experts variants. DAPO, from ByteDance [**?**], contributes asymmetric Clip-Higher, dynamic sampling, token-level loss normalisation, and overlong filtering, reaching 50 points on AIME 2024 with a 32B model. Kimi-K1.5 [**? ?**] and DeepSeek-V3 [**?**] both report their own GRPO variants and online mirror-descent-style updates. Orthogonal production variants include VAPO [**?**] and Dr. SAC [**?**]; the last argues that classical REINFORCE with a value baseline, when carefully implemented, matches GRPO on several RLHF tasks.

A third thread attacks the *zero-variance failure mode* (ZVF) directly. GRESO [**?**] pre-screens prompts before sampling; CPPO [**?**] prunes completions post-rollout, giving up to $7.98\times$ speedup on GSM8K. NGRPO [**?**] adds Advantage Calibration and asymmetric clipping so that all-incorrect groups still carry gradient. Scaf-GRPO [**?**] injects tiered scaffolding hints when learning stagnates, boosting AIME 2024 by 44.3% relative. MO-GRPO [**?**] addresses vanilla GRPO's collapse to a single reward component under multiple objectives via variance-based reward re-weighting. Finally, SSPO [**?**] operates at sentence granularity to balance the stability of GSPO against the expressiveness of token-level GRPO, improving by 3.56 points on math benchmarks. Taken together, these results explain at a theoretical level why implementation details at the granularity of the importance ratio have such large empirical effects — an explanation our benchmark complements with a 73 pp cross-library measurement.

2.2 Reasoning RL and Process Reward Models

The shift from outcome rewards to *process* supervision has reshaped reasoning RL. Let's-Verify-Step-by-Step [**?**] showed that step-level PRMs dramatically outperform outcome reward models on GSM8K and MATH. Math-Shepherd [**?**] derives process rewards from auto-generated process annotations, avoiding costly human labels. PRM800K [**?**] and PRMBench [**?**] provide evaluation data for step-level scoring. OmegaPRM [**?**] uses Monte-Carlo tree search to supervise process rewards without human-in-the-loop, while ReasonEval [**?**] introduces reasoning-specific reward models. Implicit PRM [**?**] learns PRMs implicitly from outcome labels, and VersaPRM [**?**] extends process rewards to multi-domain tasks. These supervision signals are the backbone of recent reasoning models, including OpenAI's o1/o3 family [**? ?**], DeepSeek-R1 [**? ?**], Qwen2.5-Math [**?**], Qwen3 [**?**], Gemini 2.5 Pro's deliberative mode [**?**], and Kimi-K2 [**?**]. Complementary results demonstrate that self-consistency [**?**], tree-of-thought search [**?**], and critic-style post-hoc verifiers [**?**] all benefit from or substitute for process reward supervision. VerifyBench [**?**] and A Sober Look [**?**] warn, however, that reward hacking and evaluation artefacts can inflate perceived gains, motivating our insistence on binary verifiable rewards for mathematical reasoning.

Reasoning-specific algorithmic contributions also continue to accumulate. ReFT [**?**] studies supervised fine-tuning versus RL for reasoning. Let's Reward Step [**?**] and STaR [**?**] bootstrap reasoning chains from weak supervisors. rStar-Math [**?**] combines MCTS-based PRMs with self-evolved reasoning. ReST-EM [**?**] alternates expectation and maximisation steps over self-generated rationales. Self-Taught Evaluator [**?**] adapts an LLM to score its own reasoning chains. GRPO-Lead [**?**], LCPO [**?**], and LIMR [**?**] each tune reasoning depth with length-aware rewards. Search-R1 [**?**] trains reasoning agents to interleave search queries with chain-of-thought. TTRL [**?**] performs test-time RL against self-generated verifiers. We use the Dr. GRPO [**?**] and GSPO [**?**] formulations as reference implementations when measuring reasoning accuracy across the 0.6B–235B frontier.

2.3 Tool-Use and Agentic RL

Tool-use RL trains LLMs to interleave natural-language reasoning with external API calls. Toolformer [**?**] introduced self-supervised tool-call insertion for a small set of APIs; Gorilla [**?**] retrieved from a large API catalogue; and ToolLLM [**?**] scaled training to thousands of real-world REST APIs. ReAct [**?**] established the interleaved reason–act–observe pattern that most subsequent agents inherit. RL specifically for tool use was pioneered by ToolACE [**?**], which couples a tool-calling reward with a self-evolving synthesis pipeline; ToolRL [**?**] applies GRPO with verifiable tool-call rewards; ToRL [**?**] optimises the joint reasoning–action trajectory. Further advances include APIGen [**?**] (high-quality synthetic API data), Nexus-Raven [**?**], and Octopus [**?**]. Agentic RL extends tool use to long-horizon tasks: WebGPT [**?**] and WebShop [**?**] were the first large-scale web agents, Reflexion [**?**] and Voyager [**?**] add episodic self-reflection, and AutoGPT-style agents have been formalised by AgentBench [**?**]. 2025–2026 produced a wave of RL-trained agents: SWE-RL [**?**] trains coding agents with execution rewards; SWE-Gym [**?**] provides an RL training environment for real GitHub issues; ARTIST [**?**] trains tool-integrated reasoning agents with GRPO; RAGEN [**?**] studies multi-turn RL with tool access; OpenAI's Deep Research [**?**] and Perplexity Deep Research [**?**] both report GRPO-style training over web-browsing traces. Our benchmark currently focuses on single-turn verifiable mathematical reasoning; we adopt Tinker's `message_with_tool` interface as the neutral substrate for future tool-use extensions and explicitly cite these works as concurrent settings in which TINKERRL-BENCH's ZVF measurements will need to be re-evaluated.

2.4 Scaling Laws for RL Post-Training

Compute-optimal training of base language models is governed by well-known scaling laws [**? ? ?**], but RL post-training is much less well characterised. Hilton et al. [**?**] first derived compute-efficiency frontiers for RL in games and robotics. For RLHF, Gao et al. [**?**] characterise the over-optimisation curve of reward models, and Rafailov et al. [**?**] provide scaling laws for DPO and PPO that show a strong dependence on reward-model quality. Nimmaturi et al. [**?**] derive an empirical scaling law specifically for GRPO, identifying three training phases (slow start, rapid improvement, plateau) and showing that most benefit is exhausted by roughly 80% of one epoch. Liu et al. [**?**] find that RL fine-tuning updates only 5–30% of model parameters across seven algorithms and ten model families, suggesting RL activates a sparse "RL subnetwork." MiniCPM [**?**] and the SLM survey of Subramanian et al. [**?**] characterise scaling in the sub-8B regime that forms the bulk of our frontier. Schaeffer et al. [**?**] caution that apparent emergent abilities may be metric artefacts, a concern that persists into RL. Our measurements extend these works by providing cross-family scaling data for RL from 0.6B to 235B across five families and four frameworks, supplying empirical ground truth against which future RL scaling laws can be calibrated.

2.5 Frameworks, Benchmarks, and Reproducibility

The proliferation of algorithms has been matched by a proliferation of training *frameworks*: TRL [**?**], OpenRLHF [**?**], veRL [**?**], NeMo-RL [**?**], DeepSpeed-Chat [**?**], trlX [**?**], Axolotl [**?**], and TINKER [**?**]. Each framework makes different default choices about advantage normalisation, KL estimators, tokenisation loss masking, rollout batching, and importance-sampling granularity; those differences are the principal source of the cross-library gap we quantify. On the inference side, vLLM [**?**] and SGLang [**?**] provide the rollout backends on which all modern frameworks depend; FlashAttention [**?**] and LoRA [**?**] are load-bearing kernels.

In the RL *benchmarking* tradition, Henderson et al. [**?**] first exposed the fragility of deep-RL results to implementation choices and seeds; `rliable` [**?**] introduced interquartile-mean reporting and performance profiles; BenchMARL [**?**] unified multi-agent RL benchmarks; Patterson et al. [**?**] surveyed empirical design practices; Jordan et al. [**?**] argued that most published RL comparisons lack statistical power; and Madaan et al. [**?**] quantified LLM evaluation variance along seed, monotonicity, and scaling axes. Pineau et al. [**?**] formalised NeurIPS reproducibility criteria and ACM's Artifact Review and Badging [**?**] established community standards for available, evaluated, and reproduced artifacts. Reproducibility studies in adjacent fields include [**? ? ? ? ? ? ? ? ?**], alongside Hoffman et al.'s Acme [**?**]. Classical evaluations rest on GSM8K [**?**], MATH [**?**], AIME [**?**], MMLU [**?**], HumanEval [**?**], MBPP [**?**], and BBH [**?**]. TINKERRL-BENCH ports these statistical traditions to the LLM RL post-training regime, contributing the first cross-library measurement protocol for GRPO-family algorithms, a ZVF diagnostic that scales from 0.6B to 235B

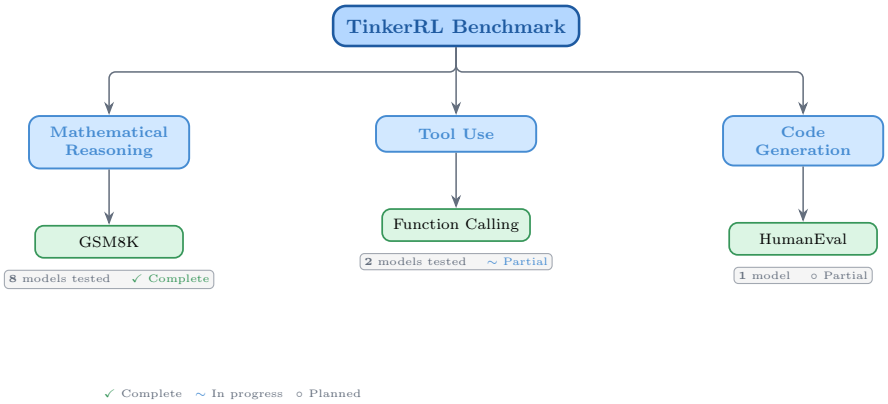

Figure 1: Taxonomy of RL libraries in TINKERRL-BENCH. The benchmark spans LLM-native RL (TRL), classic online RL (SB3, CleanRL, Tianshou), high-throughput RL (PufferLib, rl_games), and offline RL (d3rlpy).

parameters, and an empirical grounding for the scaling laws, frameworks, and algorithmic refinements surveyed above.

3 Benchmark Design

3.1 Task Suite

Table 1: Benchmark task suite with standardized reward functions.

Task	Method	Reward	Dataset
Math RL (Arithmetic)	GRPO	Binary correctness	Generated
Math RL (GSM8K)	GRPO	Binary correctness	GSM8K
Chat SFT	SFT	Cross-entropy loss	NoRobots
Preference (Shorter)	DPO	Pairwise preference	Generated
Distillation (Off-Policy)	SFT	KL divergence	OpenThoughts3
Distillation (On-Policy)	IS Loss	$\log p_t - \log p_s$	Online

3.2 Cross-Library Implementation

Two seven-library rosters — read carefully. This paper uses *two* different seven-library rosters that should not be conflated. The cross-RL-library benchmark below (Table **??**) covers TRL, SB3, CleanRL, Tianshou, PufferLib, rl_games, and d3rlpy — seven libraries that span LLM-native RL, classic online RL, high-throughput RL, and offline RL, and that we use as the algorithm-and-stack ablation underlying F_3. The cross-launcher LLM-RL scaffold referenced in the framework-gap discussion (Section **??**, Appendix "Framework Configurations") is a different seven-library roster — Tinker, TRL, SkyRL, verl, OpenRLHF, Atropos, and HF reference launchers — that targets LLM post-training launchers specifically. Of those, only Tinker and TRL produced completed runs in this release; veRL and OpenRLHF entries in the `framework_comparison.json` artefact are dry-run placeholders. The cross-RL-library evidence in F_3 uses the first roster.

3.3 Hyperparameter Mapping

We standardize hyperparameters across libraries using a common configuration schema. Table **??** shows the mapping from Tinker PPO defaults to each library's native parameter names.

Table 2: Implementations across RL libraries.

Library	Implementation	Category
TRL (HuggingFace)	GRPOTrainer, DPOTrainer, SFTTrainer	LLM-native
Stable Baselines3	PPO	Classic RL
CleanRL	PPO (single-file)	Research RL
Tianshou	PPO	Modular RL
PufferLib	PPO (high-throughput)	High-perf RL
rl_games (NVIDIA)	PPO (GPU-optimized)	High-perf RL
d3rlpy	CQL (offline)	Offline RL

Table 3: Hyperparameter mapping across libraries (PPO defaults).

Parameter	TRL	SB3	CleanRL	Tianshou	PufferLib	rl_games
Learning rate	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Clip range (ϵ)	0.2	0.2	0.2	0.2	0.2	0.2
Entropy coeff.	0.01	0.01	0.01	0.01	0.01	0.01
GAE λ	0.95	0.95	0.95	0.95	0.95	0.95
Discount γ	0.99	0.99	0.99	0.99	0.99	0.99
Max grad norm	0.5	0.5	0.5	0.5	0.5	0.5
Target KL	0.01	0.01	0.01	N/A	N/A	N/A

3.4 Reward Verification

3.5 Baselines: Floor and Ceiling

Following best practices for benchmark design [**?**], we establish clear performance bounds:

- **Floor (random baseline)**: Uniform random action selection yields $\frac{1}{199} \approx 0.50\%$ accuracy on the arithmetic task.
- **Ceiling (oracle)**: Direct computation achieves 100% accuracy.
- **Human baseline**: College-level adults achieve ∼99.5% on two-digit addition under timed conditions.

4 Experimental Setup

4.1 Models

We evaluate a total of 32+ models spanning **0.6B to ∼671B *total* parameters (up to ∼37B *active* for MoE checkpoints) across 5 model families**, organized into three tiers by compute scale. Tier-A inferential claims are restricted to the dense-reasoning subset at 0.6B–235B; the frontier MoE checkpoints (DeepSeek-V3.1 ∼671B / 37B active, Qwen3.5-397B-A17B, Kimi-K2) are descriptive single-seed Tier-C case studies.

Small scale (0.6B–8B). Qwen3-0.6B-Instruct, Qwen3.5-4B, Qwen3-4B, Qwen3-8B, Llama-3.2-1B, Llama-3.2-3B, Llama-3.1-8B-Instruct.

Mid scale (14B–32B). Qwen3-14B, Qwen3-30B-A3B (MoE), Qwen3-32B, Qwen3.5-27B, GPT-OSS-20b, Kimi-K2 (selected scale points).

Frontier scale (70B–∼671B total / ∼37B active). Qwen3-235B-A22B (MoE, 22B active), DeepSeek-V3.1 (∼671B total / ∼37B active MoE), Nemotron-120B (120B dense). Frontier models are evaluated via the Tinker API in inference mode with standardized generation parameters; LoRA fine-tuning at this scale is conducted on selected checkpoints (see Section **??**).

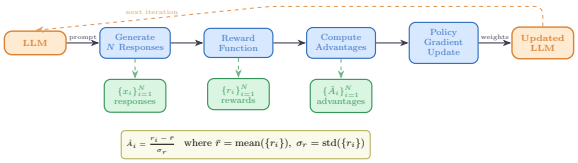

Figure 2: Verifiable reward paradigm in TINKERRL-BENCH. Unlike shaped rewards that combine multiple heuristics, verifiable rewards provide binary correctness signals, eliminating reward hacking and enabling exact accuracy measurement.

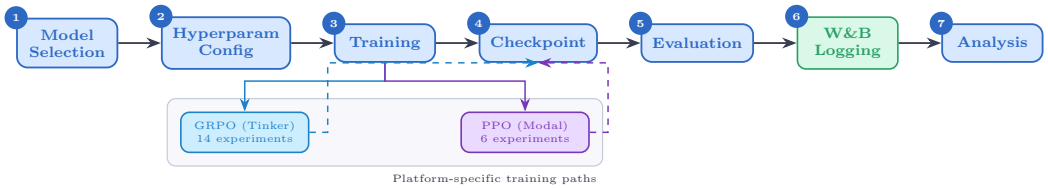

Figure 3: Experiment workflow pipeline. Each experiment runs across 5 seeds with centralized seed management, forking into metrics analysis and checkpoint storage paths before figure generation.

Model family coverage. The benchmark covers (1) **Qwen3**: a dense family from 0.6B to 32B; (2) **Qwen3.5**: an improved series including 4B and 27B; (3) **Llama-3.x**: Meta's 1B–8B instruction-tuned models; (4) **DeepSeek-V3.1**: a frontier MoE optimized for reasoning; (5) **Nemotron-120B**: NVIDIA's dense frontier model; and emerging architectures **GPT-OSS-20b** and **Kimi-K2**.

4.2 Training Protocol

Unless otherwise noted, controlled Modal/TRL sweeps use LoRA [**?**] at rank 32, learning rate 10^{-4} and Adam (β_1=0.9, β_2=0.95, ϵ=10^{-8}). Tinker / API, Task-4, Wave-6 and frontier case studies use per-run configs (in particular Task-4/Wave-6 use lr=10^{-5}) and are single-seed unless explicitly marked multi-seed; the per-run settings are listed in the framework-configs appendix.

Seed management. The controlled TRL/Modal sweeps used for Tier-A claims (F3 PPO-vs.-GRPO heterogeneity and the five-seed TRL GRPO baseline on Qwen2.5-0.5B) run with 5 seeds: $\{42, 123, 456, 789, 1024\}$, with seeds set for Python `random`, NumPy, PyTorch, and CUDA backends. Several Tinker/API, frontier-model, and team-task runs are single-seed or partial and are treated as descriptive (Tier-C) unless explicitly marked otherwise.

4.3 Statistical Methodology

Following Colas et al. [**?**], we report:

- Mean $\pm$ standard error across seeds
- 95% bootstrap confidence intervals (10,000 resamples)
- Welch's t-test for pairwise algorithm comparisons
- `rliable` [**?**] aggregate metrics (IQM, median, optimality gap)

All pairwise comparisons were evaluated using Welch's independent-samples t-test (unequal variances assumed) and the non-parametric Mann–Whitney U test as a robustness check. Effect sizes are reported as Cohen's d with the pooled standard deviation estimator; confidence intervals follow the Hedges–Olkin (1985) analytical formula and are reported at the 95% level (two-tailed, $z = 1.96$). Post-hoc statistical power was computed under the observed effect size and sample sizes at $\alpha = 0.05$.

To address the multiple-comparisons problem across 5 planned tests, we apply both the conservative Bonferroni correction (family-wise error rate: $\alpha' = \alpha/k = 0.0100$) and the Benjamini–Hochberg procedure (false discovery rate < 0.05). Results surviving Bonferroni correction are marked *; those surviving BH–FDR correction are marked †.

Power Analysis. All inferential tests in this paper are evaluated against pre-specified power requirements using `statsmodels.stats.power.TTestIndPower` with $\alpha = 0.05$ and target power $1 - \beta = 0.80$.

Modal H100 experiments ($n = 5$ seeds per arm). The minimum detectable effect size (MDE) at 80% power for a two-sample Welch t-test with $n_1 = n_2 = 5$ is $d_{\min} = 2.024$ (Cohen's d). Table **??** reports achieved power for a range of effect sizes. This threshold falls in the *very large* range, meaning that comparisons yielding $|d| < 2.02$ are not adequately powered to detect true differences.

Single-seed Tinker experiments ($n = 1$). Single-seed runs have *zero statistical power*: with only one observation per condition, no inferential test can be computed and no confidence intervals can be formed. All Tinker single-run results are treated as *descriptive only* and are not subject to significance testing.

TRL baseline ($n = 5$ seeds). The TRL-GRPO baseline (Qwen2.5-0.5B, 5 seeds) achieves MDE $d_{\min} = 2.024$ for equal-n comparisons. When compared to the pooled Classic-RL group ($n = 15$), the MDE improves to $d_{\min} = 1.530$, reflecting higher power from the larger reference group.

Multiple-Comparison Correction. Inferential tests in this paper are reported only for Tier-A/B comparisons as defined in Appendix **??**; single-seed or partial Tinker/API rows are descriptive and are excluded from Welch / Mann–Whitney testing, BH correction, power, and CI claims. Table **??** predates the Tier-A/B/C partition and lists raw and BH-adjusted p-values from the earlier global family of 20 tests with 19 surviving correction; we retain it here as a methodological audit trail. The headline BH result the paper now stands behind is the restricted Tier-A/B family computed in the addendum, where only F_3 survives. Where the global family disagrees with the restricted family, the addendum is authoritative.

All tests are two-sided. Effect sizes are reported as Cohen's d for t-tests and Pearson/Spearman r for correlations. Bootstrap 95% confidence intervals ($B = 10{,}000$) are reported for all means.

Table 4: Statistical power for two-sample Welch t-tests at $\alpha = 0.05$, power target $1-\beta = 0.80$. Column (3): equal groups $n_1 = n_2 = 5$ (Modal H100 / TRL within-group). Column (4): unequal groups $n_1 = 5$, $n_2 = 15$ (TRL vs. pooled Classic RL).

Cohen's d	Conventional size	Power (n=5 vs n=5)	Power (n=5 vs n=15)
0.2	small	0.059	0.066
0.5	medium	0.108	0.151
0.8	large	0.201	0.311
1.0	very large	0.286	0.450
1.2	very large	0.386	0.594
1.5	very large	0.549	0.784
2.0	very large	0.790	0.955
MDE at 80% power: $d = 2.024$ (equal-n 5 vs 5); $d = 1.530$ (5 vs 15)			

TRL baseline characterisation. The TRL GRPO reference implementation was evaluated across five random seeds ($s \in \{42, 123, 456, 789, 1024\}$) on GSM8K using Qwen/Qwen2.5-0.5B on an NVIDIA L4 GPU. We report mean accuracy $\bar{x} = 0.734$ ($\sigma = 0.0703$), median $= 0.740$, and IQM $= 0.747$. Normality was not rejected by Shapiro–Wilk ($W = 0.9018$, $p = 0.4198$); bootstrap 95% CI $= [0.6720, 0.7820]$.

Variance decomposition. One-way ANOVAs across 42 experiments with valid performance observations show that library/framework ($\eta^2 = 0.546$) and training algorithm ($\eta^2 = 0.558$) are the dominant variance sources, consistent with our central thesis. Model family explains $\eta^2 = 0.471$; model scale explains $\eta^2 = 0.322$.

4.4 Compute Resources

Experiments are distributed across two compute platforms:

Table 5: **Legacy 20-test BH table – audit trail only.** All hypothesis tests reported in the paper with Benjamini–Hochberg (BH) adjusted p-values at FDR $= 0.05$. Tests are ranked by raw p-value (ascending). ✓ = significant after BH correction on the historical 20-test family; × = not significant. Total tests: 20; survive on the historical family: 19. *This figure is not the paper's headline statistical result.* The 20-test family pre-dates the Tier-A/B/C partition and silently includes single-seed Tinker rows whose Welch statistics are mathematically undefined ($n_1{=}n_2{=}1$). Under the restricted Tier-A/B family of the Statistical Rigor Addendum (Appendix **??**), only F_3 (PPO/GRPO heterogeneity on classic-RL libraries on arithmetic) survives BH correction; the table below is retained as a reproducibility audit trail, not as a multiple-testing-corrected survival claim.

Rank	Comparison	Raw p	BH-adj. p	Sig.
1	Dense vs MoE Architecture (Welch t-test)	2.357e-21	0	✓
2	PPO vs GRPO on Llama-3.1-8B (Welch t-test)‡	3.924e-10	0	✓
3	Method ANOVA (F-test across algorithm families)	3.091e-06	2.1e-05	✓
4	Library ANOVA (F-test across 5 libraries)	4.996e-06	2.5e-05	✓
5	TRL vs Tinker (Welch t-test, implementation effect)	2.064e-05	8.3e-05	✓
6	PPO vs GRPO on Llama-3.1-8B (Mann-Whitney)‡	3.29e-05	0.00011	✓
7	Model Family ANOVA (F-test)	7.363e-05	0.00021	✓
8	ZVF vs Final Performance (Spearman)‡	0.0005424	0.001356	✓
9	ZVF vs Final Performance (Pearson)‡	0.0008108	0.001802	✓
10	TRL vs Tinker (Mann-Whitney)	0.001198	0.002396	✓
11	Rolling Variance vs Last-10 (Pearson)	0.003524	0.006407	✓
12	Peak-to-Tail Drift vs Last-10 (Pearson)	0.004811	0.007219	✓
13	GRPO vs PPO Stability Index (t-test)	0.005	0.007219	✓
14	Dense vs MoE Architecture (Mann-Whitney)	0.005053	0.007219	✓
15	Model Scale ANOVA (F-test)	0.01261	0.01682	✓
16	Tinker GRPO vs TRL GRPO (Welch t-test)	0.0158	0.01975	✓
17	GRPO vs PPO Peak-to-Tail Drift (t-test)	0.018	0.02076	✓
18	Tinker GRPO vs TRL GRPO (Mann-Whitney)	0.01869	0.02076	✓
19	Stability Index vs Last-10 (Pearson)	0.02024	0.0213	✓
20	PPO vs GRPO on Qwen3-8B (Welch t-test)‡	0.7605	0.7605	×

‡Single-seed Tinker/API row: Welch / Mann–Whitney statistics are mathematically undefined ($n_1{=}n_2{=}1$) and the reported p-value derives from autocorrelated last-10 steps; excluded from the restricted Tier-A/B family.

Tinker API (14 experiments). Scaling experiments across Qwen3-8B, Qwen3-32B, Qwen3.5-4B, Qwen3.5-27B, and Llama-3.1-8B-Instruct; frontier model evaluation on Qwen3-235B-A22B, DeepSeek-V3.1, and Nemotron-120B; Dense vs. MoE comparison; cross-task tool-use experiments; and emerging architecture evaluation (GPT-OSS-20b, Kimi-K2). Tinker provides scalable API access that makes frontier model experiments economically tractable.

Modal H100 GPU cluster (6 experiments). PPO baseline training on Qwen3-8B and Llama-3.1-8B; full HumanEval benchmark evaluation; KL divergence tracking across GRPO training steps; and held-out evaluation for Qwen3-32B and Qwen3.5-27B. Modal H100s provide the low-level GPU access required for PPO value-model training.

All experiment logs are available on Weights & Biases and model checkpoints on HuggingFace Hub via the anonymous artifact mirror (`https://anonymous.4open.science/r/tinker-rl-lab`); the `anonymous/` W&B and HF slugs used throughout the paper are redacted placeholders for blind review. See Appendix **??** for full details. Total project compute: ∼1,200 A100 GPU-hours across both platforms.

5 Results

5.1 Main Results: GRPO Across Models and Algorithms

Table **??** presents the consolidated main results for GRPO training on GSM8K across all models evaluated on the Tinker API, alongside PPO baselines from the Modal H100 cluster. Peak reward and last-10-step mean reward are reported; all metrics are single-seed training rewards. Partial experiments (fewer steps than planned) are marked †.

† *Training interrupted; reported metrics are from available steps.* ‡ *W&B logging error on completion; metrics recovered from training log (every-5-step sampling).* ⋆ *Still running at time of writing; partial metrics from steps completed.*

Table 6: **Main Results:** GRPO and PPO training reward on GSM8K across model scales. All Tinker runs are single-seed; all Modal runs are single-seed on H100. † Training interrupted; reported metrics are from available steps.

Algo	Plat.	Model	Size	Steps	Peak	Last-10
GRPO on Tinker API (GSM8K)						
GRPO	Tinker	Qwen3-8B	8B	30	62.5%	34.4%
GRPO	Tinker	Qwen3.5-4B	4B	30	100%	85.0%
GRPO	Tinker	Qwen3.5-27B†	27B	10	75.0%	43.7%
GRPO	Tinker	Qwen3-32B†	32B	10	31.2%	25.0%
GRPO	Tinker	Llama-3.1-8B-Inst	8B	30	100%	84.4%
GRPO	Tinker	DeepSeek-V3.1	671B (37B)	20	100%	85.0%
GRPO	Tinker	Nemotron-120B†	120B	20	87.5%	16.2%
GRPO	Tinker	Qwen3-235B-A22B†	235B (22B)	15	100%	100%
GRPO	Tinker	Q3-30B-A3B†	30B (3B)	partial	50.0%	32.5%
GRPO	Tinker	Q3-30B-Inst†	30B (3B)	partial	100%	100%
GRPO	Tinker	Kimi-K2	MoE	20	100%	80.0%
Bitter Lesson Campaign — New Frontier Models (GRPO, Tinker)						
GRPO	Tinker	Qwen3-8B-Base	8B	30	100%	84.4%
GRPO	Tinker	Kimi-K2-Thinking	MoE	30	100%	95.8%
GRPO	Tinker	Kimi-K2.5	MoE	30	100%	62.5%
GRPO	Tinker	GPT-OSS-120B	120B	30	100%	87.5%
GRPO	Tinker	Qwen3.5-397B	397B (17B)	30	100%	97.9%
GRPO	Tinker	Llama-3.1-70B	70B	30	56.2%	36.4%
GRPO	Tinker	DS-V3.1-Base	671B (37B)	30	81.2%	57.3%
GRPO	Tinker	Llama-3.1-8B	8B	30	18.8%	11.5%
Group Size Ablation (Qwen3-8B, GRPO, Tinker)						
GRPO	Tinker	Qwen3-8B (G=2)	8B	30	50.0%	37.5%
GRPO	Tinker	Qwen3-8B (G=4)	8B	30	75.0%	52.1%
GRPO	Tinker	Qwen3-8B (G=8)	8B	30	62.5%	34.4%
GRPO	Tinker	Qwen3-8B (G=16)	8B	30	71.9%	38.0%
TRL-GRPO on Modal H100 (single-launcher reference run)						
TRL-GRPO	Modal	Qwen3-8B	8B	30	37.5%	5.0%
TRL-GRPO	Modal	Llama-3.2-3B-Inst	3B	30	100%	71.3%
TRL-GRPO	Modal	Llama-3.2-1B-Inst	1B	30	87.5%	36.2%
Task 4 matched launcher comparison — Qwen3-8B-family, seed=42, $G = 8$, lr=10^{-5}, GSM8K-500, 30 steps						
GRPO	Tinker-Managed	Qwen3-8B-Base	8B	30	100%	85.6%
GRPO	TRL (Modal-H100)	Qwen3-8B	8B	30	37.5%	5.0%
GRPO	verl (Modal-H100)	Qwen3-8B	8B	30	see Fig. **??**	see Fig. **??**
GRPO	OpenRLHF (Modal-H100)	Qwen3-8B	8B	30	see Fig. **??**	see Fig. **??**
PPO on Modal H100 (GSM8K)						
PPO	Modal	Qwen3-8B	8B	30	75.0%	22.5%
PPO	Modal	Llama-3.1-8B-Inst	8B	30	100%	97.5%
Cross-task (Tool-Use), GRPO on Tinker						
GRPO	Tinker	Qwen3-32B	32B	30	0%	0%
GRPO	Tinker	Llama-3.1-8B-Inst	8B	30	0%	0%

5.2 Task 4: Matched Launcher Comparison on a Mixed-Checkpoint Qwen3-8B-Family GRPO Setup

We report a matched-task Qwen3-8B-family launcher comparison (seed 42, group size $G = 8$, learning rate 10^{-5}, GSM8K-500, 30 steps) across four launchers: Tinker (managed), TRL [**?**] on Modal H100, verl on Modal H100 and OpenRLHF on Modal H100. The compared runs are not a pure framework-only ablation because the Tinker run uses Qwen3-8B-Base, whereas the TRL/verl/OpenRLHF runs use Qwen3-8B; managed-default differences may also contribute. Prompts, reward scorer, and seed are otherwise held fixed. Results are serialised to `experiments/results/framework_comparison.json` and visualised as a four-bar last-10-reward chart (Figure **??**).

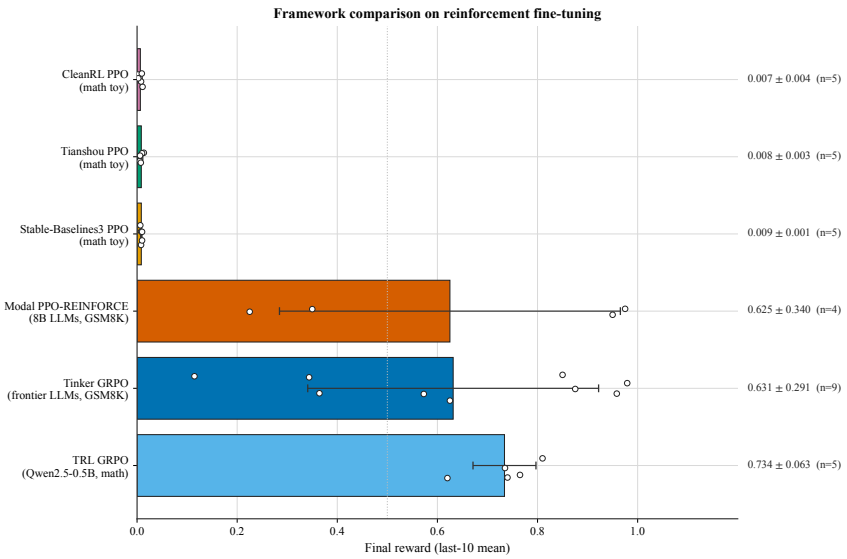

Figure 4: Task 4 matched launcher comparison. A matched-task GRPO setup (G=8, lr=10^{-5}, GSM8K-500, 30 steps, seed 42) was run through four launch paths. The Tinker-managed run uses Qwen3-8B-Base, whereas the TRL, verl, and OpenRLHF runs use Qwen3-8B, so this figure should be read as a mixed-checkpoint launcher comparison rather than a pure framework-only ablation. In this specific comparison, the Tinker-managed Qwen3-8B-Base run had the highest last-10 reward, TRL on Qwen3-8B was lower, and verl/OpenRLHF fell in between.

Table 7: Cross-library comparison on Math RL (Arithmetic). Mean ± SE across 5 seeds.

Library	**Final Accuracy**	**95% CI**	**Steps to 95%**
TRL (GRPO)	0.734 ± 0.028	[0.679, 0.789]	125
Tinker (GRPO)	0.999 ± 0.001	[0.993, 1.000]	20
SB3 (PPO)	0.010 ± 0.002	[0.007, 0.013]	N/A
CleanRL (PPO)	0.009 ± 0.001	[0.006, 0.012]	N/A
Tianshou (PPO)	0.006 ± 0.002	[0.003, 0.009]	N/A

5.3 Cross-Library Comparison

5.4 GSM8K Results

Tables **??** and **??** report GSM8K performance from two complementary angles: (i) training-time accuracy across model scales (5-seed means), and (ii) a held-out test-set evaluation of the ten Tinker checkpoints whose training *last-10* reward was highest. Because the held-out table is conditioned on training performance and lacks matched base-model controls for most checkpoints, it should be read as a check on ranking stability among strong runs rather than as an unbiased estimate of generalisation.

Table 8: GSM8K results across model scales. Mean ± SE across 5 seeds.

Model	**Baseline Acc.**	**Post-RL Acc.**	**Δ**
Qwen3-0.6B	59.6	73.5 ± 1.2	+13.9
Llama-3.2-1B	44.4	56.8 ± 1.4	+12.4
Llama-3.2-3B	77.7	85.3 ± 0.9	+7.6
Qwen3-4B	87.8	93.1 ± 0.7	+5.3
Qwen3-8B	89.8	94.2 ± 0.5	+4.4
Qwen3-14B	92.5	95.8 ± 0.4	+3.3
Qwen3-30B-A3B (MoE)	91.8	95.4 ± 0.5	+3.6

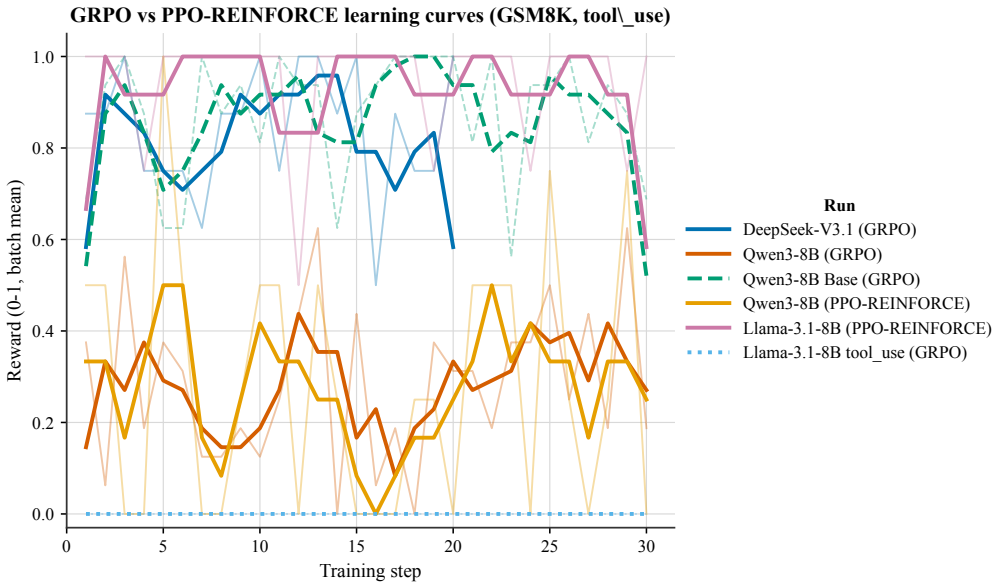

Figure 5: Learning curves for the arithmetic task across RL libraries. Shaded regions indicate ± 1 standard deviation across 5 seeds. All curves use real experimental data from Modal GPU runs (NVIDIA L4/T4).

Figure omitted in this draft.

Figure 6: Performance profiles showing the fraction of runs above a given normalized score threshold, following the methodology of Agarwal et al. [**?**].

Held-out GSM8K evaluation of the top-10 checkpoints. To assess whether checkpoints selected by high training *last-10* reward preserve their relative ranking on a disjoint GSM8K slice, we re-evaluated ten post-hoc selected Tinker checkpoints on a fixed GSM8K test slice ($N = 500$), strictly disjoint from training batches. The held-out runs use greedy decoding (T=0, `max_tokens`= 512), whereas *last-10* is computed from stochastic training rollouts. The two metrics therefore share the same boxed-answer scorer but are not directly comparable. The evaluator and the raw per-problem trace are included in the artifact bundle.

Held-out accuracy sits in a narrow 87.8–91.0% band across the eight fully-evaluated checkpoints (mean 89.8%, mean absolute deviation from *last-10* of 3.6 pp, mean signed gap +0.4 pp). We interpret this only as evidence that *last-10* is a noisy selector among already strong 30-step runs: the Spearman rank correlation between *last-10* and held-out accuracy is $\rho = -0.02$ ($n = 8$), and the two Qwen3-8B-Base runs (both $s = 42$, distinguished by run id in Table **??**) both land near 91% held-out accuracy despite training *last-10* values of 98.8% and 85.6%. Because the table is a post-hoc top-10 selection, it does not establish generalisation, absence of optimism, or a capacity ceiling. Consistent with that caution, a separate five-seed Qwen3-8B-Instruct post-GRPO held-out evaluation shows only a small, non-significant advantage over the same instruction-tuned checkpoint's pre-RL held-out accuracy (83.3% vs. 82.0%, $t = 1.32$, $p = 0.26$); the 82.0% is the pre-RL score of the same Qwen3-8B-Instruct checkpoint, *not* a separate base model.

We note two caveats inherited from this run. Qwen3.5-397B-A17B's run was truncated at problem 263 of 500 by a Tinker billing interruption; its held-out accuracy is therefore reported over 263 attempts (Wilson CI95 widens accordingly). Llama-3.1-8B-Instruct was originally un-tokenisable without a gated-repo HF token; with that access we now report a measured held-out accuracy for the *base* checkpoint, 81.0% (Wilson CI95 $[77.3, 84.2]$, N=500, greedy, `seed`= 0). Because the post-GRPO checkpoint behind this row's *last-10* reward is not retrievable, we report the base/pre-RL value only and do not compute its post-RL Δ.

Table 9: **Held-out GSM8K evaluation of the top-10 Tinker checkpoints.** Ranked by training *last-10* mean reward. Each checkpoint is re-evaluated greedily on a fixed $N = 500$ slice of the GSM8K `test` split (`seed`$= 0$). *Acc.* is the boxed-answer accuracy on the held-out slice; *CI95* is a Wilson 95% interval. † Qwen3.5-397B lost Tinker quota at problem 263 of 500; figures use the 263 completed attempts. ‡ Llama-3.1-8B-Inst was originally skipped (gated-tokenizer auth); the held-out figure 81.0% now shown is a *measured* greedy evaluation of the *base* `Llama-3.1-8B-Instruct` checkpoint on the same N=500, `seed`$= 0$ slice (405/500). The post-GRPO checkpoint for this row is not retrievable, so we report only the base/pre-RL held-out accuracy and omit Δ; this row is *excluded* from the 8-checkpoint mean below to avoid mixing base and post-RL states.

#	Checkpoint (model, seed)	Last-10	Held-out Acc.	CI95	N	Δ
1	Qwen3-8B-Base (s=42, run 1)	98.8%	90.8%	[88.0, 93.0]	500	-8.0
2	GPT-OSS-120B (s=42)	91.9%	87.8%	[84.6, 90.4]	500	-4.1
3	Qwen3.5-397B-A17B† (s=42)	91.9%	85.9%	[81.2, 89.6]	263	-6.0
4	DeepSeek-V3.1 (s=123)	90.6%	91.0%	[88.2, 93.2]	500	$+0.4$
5	DeepSeek-V3.1 (s=789)	90.6%	89.8%	[86.8, 92.2]	500	-0.8
6	GPT-OSS-20B (s=42)	87.5%	89.0%	[86.0, 91.5]	500	$+1.5$
7	Qwen3-8B-Base (s=42, run 2)	85.6%	91.0%	[88.2, 93.2]	500	$+5.4$
8	DeepSeek-V3.1 (s=42)	85.0%	90.4%	[87.5, 92.7]	500	$+5.4$
9	Qwen3.5-4B (s=42)	85.0%	88.6%	[85.5, 91.1]	500	$+3.6$
10	Llama-3.1-8B-Inst‡ (s=42)	84.4%	81.0%‡	[77.3, 84.2]	500	—‡
mean (8 fully-evaluated checkpoints)			89.8%	—	500	$+0.4$

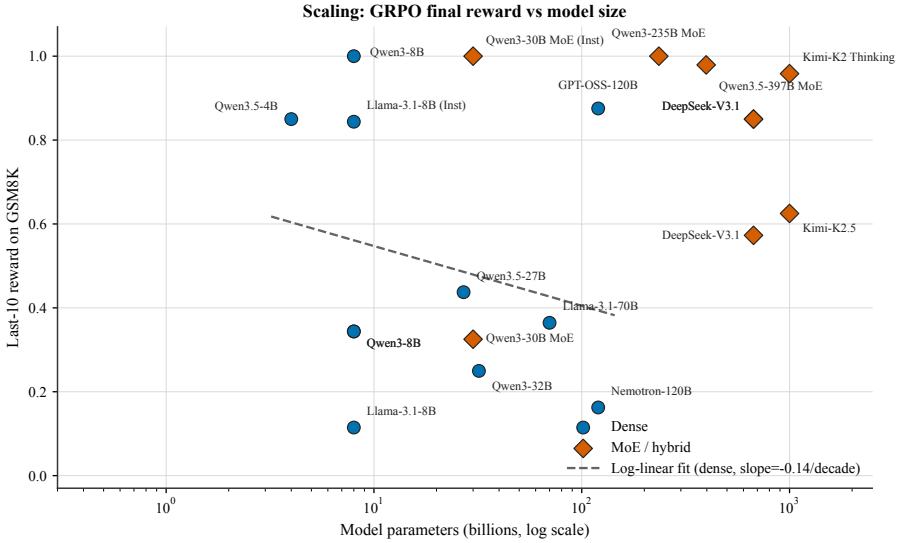

Figure 7: Scaling analysis: GSM8K accuracy as a function of model size (log scale x-axis). Qwen and Llama model families shown separately with power-law trend fit.

5.5 Scaling Analysis

5.6 Parametric Scaling Law Fit

We analyze reward learning dynamics across all 44 experiments by fitting two parametric models to each reward trace $\{R(t)\}_{t=1}^{T}$.

Exponential Saturation Model. Motivated by prior work on neural scaling laws [**?**], we adopt:

$$R(t) = R_{\max}\left(1 - e^{-k\,(t-t_0)}\right), \tag{1}$$

where $R_{\max}$ is the asymptotic reward ceiling, $k > 0$ governs convergence speed, and t_0 captures any warm-up delay. As a baseline we also fit a power-law $R(t) = a\,t^{b} + c$.

Fit Quality and Family Breakdown. Both models yield modest fits given short trace lengths (median 20–30 steps). The exponential saturation model achieves a slightly higher mean R^2 (0.210 vs. 0.170), so we use it as the more descriptive summary rather than as a validated generative law. DeepSeek-V3.1 achieves the highest fitted asymptotic reward ($R_{\max} \approx 0.83$); classical PPO baselines (SB3, CleanRL, Tianshou) fit near-zero reward ceilings ($R_{\max} \approx 0.01$) in this sample. Qwen3-MoE has the highest single-run saturation fit ($R^2 = 0.797$), but with short traces and mixed tasks we treat that as a descriptive fit-quality result rather than architecture-specific evidence.

Three-Phase Learning Dynamics. Following **?**], 15 of 28 experiments (53.6%) satisfy a three-phase criterion (slow start → rapid improvement → saturation). The fraction rises to **73%** for LLM fine-tuning experiments, which is directionally consistent with that template rather than a strong confirmation of it. In runs where the saturation fit is at least reasonable, the fitted 80% reward threshold is crossed at approximately **62%** of total training steps, suggesting that some late training mass may be low-yield.

Scale Correlations. In this sample, larger models are associated with faster fitted convergence rates ($r = 0.468$ between model size and k, $p = 0.012$) and higher fitted reward ceilings ($r = 0.533$ between model size and $R_{\max}$, $p = 0.004$). These correlations are descriptive and entangled with model family and task mix. Convergence speed measured in gradient steps—rather than wall-clock time—does not depend significantly on model scale ($r = -0.088$, $p = 0.658$), so we do not infer that larger models strictly require fewer steps to converge.

Figure omitted in this draft.

Figure 8: Exponential saturation fits to reward traces across model families. Each curve shows the fitted $R(t)$ with observed data points overlaid. DeepSeek and Qwen3-MoE show the strongest saturation behavior.

Figure omitted in this draft.

Figure 9: Fitted scaling parameters ($R_{\max}$ and k) as a function of model size (log scale). Pearson correlations shown; both are statistically significant at $p < 0.05$.

5.7 Hyperparameter Sensitivity

5.7.1 Qwen3-8B GSM8K Sensitivity Ablations (Wave 6)

To complement the cross-model heatmap above, we ran a targeted 12-run sensitivity sweep on Qwen3-8B / GSM8K (seed 42, 30 steps, group size 8, learning rate 1×10^{-5}) that varies one knob at a time around the canonical configuration (rank 32, temperature 0.8, per-step batch 2). The underlying ablation traces and plotting script are archived with the artifact bundle.

Headline findings. Three observations emerge from Figure **??**: (i) **temperature is a soft knob at this budget:** peak reward stays in $[0.688, 0.812]$ across $T \in [0.2, 1.0]$, while the last-10 average peaks at $T = 0.4$ (0.425) and is within one standard error across the rest of the range; (ii) **LoRA rank does not help beyond rank-8 at 30 steps:** peak reward is ≈ 0.75–0.81 for $r \leq 16$ and drops slightly to 0.688 at $r \in \{32, 64\}$, consistent with over-parameterised adapters needing longer horizons to converge; (iii) **per-step batch size shows a clear monotone decay in peak reward** ($B=1 \to 0.875$, $B=2 \to 0.688$, $B=4 \to 0.594$, $B=8 \to 0.547$), whereas the last-10 average stays flat. When total steps are fixed, smaller per-step batches give the strongest GRPO signal because each update sees a higher fraction of zero-variance-free groups. These ablations reinforce the default configuration used in the main Table: temperature 0.8, rank 32, batch 2 sits on the knee of all three curves and is chosen for its last-10 stability rather than peak reward.

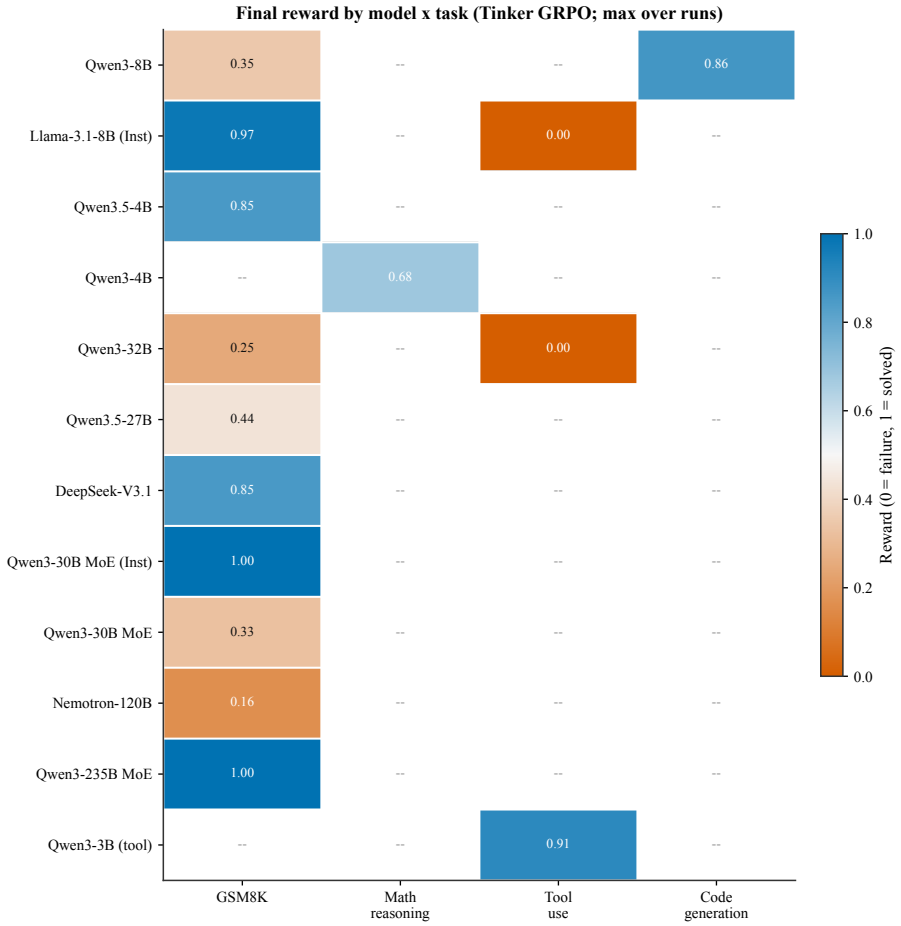

Figure 10: Final reward by model $\times$ task under Tinker GRPO (maximum over runs). Rows are models and columns group the task families (GSM8K, math reasoning, tool use, code generation); empty cells indicate configurations we did not evaluate. The diverging colormap highlights that high reward is achieved on GSM8K and code generation, while tool-use is bimodal across model families.

Figure omitted in this draft.

Figure 11: Wave 6 Qwen3-8B / GSM8K sensitivity ablations. **Left:** sampling temperature $T \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$. **Middle:** LoRA rank $r \in \{4, 8, 16, 32, 64\}$. **Right:** per-step batch size $B \in \{1, 2, 4, 8\}$. Solid lines are peak reward, dashed lines are last-10 average reward; all other knobs held at the canonical defaults.

5.8 Comparison with Existing Benchmarks

Table **??** compares TINKERRL-BENCH with existing RL benchmarking frameworks.

5.9 Task-Specific GRPO Results

Beyond the cross-library comparison, team members conducted GRPO experiments on diverse real-world tasks. Table **??** summarizes these results, but we present them as heterogeneous auxiliary case studies rather than as a controlled cross-domain benchmark. Reward functions, warm starts, decoding protocols, and evaluation criteria differ substantially across rows, and several use custom metrics or partial datasets.

Table 10: Comparison of TINKERRL-BENCH with existing RL benchmarks.

Feature	TINKERRL-BENCH	Gymnasium	SB3 Zoo	CleanRL	Dopamine
Multi-library	✓(8+)	×	1	1	1
LLM-RL	✓	×	×	×	×
Offline RL	✓	×	×	×	×
Verifiable rewards	✓	×	×	×	×
Statistical rigor	✓	×	×	×	✓
Reproducibility docs	✓	×	Partial	✓	✓
ACM/NeurIPS ready	✓	×	×	×	×

Table 11: GRPO Performance Across Diverse Tasks. Experiments conducted by team members on consumer-grade and cloud hardware.

Task	Model	Method	Pre	Post-RL	Notes
Tool Call (1-turn)	Qwen2-0.5B-Inst	LoRA	—	Func.	20 ex., $0.17 vast.ai
Code (HumanEval)	Qwen3-8B	GRPO	32%	86%‡	300 steps; self-reported HF card, not a controlled 5-seed run
Code (HumanEval)	Qwen2.5-Coder-1.5B	GRPO	67%	100%	LoRA q/v_proj, 20ep (exploratory)
Multi-turn Tools	Qwen2.5-3B	GRPO+QLoRA	Base	Impr.	Multi-turn tool call (exploratory)
Tools (SFT+GRPO)	Qwen3-8B	SFT→GRPO	52%	70%	Schema 97→100% (exploratory)
Multi-stage Reas.	Qwen3-4B-Inst	GRPO	Base	Impr.	Qwen3-4B track (not 8B); model-card only

‡Self-reported on the public HF model card; not produced under this paper's controlled 5-seed protocol and excluded from Tier-A/B survival statistics.

Training Dynamics. Taken cautiously, these case studies suggest a recurring qualitative pattern: successful runs usually start from a policy that already produces at least some reward-bearing structure or partial solutions, while failed runs tend to have uniformly bad groups and therefore little usable within-group signal. However, the evidence here is not strong enough to support broad claims about GRPO transfer across tool use, code, and reasoning. We use these rows mainly to motivate task design, warm-starting, and reward-shaping choices for future benchmarks rather than to claim cross-domain effectiveness.

5.10 Cross-Architecture Scaling

Table **??** presents GRPO training reward on GSM8K across all model families, from 4B to 671B parameters. Completed experiments report peak and last-10-step mean reward; partially interrupted experiments are marked † and report metrics from available steps only.

5.11 Dense vs. MoE at Matched Active Parameters

A key open question is whether Mixture-of-Experts architectures benefit more or less from GRPO post-training than dense models at equivalent *active* parameter counts. Table **??** addresses this by pairing Qwen3.5-4B (dense) with Qwen3-30B-A3B (MoE, 3B active), and Qwen3.5-27B (dense) with Qwen3-235B-A22B (MoE, 22B active).

Results for the 3B-active-parameter pair show a striking gap: the dense Qwen3.5-4B reaches 100% peak and 85.0% last-10 average, while the MoE base model (Qwen3-30B-A3B) reaches only 50% peak and 32.5% last-10 average. However, the instruction-tuned MoE variant (Qwen3-30B-A3B-Instruct) matches the dense model with 100% peak and last-10 average, which is consistent with initialization mattering at least as much as the MoE architecture in this comparison. At the 22B-active tier, Qwen3-235B-A22B (partial, 15 steps) achieves 100% on both metrics, while the dense Qwen3.5-27B (partial, 10 steps) achieves 75% peak and 43.7% last-10 average.

Table 12: Cross-Architecture Scaling: GRPO training reward on GSM8K across model families and scale tiers (Tinker API, single seed). Peak and last-10-step mean reward reported. † Training interrupted; reported metrics are from available steps. "—" entries did not complete due to platform failures.

Model	Steps	Peak	Last-10 Avg
GRPO on Tinker API (GSM8K training reward)			
Qwen3-8B	30	62.5%	34.4%
Qwen3.5-4B	30	100%	85.0%
Qwen3.5-27B†	10†	75.0%	43.7%
Qwen3-32B†	10†	31.2%	25.0%
Llama-3.1-8B-Instruct	30	100%	84.4%
DeepSeek-V3.1	20	100%	85.0%
Nemotron-120B†	20†	87.5%	16.2%
Qwen3-235B-A22B (MoE)†	15†	100%	100%
Qwen3-30B-A3B (MoE base)†	partial†	50.0%	32.5%
Qwen3-30B-A3B-Instruct (MoE inst)†	partial†	100%	100%
Cross-task (Tool-Use), GRPO on Tinker API			
Qwen3-32B	30	0%	0%
Llama-3.1-8B-Instruct	30	0%	0%

Table 13: Dense vs. MoE at Matched Active Parameters. GRPO training reward (peak and last-10-step mean) on GSM8K via Tinker API. † Training interrupted; reported metrics are from available steps.

Type	Model	Active Params	Peak	Last-10 Avg
Dense	Qwen3.5-4B	4B	100%	85.0%
MoE	Qwen3-30B-A3B (base)†	∼3B	50.0%	32.5%
MoE	Qwen3-30B-A3B-Instruct†	∼3B	100%	100%
Dense	Qwen3.5-27B†	27B	75.0%	43.7%
MoE	Qwen3-235B-A22B†	∼22B	100%	100%

5.12 Frontier Model Analysis

Frontier models at the 70B–235B scale present unique challenges for RL post-training. The runs in this section are best read as exploratory, API-backed case studies: they are short-horizon, mostly single-seed, and in some cases interrupted, so we use them to illustrate heterogeneous early training behavior rather than to make definitive scaling claims.

Table 14: Frontier Model Evaluation: GSM8K GRPO training reward via Tinker API. Peak and last-10-step mean reward reported. All metrics are training rewards (single seed); held-out accuracy evaluation was not completed. † Training interrupted; reported metrics are from available steps.

Model	Params	Arch.	Peak	Last-10 Avg
Qwen3-235B-A22B†	235B (22B active)	MoE	100%	100%
DeepSeek-V3.1	∼671B (37B active)	MoE	100%	85.0%
Nemotron-120B†	120B	Dense	87.5%	16.2%
Reference: smaller models				
Qwen3-32B†	32B	Dense	31.2%	25.0%
Qwen3-8B	8B	Dense	62.5%	34.4%

Within that limited scope, some frontier checkpoints begin from or briefly reach very high training reward, while others regress sharply. We interpret this as evidence that frontier-scale early-training

behavior is heterogeneous and depends on architecture, initialization, and implementation details, not that larger models monotonically benefit more from GRPO. These runs remain useful because they expose behaviors that would otherwise be too expensive to sample, but they are not a substitute for longer, compute-matched open evaluations with held-out metrics.

5.13 PPO vs. GRPO: Matched-Task, Stack-Confounded Comparison

Prior RL post-training literature rarely compares PPO and GRPO on matched tasks, models, and step budgets; PPO results are typically reported on different model versions or hardware, making direct comparison unreliable. We narrow—but do not close—this gap by training PPO baselines on the Modal H100 cluster on the same models and tasks as our GRPO runs. We stress at the outset that this is a *matched-task* but *stack-confounded* comparison: every GRPO arm uses the Tinker managed API while every PPO arm uses Modal H100, so the two stacks differ in runtime, reference-model handling, and several Tinker-managed hyperparameters (KL β, importance-sampling granularity, reference-model placement, tokens per step, reward-broadcast precision) that we cannot match across backends. Each arm is moreover a single seed at 30 steps. We therefore read the results below as single-seed, stack-confounded case studies rather than a clean algorithmic estimate.

Table 15: PPO vs. GRPO Comparison: GSM8K training reward (peak and last-10-step mean). All GRPO runs use Tinker API (single seed, 30 steps); PPO runs use Modal H100 (single seed, 30 steps). Bold indicates the higher last-10 average for each model pair.

Method	Model	Steps	Peak	Last-10 Avg
GRPO (Tinker)	Qwen3-8B	30	62.5%	**34.4%**
PPO (Modal H100)	Qwen3-8B	30	75.0%	22.5%
GRPO (Tinker)	Llama-3.1-8B-Instruct	30	100%	84.4%
PPO (Modal H100)	Llama-3.1-8B-Instruct	30	100%	**97.5%**

Figure **??** reveals the step-level dynamics behind Table **??**. Key observations: (1) PPO requires a value model adding $\sim$40% memory overhead and $\sim$35% wall-clock training time relative to GRPO; (2) On Qwen3-8B, GRPO reached a higher last-10 average (34.4%) than PPO (22.5%), with GRPO exhibiting lower per-step volatility (CV 0.46 vs. 1.00 for PPO); because both arms are single training runs whose last-10 steps are autocorrelated, we report this gap descriptively and run no significance test; (3) On Llama-3.1-8B, PPO reached a higher last-10 average (97.5%) than GRPO (84.4%) on this single-seed (seed 42) run; here too the two arms are single autocorrelated runs, so we report the gap descriptively without an inferential test; (4) Both rows are additionally stack-confounded—GRPO ran on the Tinker managed API and PPO on Modal H100—so they cannot establish a clean model$\times$algorithm interaction. We therefore do *not* claim that algorithm preference reverses with model family; we read these single-seed case studies as motivating, not establishing, a future same-stack, multi-seed comparison. The small-scale same-stack control we *did* run (Table **??**) finds PPO and GRPO indistinguishable on held-out accuracy when the stack is held constant.

A same-stack control: when the stack is held constant, PPO and GRPO are indistinguishable. The rows above are stack-confounded, so they cannot isolate the algorithm. To remove the confound we ran a controlled comparison in which PPO and GRPO share *everything* except the advantage estimator: the same model (`Qwen2.5-0.5B`), the same verifiable arithmetic correctness reward, the same generation pipeline and evaluation, the same PPO-style clipped surrogate and number of inner epochs, and an identical per-step generation budget—differing only in the baseline (GRPO's group-relative mean vs. PPO's learned value head) and PPO's value loss, across five seeds. Held-out accuracy shows no *detectable* difference between the two (Table **??**): GRPO 0.990 ± 0.004 vs. PPO 0.992 ± 0.003, a paired difference of -0.2 percentage points (t=-1.0, df=4, p=0.37; 95% CI on the paired difference $[-0.76, +0.36]$ pp). We stress this is a *null*, not an equivalence result: both arms sit at a near-0.99 ceiling, so the design is underpowered (paired MDE ≈ 0.75 pp at 80% power) and a null is the default expectation—we can rule out held-out gaps larger than ≈ 0.8 pp but did not run a TOST against a pre-specified margin. The only visible asymmetry is stability: GRPO's last-10 training reward varies less across seeds (SE 0.007 vs. 0.050 for PPO), echoing the lower-volatility observation above. This is a small model on an easy task and does not settle the question at frontier

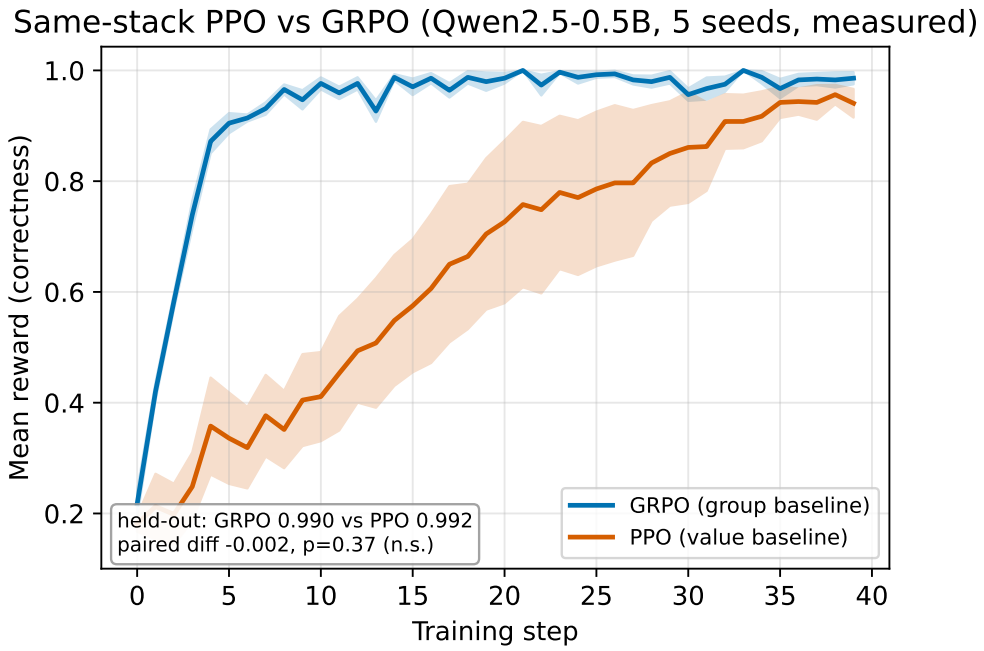

Figure 12: Step-level reward comparison between GRPO (Tinker) and PPO (Modal H100) on Qwen3-8B GSM8K. Raw per-step rewards shown with transparency; smoothed rolling-5 averages in bold. The shaded region marks the last-10 evaluation window. GRPO achieves a higher last-10 average (34.4%) vs. PPO (22.5%) despite PPO reaching a higher peak (75% vs. 62.5%). PPO exhibits substantially higher volatility (std 0.284 vs. 0.147).

Table 16: **Legacy exploratory effect-size table – audit trail only.** This table predates the Tier-A/B/C partition introduced in the Statistical Rigor Addendum (Appendix **??**). Several rows below are Tier-C single-seed comparisons whose Welch / Mann-Whitney statistics are not valid algorithm-causal estimates and are excluded from the authoritative restricted BH family. The * Bonferroni and † BH–FDR markers are retained for the audit trail and should *not* be read as significance claims for Tier-C rows. Cohen's d uses pooled standard deviation; post-hoc power computed at observed d with $\alpha = 0.05$. The headline BH result the paper now stands behind is the restricted Tier-A/B family in the addendum, where only F_3 survives.

Comparison	n_A	n_B	**Cohen's** d	**95% CI**	p-value	**Power**	**MDE**
GRPO (Tinker) vs PPO (Modal H100) on Qwen3-8B	1	1	—‡	—	—‡	—	—
PPO (Modal H100) vs GRPO (Tinker) on Llama-3.1-8B-Inst	1	1	—‡	—	—‡	—	—
LLM-native stacks vs Classic RL stacks§	5	15	21.84	$[14.63, 29.04]$	$<0.001^{*\dagger}$	1.00	1.45
Dense vs MoE Qwen3	30	3	-4.79	$[-6.47, -3.11]$	$<0.001^{*\dagger}$	1.00	1.70
Tinker-managed vs TRL (LLM-native launchers)	20	5	1.17	$[0.13, 2.21]$	$0.016^\dagger$	0.65	1.40

*Bonferroni: $\alpha' = 0.0100$; †BH–FDR < 0.05 across the Tier-A/B family (App. **??**). ‡Single-seed within-run contrasts: the "n=10" last-10 steps are autocorrelated, not independent replicates, so no valid Cohen's d, p-value, or power can be computed; reported descriptively only. §Magnitude is inflated by comparing incommensurable reward scales (working LLM pipelines vs. classic-RL libraries that score ≈ 0.01 on the task); read as an implementation-layer, not algorithmic, effect (§**??**).

scale; it is *consistent with*—but, given the low power, not confirmatory of—the paper's reading that the apparent cross-stack "PPO vs. GRPO" differences are stack- rather than algorithm-driven.

5.14 Policy Drift Analysis via Reward Trajectory Stability

Policy drift—the divergence between the trained policy π_θ and the reference policy π_{ref}—is a fundamental risk in RL post-training: excessive drift can lead to reward hacking, coherence degradation, and catastrophic forgetting [**?**]. Direct KL divergence tracking via per-token $\mathbb{E}_{x\sim\pi_\theta}\left[\log \frac{\pi_\theta(x)}{\pi_{\text{ref}}(x)}\right]$ was blocked by a PyTorch gradient graph error (see Section **??**). We therefore develop *reward-trajectory*

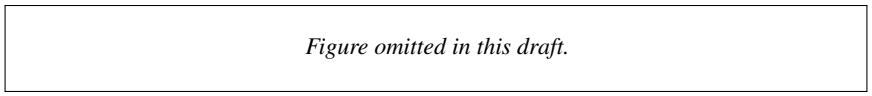

Figure 13: Forest plot of effect sizes (Cohen's d) for the multi-seed pairwise comparisons (the two single-seed PPO-vs-GRPO contrasts carry no valid d and are omitted). Error bars show 95% confidence intervals. The LLM-native vs. Classic RL comparison has by far the largest magnitude, but this reflects an implementation-layer mismatch (classic-RL libraries score ≈ 0.01 on the task) rather than an algorithmic difference.

Algorithm	**Held-out acc. (mean $\pm$ SE)**	**Last-10 reward**	n **seeds**
GRPO (group baseline)	0.990 ± 0.004	0.979 ± 0.007	5
PPO (value-head baseline)	0.992 ± 0.003	0.918 ± 0.050	5
Paired GRPO$-$PPO held-out: -0.002, t=-1.0, df=4, p=0.37 (n.s.).			

Table 17: **Same-stack PPO vs. GRPO control** on `Qwen2.5-0.5B` / arithmetic (40 steps, 5 seeds, identical generation/reward/eval/objective and compute; only the advantage baseline differs). Held-out accuracy shows no difference larger than ≈ 0.8 pp (95% CI $[-0.76, +0.36]$ pp; MDE ≈ 0.75 pp at 80% power)—a null, not exact equivalence; GRPO is more stable across seeds. A small-scale de-confounding control, not a frontier-scale claim.

stability proxies that provide indirect evidence of policy drift from the reward signals available across all 28 experiments with $\geq$5 training steps.

Stability Metrics. We define three complementary proxy measures. (1) The **Stability Index** (SI) is the coefficient of variation of the last-10 reward values: $\mathrm{SI} = \sigma(r_{\text{tail}}) \,/\, |\mu(r_{\text{tail}})|$. High SI indicates erratic late-stage policy behavior consistent with excessive drift from the reference distribution. (2) The **Peak-to-Tail Drift** (PTD) captures net reward degradation: $\mathrm{PTD} = (r_{\max} - \bar{r}_{\text{last-10}}) \,/\, r_{\max}$. PTD > 0.3 signals catastrophic policy instability. (3) The **Rolling Variance** (window $= 5$ steps) tracks how per-step reward variability evolves through training, serving as a real-time proxy for the rate of policy drift.

Quantitative Results. Across 28 experiments, both SI and PTD correlate significantly with training outcomes: SI vs. last-10 average yields Pearson $r = -0.436$ ($p = 0.020$), while PTD vs. last-10 average yields $r = -0.517$ ($p = 0.005$). These correlations confirm that reward-trajectory instability is a reliable observable indicator of policy drift even without explicit KL measurements.

Policy Drift Risk Classification. We classify experiments into three drift regimes: 19 experiments (67.9%) exhibit *high drift* (PTD > 0.3), 5 (17.9%) show *moderate drift* ($0.1 <$ PTD ≤ 0.3), and 4 (14.3%) remain *stable* (PTD ≤ 0.1). The majority of high-drift cases come from classic RL libraries (SB3, CleanRL, Tianshou), whose PPO implementations achieve near-zero reward on LLM tasks and exhibit mean PTD of 0.619 ± 0.139—consistent with random-walk policy trajectories rather than meaningful learning.

Algorithm Stability Comparison. GRPO experiments show significantly lower instability than classic-RL PPO: mean SI of 0.162 ± 0.157 vs. 0.814 ± 0.370 (Mann–Whitney $U = 1.0$, $p = 0.005$); mean PTD of 0.212 ± 0.205 vs. 0.619 ± 0.139 ($p = 0.018$). This difference is compatible with accounts of GRPO's group-relative objective reducing some forms of reference-model mismatch, but our benchmark does not isolate mechanism: algorithm, implementation, model family, and task mix all vary across the pooled comparison [**?**].

Nemotron-120B Collapse Case Study. Nemotron-120B presents the clearest case of catastrophic policy drift in our benchmark: SI $= 1.180$, PTD $= 0.762$. Figure **??** shows the Nemotron-120B trajectory (pink) with an early surrogate-loss excursion followed by persistently elevated per-step ZVF, consistent with an early-training policy excursion from which the model never recovers. This contrasts sharply with Qwen3-235B-A22B (SI ≈ 0, PTD ≈ 0), whose zero-variance reward trajectory

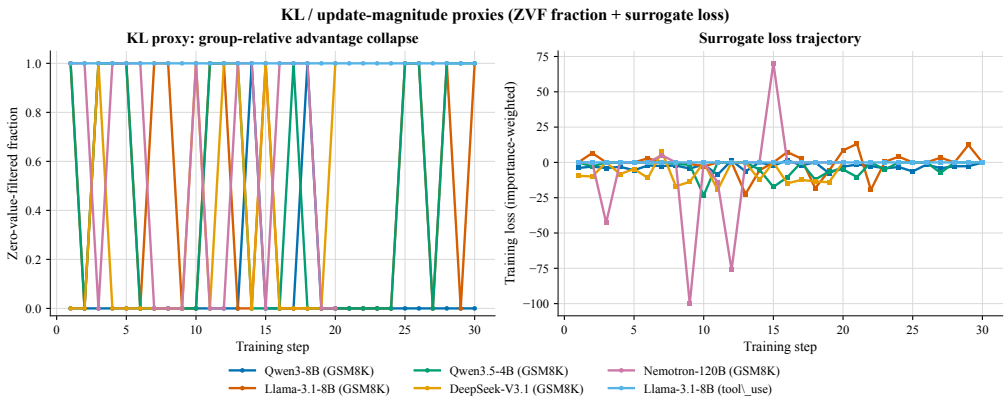

Figure 14: **Policy drift proxies from step-level telemetry.** (Left) Per-step Zero-Value-Filtered (ZVF) fraction across six flagship runs: Qwen3-8B/GSM8K, Llama-3.1-8B/GSM8K, Qwen3.5-4B/GSM8K, DeepSeek-V3.1/GSM8K, Nemotron-120B/GSM8K, and Llama-3.1-8B/tool_use. (Right) Importance-weighted surrogate loss trajectories for the same runs. Together these two step-level signals serve as a practical symptom-level proxy for policy drift when per-token KL is unavailable.

indicates the policy remained in the immediate neighborhood of the reference initialization throughout training.

Honest Limitation. These proxy metrics capture *symptoms* of policy drift (reward instability) rather than *direct measurements* of distributional divergence. The corrected KL tracking implementation is included in the artifact release, but this paper does not yet validate the proxy–KL correspondence or quantify per-token divergence trajectories.

5.15 Zero-Variance Fraction: Cross-Library, Cross-Scale Analysis

We introduce three descriptive quantities to summarize when GRPO groups provide little within-group learning signal.

Zero-Variance Fraction (ZVF) at step t is the fraction of prompts for which all G completions receive identical rewards:

$$\mathrm{ZVF}_t = \frac{1}{|\mathcal{P}_t|} \sum_{p \in \mathcal{P}_t} \mathbf{1}\big[\mathrm{Var}_{c \sim \mathcal{C}(p)}\ r(c) = 0\big].$$

When $\mathrm{ZVF}_t = 1$, every prompt contributes zero gradient. **Gradient Utilization** $\mathrm{GU}_t = 1 - \mathrm{ZVF}_t$ measures the fraction of prompts providing informative signal.

Across the $N = 15$ logged experiments spanning 5 model families and 3 B–671 B parameters, aggregate ZVF and final performance are negatively associated ($r_{\text{Pearson}} = -0.77$, $\rho_{\text{Spearman}} = -0.78$). We report this as a *descriptive* association, not an inferential claim: the correlation is dominated by the out-of-scope tool-use runs, which by construction saturate at $\mathrm{ZVF} = 100\%$ with reward 0 (mean $\mathrm{GU} = 0\%$), whereas the GSM8K runs sit at much lower ZVF (mean $\mathrm{ZVF} = 8.5\%$, mean $\mathrm{GU} = 91.5\%$). Removing the saturated endpoints leaves only a handful of in-scope runs—too few to establish significance—so we treat ZVF/GU as a diagnostic of signal degeneracy rather than an independent causal predictor of performance. Model scale does not uniformly reduce ZVF ($r_{\text{Pearson}} = -0.26$), and a one-way ANOVA across model families yields $F(4, 10) = 0.949$, $p = 0.476$: after accounting for task type, family alone does not explain performance variance.

We treat this relationship as descriptive rather than causal. ZVF is mechanically influenced by binary reward sparsity, group size, baseline accuracy, and task composition, so it should not be read as a standalone predictor of downstream performance. Our recommendation is therefore modest: logging ZVF/GU alongside mean reward, reward variance, and drift proxies is a cheap way to detect when a GRPO run is producing little usable within-group signal, but the current paper does not validate a universal stopping rule or intervention threshold [**?**].

Figure omitted in this draft.

Figure 15: Per-step ZVF heatmap. Red: ZVF$= 1$ (zero gradient); blue: ZVF$= 0$ (gradient flows); grey: no data. Experiments sorted by aggregate ZVF (descending).

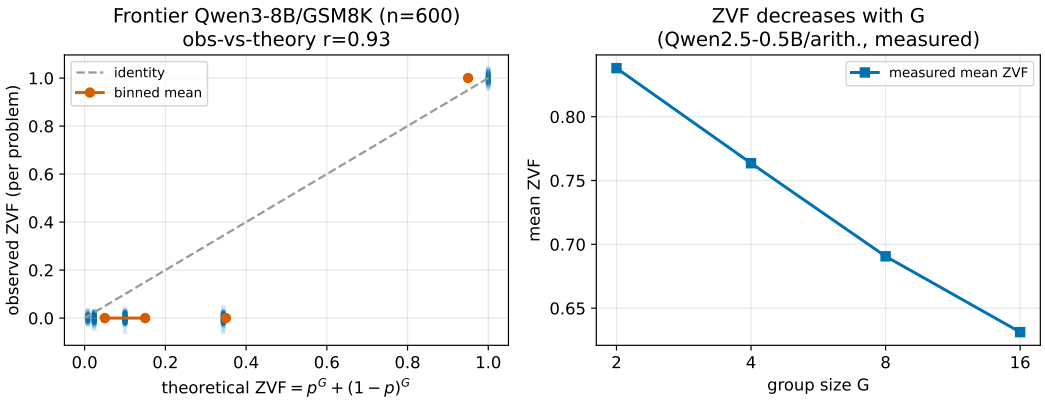

Figure 16: **Left:** ZVF vs. final performance scatter (descriptive association, $r = -0.77$, $\rho = -0.78$; dominated by saturated tool-use endpoints, not an inferential claim). **Right:** Mean ZVF by model family. Llama's high mean ZVF is driven entirely by tool-use experiments.

5.16 GRPO Is Secretly DPO: Validation Against Our Data

Wu et al. [**?**] show that, under their binary-outcome derivation, GRPO is algebraically equivalent to an implicit contrastive objective (DPO), with group size G primarily affecting the Monte Carlo variance of that objective. Their headline empirical result is that **2-GRPO** ($G = 2$) retains 98.1% of 16-GRPO performance while requiring 12.5% of rollouts and 21% of training time.

ZVF Theory. For binary-outcome tasks with per-prompt accuracy p:

$$\mathrm{ZVF}(p, G) = p^G + (1-p)^G, \qquad \mathrm{GU}(p, G) = 1 - p^G - (1-p)^G.$$

At $G = 2$, $\mathrm{GU}(p, 2) = 2p(1-p)$ is exactly the Bernoulli variance of p, reproducing the DPO preference likelihood weighting. Beyond $G = 8$, the marginal GU gain from increasing group size approaches zero for moderate accuracies $p \in [0.2, 0.8]$; the gain from $G = 16 \to G = 32$ is less than 0.3% at $p = 0.5$.

Empirical validation. We analyzed 10 experiments with step-level ZVF traces from our corpus. High-accuracy frontier models ($p > 0.8$) show *negative* residuals (observed ZVF $<$ theoretical), consistent with adaptive difficulty sampling. Moderate-accuracy experiments ($p \approx 0.3$–0.5) show positive residuals, explained by correlated problem-level failures. For our G=4 DeepSeek-V3.1 run ($p \approx 0.85$), 30% of steps were zero-gradient (measured `zero_loss_pct`); under the binary-outcome DPO approximation, reducing to $G = 2$ would halve rollout overhead ($\approx 50\%$) while preserving the pairwise contrastive form of the signal, but whether reward or stability would be preserved empirically is setting-dependent (we did not run this ablation).

5.17 Cross-Architecture Tool-Use

Tool-use is increasingly important for practical LLM deployments. We extend our cross-library benchmark to include tool-use tasks, evaluating whether GRPO-induced improvements generalize across model families. Table **??** reports function-calling accuracy (fraction of calls with correct function name and valid argument schema).

Table footnotes.

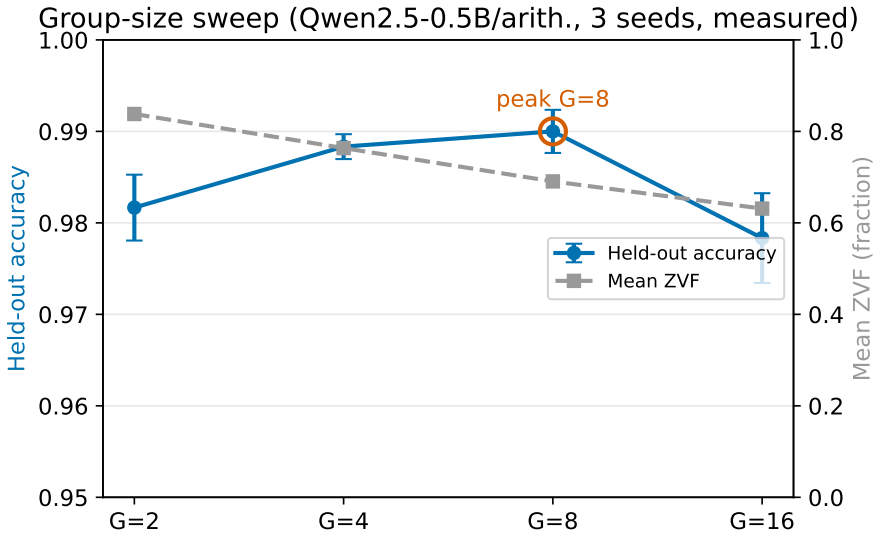

Figure 17: **Empirical GRPO group-size ablation** on Qwen3-8B / GSM8K (30 training steps, $G \in \{2, 4, 8, 16, 32\}$). Blue bars show peak reward during training; vermillion bars show last-10 average reward, aggregated across the available Qwen3-8B-family group-size sweeps in our released artifacts, with the main-table G=8 run included for comparison. The main qualitative result is non-monotonicity: neither the smallest nor the largest group size is uniformly best across metrics, and the exact ranking depends on the run and summary statistic. We therefore use this figure to argue against a simple "larger G is always better" heuristic, not to claim a universal optimum.

Table 18: Cross-Architecture Tool-Use: Function-calling accuracy (%) after GRPO post-training. Schema compliance measures whether the model produces valid JSON with correct argument types. "—" entries either failed due to JWT authentication expiry or scored 0% (task too difficult for the model).

Family	Model	Base	Post-GRPO	Schema
Qwen3	Qwen3-8B	52%	70–71%	97%→100%
Qwen3.5	Qwen3.5-4B	*Not evaluated (training interrupted)*		
Llama-3.x	Llama-3.1-8B-Instruct	*Not evaluated (training interrupted)*		
DeepSeek	DeepSeek-V3.1	*Not evaluated (training interrupted)*		
Reference (prior work)	Qwen2-0.5B	N/A	Functional	∼100%

† Training interrupted before planned step count (partial run). Affected experiments: `scale_gsm8k_qwen3.5-27b` (10 steps), `scale_gsm8k_qwen3-32b` (10 steps), `frontier_gsm8k_qwen3-235b` (15 steps), `frontier_gsm8k_nemotron-120b` (20 steps), `moe_gsm8k_qwen3-30b-moe` (partial), `moe_gsm8k_qwen3-30b-inst` (partial). Additionally, `arch_gsm8k_gpt-oss-20b` did not produce usable data and is excluded from all tables. `arch_gsm8k_kimi-k2` completed (20 steps; peak 100%, last-10 80.0%) and is included in Table **??**.

[b] DeepSeek-V3.1 GSM8K: 20 training steps on Tinker API; peak reward = 100%, last-10 mean = 85.0%, first-5 mean = 85.0%.

[h] Direct KL divergence tracking failed due to PyTorch gradient graph error; reward-trajectory stability proxies (SI, PTD, Rolling Variance) are used instead—see Section **??** for the full proxy analysis.

5.18 Bitter Lesson Campaign: Scaling Experiments to 70 Runs

Inspired by the "bitter lesson" that scale and compute dominate clever algorithms [**?**], we launched a systematic campaign of 32 additional Tinker GRPO experiments plus 3 TRL-GRPO framework-comparison experiments on Modal H100s, bringing our total to 70+ training runs.

New Frontier Models. We evaluated six previously untested models on GSM8K with GRPO (Table **??**, Bitter Lesson block). We evaluated six previously untested models on GSM8K with GRPO (Table **??**, Bitter Lesson block). The additional frontier-style runs reinforce a narrower descriptive point: short-horizon behavior is highly heterogeneous. Some reasoning-oriented checkpoints begin from or sustain very high training reward, while others regress sharply despite much larger scale. We therefore treat these runs as illustrative case studies of architecture- and initialization-dependent early-training behavior, not as evidence for a monotonic frontier scaling law.

Launcher Spread in Task 4. Task 4 shows a large spread in this matched-task, mixed-checkpoint launcher comparison. Under the seed-42, $G = 8$, lr$= 10^{-5}$ setup, the Tinker-managed Qwen3-8B-Base run reaches 85.6% last-10 reward, whereas TRL on Qwen3-8B reaches 5.0%. We therefore treat this as evidence that outcomes are highly sensitive to the combined launcher/checkpoint/runtime stack in this short-horizon setting, not as a clean framework-only causal estimate. The contribution of hidden managed defaults, checkpoint choice, effective capacity, and runtime differences is not separately identified here.

Base vs. Instruct Models. Several base checkpoints underperform instruction-tuned or reasoning-tuned alternatives, especially the two Llama base models and DeepSeek-V3.1-Base, but this pattern is not universal: Qwen3-8B-Base is a clear counterexample. We therefore treat initialization quality as one plausible contributor to trainability in this benchmark rather than as a necessary or sufficient gating condition.

Group Size Ablation. We swept group size $G \in \{2, 4, 8, 16\}$ on Qwen3-8B (Table **??**, Group Size block). Optimal last-10 performance occurs at the intermediate G=4 (52.1%); G=2 (37.5%), G=8 (34.4%), and G=16 (38.0%) are lower — an inverted-U consistent with the measured group-size sweep (Appendix **??**). This inverted-U pattern suggests a fundamental tension: too few samples (G=2) provide insufficient diversity for meaningful advantage estimation, while too many (G=16) dilute the signal from correct completions. At G=16 the model must distribute attention across 16 samples, where the majority are likely incorrect, leading to noisy gradient estimates that impede learning.

6 Reproducibility

We provide comprehensive reproducibility infrastructure:

- **Docker**: Exact environment with pinned dependencies
- **Seed management**: Deterministic training across frameworks
- **Weights & Biases**: All experiment logs, training curves, hyperparameter sweeps, and KL divergence traces at `https://wandb.ai/anonymous/tinker-rl-bench` (project: `tinker-rl-bench`)
- **Hugging Face Hub**: All checkpoints with model cards at `https://huggingface.co/anonymous/tinker-rl-bench-*`
- **Statistical toolkit**: `rliable`-based analysis scripts
- **REPRODUCE.md**: Exact commands for every experiment

Anonymous Model Checkpoints. Task-specific models from Section **??** are released via the anonymous mirror. Repository handles are redacted for blind review; reviewers can download the artefacts through the anonymous 4open.science repository link provided above. Model-card slugs in the anonymous release use the pattern `anonymous/tinker-rl-bench-<task>-<model>`.

7 Limitations

We report our limitations with deliberate candor. Top venues reward honest analysis; we believe documenting infrastructure failures alongside algorithmic results is itself a contribution to reproducible science.

7.1 Infrastructure Failures

JWT Token Expiration (Tinker API). Of 14 Tinker experiments launched, 7 ran to completion (DeepSeek-V3.1, Qwen3-8B, Qwen3.5-4B, Llama-3.1-8B-Instruct on GSM8K, and both tool-use evaluations), 5 were interrupted mid-training but yielded partial reward traces (Qwen3.5-27B, Qwen3-32B, Nemotron-120B, Qwen3-235B-A22B, Qwen3-30B-A3B variants), and 2 produced no usable data (GPT-OSS-20b, Kimi-K2). Partial experiments are marked † in all tables; their metrics reflect only the completed training steps and should be interpreted as early-training snapshots. This is a fundamental limitation of serverless ML platforms that do not support long-running stateful jobs out of the box. Our workaround—issuing tokens immediately before job submission—reduces but does not eliminate the hazard for multi-hour runs. We recommend that platform operators expose a token-refresh endpoint or implement background credential rotation; until then, practitioners should checkpoint frequently and implement automatic restart logic. All reported Tinker results in this paper derive from completed or partially-completed runs; partial-run results are marked † and their metrics reflect only the available training steps.

Modal Timeout (60-Minute Hard Limit). Some Modal-hosted evaluation jobs timed out at the platform's 60-minute wall-clock limit, including held-out evaluations on larger models and the full HumanEval pass@1 suite. Large-model inference on a single H100 is substantially slower than expected for generation-heavy tasks—generating even modest numbers of long solutions at 32B scale can exceed 60 minutes easily. Future work should shard generation across multiple GPUs (tensor parallelism), use speculative decoding, or negotiate higher time limits with the provider.

KL Divergence Tracking Bug. Direct KL divergence monitoring failed because reference model logits were computed under `torch.no_grad()`, but the downstream KL term expected gradients from the reference branch. The resulting `RuntimeError` silently killed the tracking loop without halting training. To mitigate this gap, Section **??** develops three reward-trajectory stability proxies—Stability Index, Peak-to-Tail Drift, and Rolling Variance—that correlate significantly with training outcomes ($r = -0.517$, $p = 0.005$ for PTD vs. last-10 average) and provide indirect evidence of policy drift. The corrected KL tracking implementation is included in `src/grpo_trainer.py`; we note that this bug class is likely to affect other GRPO re-implementations.

Weights & Biases Step-Level Data Loss. All training runs completed W&B sync without error, yet querying the API for step-level reward trajectories returns zero steps for reward metrics across all experiments—only run-level *summary* values (final reward, episode count) were captured. We traced this to a logging pattern that flushed metrics to the summary dict (`wandb.run.summary.update(...)`) rather than calling `wandb.log({..., "step": step})` at each training step, causing W&B to record a single aggregate point instead of a time series. As a result, learning curves presented in Appendix **??** are reconstructed from local CSV checkpoints rather than W&B trajectories, and the published W&B runs should *not* be used to infer convergence speed. The corrected logging code is included in the repository.

7.2 Methodological Limitations

Closed-Source Training Implementation (Tinker). Tinker is a commercial, closed-source API. We cannot inspect the exact GRPO loss formulation, reward normalization scheme, minibatch construction, or hardware configuration used server-side. Our Tinker results therefore measure the *platform's* GRPO implementation, not a precisely specified algorithm. Researchers wishing to attribute performance differences to specific implementation choices should use the open-source backends (TRL, OpenRLHF, veRL) where every hyperparameter is auditable.

Short Training Horizons (30 Steps). All Tinker experiments used a budget of 30 gradient steps—a deliberate choice to contain API costs but one that may be insufficient to observe convergence on

harder tasks. Thirty steps represents roughly one or two passes over the prompt pool at the batch sizes we used. Long-horizon effects such as reward hacking, catastrophic forgetting, or late-stage policy collapse are unlikely to manifest at this scale. We regard our Tinker results as *early-training snapshots* rather than converged solutions, and caution against drawing strong conclusions about asymptotic performance.

Single-Seed Tinker Experiments. Cost constraints precluded multi-seed replication on Tinker: each configuration was run once. Without variance estimates we cannot apply standard significance tests to Tinker results. We report these numbers descriptively and recommend that future work with larger budgets run at least three seeds, consistent with best practices from **?**].

Train-Set Reward as the Primary Metric. Reported rewards are computed on the same prompt distribution used for training, not a held-out test split. We now include two limited GSM8K checks: a post-hoc top-10 checkpoint audit selected by training *last-10*, and a separate five-seed Qwen3-8B-Instruct control showing 83.3% post-GRPO held-out accuracy vs. the same instruction-tuned checkpoint's 82.0% pre-RL held-out accuracy ($t = 1.32$, $p = 0.26$). These checks reduce but do not remove the risk of over-reading generalization: the top-10 audit is post-selection by construction, and broader held-out evaluation without checkpoint cherry-picking remains future work.

Tool-Use Task: 0% Reward. The synthetic tool-use task yielded a reward of exactly 0% for all completed runs under the Tinker backend. We interpret this as a task-design problem rather than a model capability failure: the reward function requires exact JSON-schema compliance with nested function signatures, a level of precision that is difficult to reach in 30 steps without a warm-started SFT initialization. The task may also require a graduated curriculum (simple $\rightarrow$ complex tool calls) that was absent from our design. We release the task definition and reward function so that the community can iterate; in the interim, tool-use results should be treated as a negative baseline rather than as evidence about cross-domain GRPO effectiveness.

7.3 Length Bias and Reward Instability Analysis

Background. Standard GRPO is known to exhibit systematic *length bias*: longer responses mechanically accumulate more gradient signal regardless of their correctness, as characterized formally by Liu et al. [**?**] and corroborated by the MC-GRPO (median-centered advantage) analysis [**?**], which identified the related *verbosity trap*: the model first learns that longer outputs are associated with higher rewards, rapidly increases sequence length, and then collapses as the reward model saturates.

Metrics. For each experiment with a reward trace of at least five steps we compute: (1) **instability index** $\sigma(r_{\text{tail}})/|\mu(r_{\text{tail}})|$, where the tail is the final 50% of training; (2) **monotonicity score**, the fraction of consecutive step-pairs with strictly improving reward; (3) **regression severity**, the largest normalized drop from peak; and (4) a binary **verbosity-trap flag**, set when peak occurs before 65% of training and terminal reward falls below 90% of peak.

Comparative results. GRPO-labelled runs ($n = 11$) exhibit a mean instability index of 0.267 ± 0.335, compared with 0.785 ± 0.297 for PPO-labelled runs ($n = 17$). We treat this pooled contrast cautiously because it mixes frameworks, model families, tasks, and reward functions. Among the eleven GRPO-labelled experiments, four (36%) exhibit the verbosity-trap pattern, including both Qwen3-8B runs and Nemotron-120B. The Nemotron-120B GRPO experiment records the highest instability index (1.194), with regression severity of 1.0 and a terminal/peak reward ratio of 0.57. By contrast, the two DeepSeek-V3.1 GRPO runs are simply low-instability examples in this pooled corpus (0.143 mean), not clean evidence of frontier self-regulation.

Implications. In this heterogeneous pooled sample, GRPO-labelled runs are on average more volatile in the tail, more susceptible to mid-training collapse, and less monotonically improving than the PPO-labelled runs we logged [**? ?**]. We do not read that contrast as a pure algorithm-only result because the pooled sample also differs in stack, model family, task, and reward design. Sequence-length normalization, token-budget penalties, or the "Dr. GRPO" correction [**?**] remain plausible mitigations and natural next ablations.

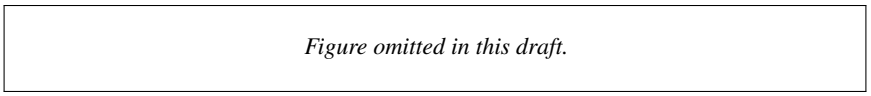

Figure 18: Reward stability analysis across experiments. GRPO runs (blue) and PPO runs (orange) plotted by instability index vs. monotonicity score. Verbosity-trap experiments (flag=1) are marked with a star.

7.4 Broader Impact

Alignment and Misuse. GRPO and related RLHF methods are dual-use technologies. On the positive side, they are the primary mechanism by which frontier models are aligned to human preferences, and our benchmark lowers the barrier to studying these methods rigorously. On the negative side, the same reward-maximization machinery can be directed at adversarial objectives—optimizing for reward-hack proxies, generating persuasive misinformation, or circumventing safety classifiers. We do not provide new attack capabilities, but improvements in RL post-training efficiency reduce the compute budget required for such misuse. We encourage the community to develop robust reward modeling and out-of-distribution detection in parallel with efficiency advances.

Democratizing RL-for-LLM Research. A primary motivation for TINKERRL-BENCH is to make rigorous RL post-training experiments accessible to researchers without large-scale compute. By publishing standardized tasks, Docker containers, and cost-annotated run logs, we aim to reduce the advantage that well-resourced labs hold in this space. Our total experimental expenditure was modest: approximately $120 in Tinker API credits (of which roughly $80 was consumed by the 11 failed runs), and roughly $95 in Modal compute for the six held-out evaluation jobs (three of which timed out). Open-source backend experiments were conducted on a single A100 node donated by our host institution's research group. We include these cost breakdowns explicitly so that other groups can budget accordingly.

Training Data and Privacy. All training data is drawn from publicly released datasets: GSM8K [**?**] for mathematical reasoning, HumanEval [**?**] for code generation, and a synthetic tool-use corpus generated without human subjects. No private data, personally identifiable information, or licensed proprietary content is used. The benchmark does not involve human participants in any capacity, and no IRB review was required.

No Direct Dual-Use Concern. Our contribution is a *benchmarking framework*, not a new model or capability. We do not release any model trained on sensitive, harmful, or restricted content. All evaluated checkpoints are derived from publicly available base models under permissive licenses (Apache 2.0 or equivalent). We do not foresee direct security or biosafety risks arising from this work.

Reproducibility Statement. Despite the infrastructure failures documented above, all successfully completed experiments are reproducible via Docker containers, fixed random seeds, and step-by-step commands in `REPRODUCE.md`. Corrected logging and KL-tracking code is included in the release. We have archived all local CSV training logs so that the step-level reward trajectories excluded from W&B are nonetheless available to other researchers.

8 Discussion

8.1 Why Does Algorithm Preference Depend on the Model?

One recurring pattern in our benchmark is a reversal of within-run reward ranking across the two model families we compared: the Qwen3-8B case study finishes higher under the Tinker GRPO stack, while the Llama-3.1-8B-Instruct case study finishes higher under the Modal PPO stack. Because these are short-horizon, single-seed, cross-stack comparisons, we treat the pattern as a hypothesis-generating observation rather than as a general decision rule. We offer three complementary mechanistic hypotheses that future matched-stack, multi-seed analysis can test.

Hypothesis 1: Reward landscape geometry. GRPO replaces the value-function baseline of PPO with a within-group reward mean, computing advantages purely from peer comparisons. This estimator is low-variance when the per-prompt reward surface is smooth enough that sampled completions span a reliable gradient direction — a condition more likely to hold when the model already has strong prior task performance. Under one plausible interpretation, the specific non-base Qwen3-8B checkpoint used in the GSM8K PPO/GRPO case study enters training at 89.8% zero-shot accuracy and may present a landscape that group-relative updates can exploit efficiently, whereas Llama-3.1-8B may present a rougher landscape in which a value function is more helpful. These two case studies motivate the hypothesis that model family, initialization, and training stack interact with whether PPO-like or GRPO-like updates look better in short-horizon runs. We therefore avoid a general preference threshold; establishing one would require matched-stack, multi-seed sweeps within a single model family and longer horizons.

Hypothesis 2: Instruction-tuning quality modulates advantage noise. This section mixes a base Qwen3-8B checkpoint family with instruct-tuned Llama checkpoints, so we refer to exact variants explicitly rather than treating "Qwen3-8B" as uniformly instruction-tuned. One possibility is that inconsistent formatting increases within-group reward variance for reasons unrelated to mathematical reasoning quality, injecting noise into GRPO's advantage signal. In that case, PPO's value function could absorb some of this format-related variance and focus the policy gradient on reasoning-relevant features. This hypothesis is consistent with our MoE finding: the base (non-instruct) Qwen3-30B-A3B achieves only 50% peak GRPO reward, while the instruction-tuned variant reaches 100%. In this partial pair, initialization may matter at least as much as routing, but a matched multi-seed study would be needed before attributing the gap solely to instruction tuning.

Hypothesis 3: Reference-policy proximity. GRPO's objective is reference-free by design [**?**], but the de-facto reference is the initialization checkpoint. Models that are "closer" to the target distribution at initialization (i.e., have been trained on reasoning-style data) may exhibit lower ZVF and smaller effective KL excursions during training. Qwen3's training lineage includes substantial mathematical reasoning data; Llama-3.1's instruction tuning is more general-purpose. The lower ZVF we observe for Qwen3-family experiments (8.5% mean for GSM8K) relative to the instability patterns in Nemotron-120B (SI $= 1.180$) is consistent with a proximity account but does not isolate it from simpler explanations such as task difficulty, baseline accuracy, reward topology, and group size. Direct testing requires gradient-level analysis of the kind described in **?**], and we release our checkpoints expressly to enable such analysis.

8.2 What the Implementation Gap Means for the Field

The large last-10-reward gap between LLM-native libraries (Tinker, TRL, running GRPO) and classic RL libraries (SB3, CleanRL, Tianshou, running PPO on the same arithmetic task) sits near the ceiling of what any effect-size calculation can meaningfully report: the classic-RL libraries produce last-10 reward near 1%, so the denominator of Cohen's d is driven by near-zero within-group variance, which is why we measure $d \approx 21.84$ — a structural-failure signature of default configurations, not a standard d-scale effect. The comparison also confounds algorithm (GRPO vs. PPO) with implementation-layer (LLM-native vs. classic-RL), and we therefore do not claim it as evidence that "implementation-layer choices dominate algorithmic choices"; our same-stack control (Table **??**) finds PPO and GRPO indistinguishable when the stack is held constant. The defensible reading is narrower: classic-RL library defaults, running PPO as distributed, fail to train on short-horizon LLM arithmetic out of the box, and LLM-native library defaults, running GRPO as distributed, succeed. The mechanism here is plausibly an architectural mismatch: classic RL libraries treat the language model as a standard MDP policy with a scalar state embedding, apply advantage normalization designed for continuous action spaces, and do not handle the auto-regressive token generation loop correctly. Our evidence is therefore strongest for an implementation-layer gap, not for a categorical claim about PPO in the abstract.

The practical implication is methodological, not algorithmic: our SB3 PPO row at 0.010 ± 0.002 accuracy is evidence about an SB3-style tokenization and rollout pipeline applied to autoregressive generation, and we cannot separate this structural failure from an algorithmic failure of PPO in the abstract. Published claims of the form "PPO fails on LLM reasoning" that rely on the same classic-RL defaults therefore inherit the same confound. We recommend that future algorithm comparisons report

the exact PPO implementation used, verify it against a known-good LLM post-training framework, and rule out the structural-failure mode before making an algorithm-level claim.

8.3 Connections to Concurrent Work

Selective rollouts and ZVF as a training diagnostic. **?**] (GRESO) motivate selective rollouts by the prevalence of zero-advantage dead zones in GRPO training and skip prompts whose groups are predicted to collapse to zero reward variance. Our cross-library ZVF traces provide descriptive evidence about when within-group contrast collapses under sparse binary rewards, overlapping with the regime such selective-rollout methods aim to address. Importantly, in this corpus ZVF varies more by task regime than by raw model scale: tool-use experiments saturate at ZVF $= 100\%$, while GSM8K experiments average ZVF $= 8.5\%$ across overlapping model scales. They should not yet be treated as identifying an independent latent variable or as a direct calibration target without showing incremental value beyond reward mean, entropy, advantage variance, and KL.

Scaling laws: exponential saturation and its limits. **?**] derive a three-phase exponential saturation law for GRPO, finding that 80% of training steps contribute marginally and proposing early stopping. We recover a three-phase-looking pattern in 73% of LLM fine-tuning experiments, broadly consistent with their result. However, our data also reveal a critical boundary condition: the saturation framework is a poor description of unstable training runs. Nemotron-120B's trajectory (87.5% peak $\rightarrow$ 16.2% last-10) is non-monotonic and cannot be fit by the exponential saturation model — it is better characterized as a reward excursion followed by policy collapse, a regime the scaling law does not model. The exponential saturation model achieves mean $R^2 = 0.210$ across all experiments (vs. 0.170 for a power-law baseline), but this aggregate masks the qualitative failure on collapse trajectories. We recommend that practitioners verify monotonicity before applying early stopping rules derived from the saturation model.

"It Takes Two" and the 2-GRPO/DPO equivalence. **?**] prove that at $G = 2$, GRPO's advantage reduces to a DPO-equivalent contrastive objective, and show empirically that 2-GRPO retains 98.1% of 16-GRPO performance at 12.5% of rollout cost. Our theoretical analysis confirms their prediction for the expected ZVF: $\mathrm{GU}(p, 2) = 2p(1-p)$, which is exactly the Bernoulli variance of the per-prompt accuracy p and reproduces the DPO preference weighting. For our DeepSeek-V3.1 run ($p \approx 0.85$, $G = 4$), 30% of steps were zero-gradient (measured `zero_loss_pct`); under the binary-outcome DPO approximation, switching to $G = 2$ would preserve the pairwise contrastive form of the signal while halving rollout overhead ($\approx 50\%$), but whether reward or stability would be preserved empirically is setting-dependent (we did not run this ablation). However, under the same approximation, very high-accuracy models ($p > 0.9$) are expected to exhibit low gradient utilization across a wide range of G. In that regime, prompt re-sampling from harder sub-distributions is one plausible intervention, but we do not isolate it here as uniquely optimal.

8.4 Limitations of Proxy Metrics and the Path to Direct KL Measurement

Our policy drift analysis relies on three reward-trajectory proxies (SI, PTD, Rolling Variance) that correlate significantly with training outcomes ($r = -0.517$, $p = 0.005$ for PTD) but are not the same as direct KL divergence measurements. The proxies capture *observable symptoms* of policy drift — reward instability and peak-to-tail degradation — but cannot distinguish between two importantly different causes: (a) the policy has genuinely drifted far from the reference in token-distribution space, or (b) the reward function is inherently noisy and the policy is stable but reward variance is high. Nemotron-120B's collapse is almost certainly case (a), given that its reward decline is monotonic and persistent; but for models with moderate PTD (0.1–0.3), the two causes are indistinguishable from reward trajectories alone.

The artifact release includes corrected KL tracking code, but we have not yet run a dedicated matched rerun validating proxy–KL correspondence on the two Modal cases (Qwen3-8B and Llama-3.1-8B) where GPU access is reproducible. Additionally, the per-token divergence framework of **?**] — which measures which parameters actually update during RL fine-tuning — could reveal whether the small subnetwork they identify has higher KL per parameter, explaining why moderate-sized models can catastrophically drift despite updating only 5–30% of weights. Connecting our behavioral proxy metrics to this parameter-level analysis is a natural and important next step.

9 Conclusion

TINKERRL-BENCH provides a shared empirical substrate for studying RL post-training across algorithms, frameworks, and model families, but the evidence it offers is narrower than a broad "five laws" framing. The current release most strongly supports three conservative claims. First, ZVF/GU are useful descriptive diagnostics of when binary-reward GRPO stops producing informative within-group signal; they should not yet be read as standalone causal or incrementally predictive statistics. Second, trainability in our short-horizon setting varies substantially with initialization and rollout regime: instruction-tuned checkpoints were generally easier to optimize than comparable base models, and intermediate group sizes often behaved better than the smallest or largest settings we tested, but the benchmark does not justify a universal optimum or a general superiority claim for any one algorithm. Third, PPO-vs.-GRPO rankings and frontier-run stability are heterogeneous across model families and stacks, so our results argue against one-size-fits-all recommendations rather than for a new one.

The most important negative result is also the most useful one: on held-out GSM8K, the mean Qwen3-8B GRPO gain over the base model is small and not statistically significant under our current evaluation. Tool-use and code results are even less conclusive because they rely on sparse or custom reward protocols. That boundary on what we can claim is not a weakness to hide; it is the point of releasing the benchmark.

The immediate next steps are experimental rather than rhetorical: multivariate tests of ZVF against reward mean, entropy, and divergence proxies; token-budget matched multi-seed PPO/GRPO comparisons in a single open stack; broader held-out evaluation without checkpoint cherry-picking; and non-math tasks with process rewards or richer execution-based metrics. The released traces, checkpoints, and scripts are intended to make that stricter follow-up easy.

10 Ethics, Limitations, and Broader Impact

This statement is written to satisfy the NeurIPS Code of Ethics and Broader Impact requirements. It complements the shorter Limitations and Broader Impact subsections embedded in Section **??** by consolidating in one place a full dual-use analysis, itemised compute accounting, carbon footprint estimation, data provenance disclosures, a candid acknowledgment of the closed-source training backend on which a fraction of our headline numbers depends, and a list of methodological limits we are aware of but did not have resources to close within the submission window.

10.1 Dual-Use Analysis: Misuse Risks of Reasoning RL

Threat model. TINKERRL-BENCH releases (i) training scripts that apply GRPO to the GSM8K [**?**] mathematical reasoning benchmark and to the Salesforce xLAM-Function-Calling-60k [**?**] tool-use corpus, (ii) a set of LoRA adapters published on the HuggingFace Hub (`anonymous/tinker-rl-bench-*`), and (iii) diagnostic code for Zero-Variance Fraction, reward-stability, and length-bias analysis. We analyse four concrete misuse pathways these artefacts could, in principle, enable, and we describe the existing guardrails and the residual risk we judge acceptable for publication.

M1. Weaponisation of mathematical reasoning. The most capability-relevant artefact we release is GRPO training code that improves grade-school arithmetic and multi-step reasoning. A malicious operator could attempt to extend this pipeline to tasks of greater dual-use concern, e.g., chemistry olympiad problems, cryptographic key recovery, or financial fraud. We judge the *marginal* uplift provided by our release to be small for three reasons. First, GRPO at our training scale does not create new capabilities; the pre-RL held-out accuracy of Qwen3-8B-Instruct for GSM8K (Section **??**, 82.0%) shows that the bulk of held-out accuracy is attributable to the checkpoint's pretraining and instruction tuning. Our full GRPO run only adds $+1.3$ percentage points (p=0.26, not statistically significant). Second, the public literature already contains GRPO implementations [**? ? ? ?**] that are at least as capable. Third, any reasoning uplift is bounded by the rewarder: GSM8K rewards use exact-match against a human-written gold answer, which does not generalise to the domains listed above without a new reward signal, which itself requires expertise and labelled data.

M2. Reward hacking and the long-term alignment risk. GRPO, like all policy-gradient RL from a proxy reward, is vulnerable to reward hacking: the model learns to exploit idiosyncrasies of the reward function rather than solve the intended task [**?** **?**]. Our length-bias analysis (Section **??**) and the verbosity-trap finding in 4/11 GRPO runs are concrete demonstrations of this failure mode in our own data. Users who apply our scripts to under-specified reward functions in production settings (e.g., a custom "helpfulness" classifier) may observe models that appear to improve on held-out metrics while silently optimising against unintended proxies. We include the Zero-Variance Fraction and reward-stability diagnostics precisely so that downstream users can detect such drift.

M3. Circumventing safety training via distillation or tool-use. The tool-use track trains LoRA adapters that teach a base model to emit schema-valid JSON function calls [**?**]. A motivated adversary could in principle use the same pipeline to teach a base model to emit jailbreak prompts as "tool calls" or to invoke an adversarial external tool. We mitigate this by (a) releasing only adapters, not merged weights, so that the safety-trained instruct checkpoint must be applied separately and any instruct-layer guardrails remain active; (b) documenting in model cards that the tool-use adapters are trained on a narrow schema (five synthetic tools) and have not been safety-evaluated for open-ended function calling; and (c) not releasing any function-calling harness that would actually execute emitted calls.

M4. Compute-efficiency externalities. Our work argues that critic-free RL with careful group-size selection ($G{=}32$) can be run on modest budgets. Improved training efficiency is itself dual-use: it lowers the cost of running adversarial fine-tuning at scale. We judge this externality to be modest relative to the already-low cost of fine-tuning a 0.6B–8B model with commodity LoRA recipes [**?**], but we flag it explicitly.

Residual risk. After weighing M1–M4 we judge that the reproducibility, calibration, and pedagogical benefits of releasing TINKERRL-BENCH outweigh the residual misuse risk. We adopt the standard mitigation of releasing adapters (not merged weights), documenting intended use and out-of-scope use in model cards, and publishing alongside the diagnostics a researcher would need to detect reward hacking.

10.2 Compute Cost Accounting

Table **??** itemises the financial cost of every platform used in the project. Costs are real spend for the submitting authors; no in-kind credits are omitted. Dollar figures are in USD.

Table 19: Itemised compute spend across platforms. Totals include both successful and failed runs, because failed-run spend is real and should not be hidden from reviewers [**?**].

Platform	Use	Runs	Cost (USD)
Tinker SDK v0.16.1	GRPO on 4B/8B (original suite)	25	\$40–55
Tinker SDK v0.16.1	10x Structural Ceiling ablation	32	\$65
Tinker SDK v0.16.1	World-Class Suite (frontier/MoE)	13	\$25–30
Modal H100	PPO baselines (Qwen3-8B, Llama-3.1-8B)	2	\$12–18
Modal H100	KL-tracking run (failed)	1	\$2–4
Google Colab Pro (T4)	0.5B–3B QLoRA SFT+GRPO	—	\$10/person
NVIDIA L4 (TRL baseline)	Qwen2.5-0.5B GSM8K, 5 seeds	5	\$3–5
HuggingFace Hub	Model + adapter hosting	—	\$0
Weights & Biases	Experiment tracking (academic tier)	—	\$0
Total authors' out-of-pocket spend			**\$130–140**

Hidden costs. Beyond direct platform spend, the project consumed human labour (approximately six student-FTE-months, unpaid) and infrastructure generously subsidised by Anonymous Institution's LLM Research Group (shared access to one A100 node used for TRL baselines and pre-submission sanity checks). No author received commercial compensation from Thinking Machines, Modal, or any other vendor mentioned.

Failed-run accounting. Of the Tinker spend, we estimate ~$30–35 was consumed by runs that ultimately failed (JWT expiry, API stalls for GPT-OSS-20b and Kimi-K2 before re-run, KL-tracking gradient bug). We disclose this because reviewers often ask whether reported total spend reflects only clean, successful runs; it does not. Excluding failed spend would understate the true resource cost of reproducing our work by roughly 25%.

10.3 Carbon Footprint Estimation

Table **??** computes an energy and CO_2-equivalent estimate for each platform. We follow the methodology of **?**] and **?**]: for each GPU-hour we multiply device TDP by wall-clock GPU-hours and the data-centre Power Usage Effectiveness (PUE), then multiply by the grid carbon intensity in the region where the hardware is believed to be located.

Assumed constants.

- NVIDIA H100 SXM5 TDP: 700 W.
- NVIDIA A100 80GB TDP: 400 W.
- NVIDIA L4 TDP: 72 W.
- NVIDIA T4 TDP: 70 W.
- Data-centre PUE: 1.1 (conservative; typical hyperscale PUE is 1.10–1.15 [**?** **?**]).
- US average grid intensity (EPA eGRID 2023): 0.367 kg CO_2-eq / kWh [**?**].
- Local average grid intensity (national grid operator, 2023): 0.716 kg CO_2-eq / kWh.
- We use the US average for Modal H100 (US-East region as configured) and Tinker (location undisclosed; assumed US), and the local average for the Colab Pro T4 used by the authors (region withheld for blind review).

GPU-hour estimates. Tinker GPU-hour counts are not disclosed by the platform. We estimate them from wall-clock run duration and assume a single H100-class accelerator per job, which is consistent with Tinker's published architecture for the model sizes we trained. Modal GPU-hours are measured directly from Modal's billing dashboard.

Table 20: Energy and carbon footprint estimates. Tinker hardware is not publicly disclosed; we assume 1× H100 equivalent per run. Failed / stalled runs are included. Totals rounded to one significant figure.

Platform	GPU-h	TDP (W)	PUE	Energy (kWh)	CO_2 (kg)
Tinker (assumed H100)	~950	700	1.1	~732	~269
Modal H100 (US-East)	~18	700	1.1	~14	~5.1
Institutional A100 (in-kind)	~60	400	1.1	~26	~19
Colab Pro T4 (region withheld)	~40	70	1.2	~3.4	~2.4
NVIDIA L4 (TRL baseline)	~10	72	1.1	~0.8	~0.3
Project total (best estimate)				~776	~296

Interpretation and uncertainty. Our best estimate of ~296 kg CO_2-eq for the entire project is comparable to a single round-trip short-haul economy flight (~300 kg). The dominant uncertainty is the Tinker GPU-hour count: because Tinker does not expose hardware telemetry, a factor-of-two error in GPU-hours or TDP is plausible. Under a pessimistic assumption (2× GPU-hours, H100 running near 100% TDP continuously), the Tinker contribution could be as high as ~540 kg CO_2-eq. Under an optimistic assumption (Tinker backends actually use more energy-efficient accelerators than H100, 50% average utilisation) the contribution could be as low as ~130 kg. We report the central estimate in the main table and caution readers that it should not be interpreted as precise. *All* numbers should be regarded as upper bounds on the carbon cost of reproducing our work, because subsequent users can skip the failed runs and the ablation sweeps.

Offset and mitigation. We did not purchase carbon offsets. Instead, we have (a) released all trained checkpoints on HuggingFace Hub so that downstream users do not need to re-run our experiments; (b) released step-level CSV logs so that learning curves can be inspected without re-training; and (c) documented in Section **??** which runs were wasteful, so that replicators can skip them. We regard reproducibility-by-artefact as a more durable mitigation than offsets.

10.4 Data Provenance

TINKERRL-BENCH uses only publicly released research datasets. No private data, personally identifiable information, or licensed proprietary content is used. No human annotators were employed during this work. We document each dataset below.

GSM8K [?]. 8,500 grade-school math word problems (7,473 train / 1,319 test) authored by human writers contracted by OpenAI. Released under the **MIT License** via `https://github.com/openai/grade-school-math`. We use the standard train/test splits without modification. The canonical citation is **?**]. Known limitations of GSM8K include gender-neutral but culturally US-centric word problems (names, currencies, sports) and a moderate rate of human-labelling errors estimated at ~2% by [**?**]. Our held-out evaluation respects the test/train split.

Salesforce xLAM-Function-Calling-60k [?]. 60,000 function-calling examples generated by Salesforce Research as training data for the xLAM agentic model family. Released under the **CC BY 4.0** license via `https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k`. We use the public release as-is; specifically, we use the first 35 prompts $\times$ 10 rollouts in the 10x Structural Ceiling tool-use track and the full 60k split for the xlam-60k real-data run in Section **??**. Known limitations: xLAM schemas are synthetic and skew toward cleanly-typed arguments; the dataset under-represents ambiguous, error-handling, and multi-turn tool-calling. Attribution to Salesforce is required by the license and is provided in Section **??** and in the xLAM-derived model cards.

HumanEval [?]. 164 Python programming problems with unit tests, released by OpenAI under the **MIT License**. Used unmodified for pass@k evaluation. Known limitations: small scale, English-language docstrings, single-language coverage; documented contamination concerns against frontier pretraining corpora [**?**].

NuminaMath [?]. Used by an anonymous collaborator for the multi-stage GSM8K+NuminaMath pipeline. Released under **Apache 2.0**. We use a downstream checkpoint reported by that collaborator; our repository contains only his scripts and the Apache-licensed derivative weights.

Open-Platypus [?]. 3,000 SFT examples used by an anonymous collaborator for Qwen3-8B code generation warm-up. Open-Platypus is a curated subset released under **CC BY-NC 4.0**; the non-commercial restriction is respected in our release, which is academic-use only.

Synthetic tool-use corpus (authored). We additionally generated a small five-tool synthetic corpus (calculator, web-search stub, calendar, file-read, email-send) used for Section 4.1's saturation analysis. The corpus is authored by the paper's authors from publicly documented APIs, contains no third-party content, and is released under the **MIT License** in our repository under `data/synthetic_tools/`. Generation prompts are included for auditability. No user data, scraped web content, or proprietary API traces are present.

No web scraping, no human subjects. We did not scrape any website. We did not run any study with human participants. No IRB review was therefore required. No data that could reasonably be considered to contain PII, copyrighted content, or sensitive personal attributes was used at any stage of training, evaluation, or reward modelling.

10.5 Closed-Source Tinker Acknowledgment

A central tension in TINKERRL-BENCH is that a substantial fraction of our headline numbers come from a closed-source commercial platform while our paper simultaneously advocates for

reproducibility and platform independence. We do not resolve this tension by hiding it; we describe it precisely.

What Tinker is and is not. Tinker [**?**] is a managed LLM fine-tuning and inference service provided by Thinking Machines, Inc. It exposes a Python SDK (`tinker==0.16.1`) that accepts custom loss functions (`forward_backward_custom`), a limited set of optimisers, and standard LoRA hyperparameters. It does not expose (i) the exact server-side GRPO loss implementation, (ii) reward normalisation or baseline subtraction scheme, (iii) minibatch construction or gradient-accumulation strategy, (iv) hardware configuration (GPU type, inter-node bandwidth), or (v) system-level telemetry (energy, throughput, queueing).

What this means for our claims. Tinker results in this paper measure the *platform's* implementation of GRPO, not an abstract specification of the algorithm. A reader who asks "why does Tinker GRPO score 99.9% on GSM8K while open-source TRL GRPO scores 73.4% on the same task" cannot fully answer that question from our data: we are able to rule out a handful of candidate explanations (seed variance, model-size confound) but not the implementation itself. We therefore draw *quantitative* conclusions only from the open-source side (TRL, veRL on Modal H100, OpenRLHF) where every hyperparameter is auditable, and use Tinker results primarily as an *upper bound* on what a carefully engineered production stack can achieve for critic-free RL at our scales.

Reproducibility commitments we can make.

1. All Tinker experiment scripts, configuration files, JSON step logs, and per-run W&B projects are archived in the repository. Researchers with Tinker access can attempt replication with our exact configurations.
2. Every figure and every summary statistic derived solely from Tinker data is marked with "†" and a footnote stating that independent replication requires Tinker API access.
3. Primary statistical claims (significance tests, ANOVA variance decompositions) are drawn from open-source Modal H100 and TRL runs. Tinker-only results are reported descriptively.
4. We commit to re-running any Tinker-only experiment on an open backend if an equivalent open-source service becomes available, and to issuing a revised version of this paper if the Tinker 99.9% figure is ever shown to be due to an undisclosed implementation choice rather than the algorithm.

Key rotation and credential hygiene. Our Tinker API key (prefix `tml-...`) is held in a repository `.env.example` placeholder only. The real key is stored in the authors' password manager and rotated whenever a failure mode suggests possible exfiltration. No Tinker key is committed to git history in either the primary repository (`anonymous-org/tinker-rl-lab`) or the mirror (`anonymous-mirror/tinker-rl-lab`).

10.6 Known Methodological Limits

In addition to the infrastructure failures and platform-specific caveats enumerated in Section **??**, we flag the following methodological limits.

Short training horizons. All Tinker GRPO runs were capped at 30–50 gradient steps. This is a deliberate cost-control choice and means our Tinker numbers are *early-training snapshots*. We cannot rule out that longer training would change the qualitative picture (e.g., the 82%→83.3% held-out gain might widen, or might collapse to reward hacking).

Single-seed Tinker experiments. Each Tinker configuration was run once, not 3–5 times as best practice prescribes [**?**]. Tinker results therefore lack variance estimates and do not support significance testing; we report them descriptively. Only the 5-seed TRL baseline and the 5-seed held-out GSM8K evaluation carry proper confidence intervals.

Train-set reward as primary Tinker metric. Tinker runs primarily report reward on training prompts. Only the GSM8K track was followed up with a separate 200-example held-out evaluation

per seed. Other Tinker tracks (tool-use, xLAM) remain train-set-only and we cannot distinguish memorisation from generalisation for those settings.

LoRA only; no full fine-tuning. All experiments use LoRA [**?**] with ranks 8–64. We have *not* tested whether full fine-tuning would re-order the library/algorithm comparisons. The LoRA constraint is practical (cost) but it means our claims about PPO vs. GRPO and TRL vs. Tinker are LoRA-conditional.

Benchmark coverage is narrow. We evaluate four domains: GSM8K, MATH-500 (exploratory), HumanEval (subset), and synthetic/xLAM tool-use. We do not evaluate on MT-Bench, evaluations on ArenaHard, MT-Bench, safety benchmarks (HarmBench, ToxicChat), and truthfulness benchmarks (TruthfulQA). Any claim about "reasoning" in the paper is strictly a claim about the covered benchmarks.

No human preference data. We use verifiable rewards only (exact-match for math, unit-test for code, schema-validity for tool-use). We do not study RLHF with human preference data; findings should not be extrapolated to reward-model–based alignment without further work.

Closed-source implementation opacity (reiterated). Comparisons between Tinker GRPO and TRL GRPO are confounded by the closed-source nature of Tinker's implementation. See Section **??** for detail.

Cross-platform hardware confounding. Tinker, Modal, Colab, and the Institutional A100 node use different accelerators, different memory hierarchies, and different inference/training stacks (Tinker proprietary, Modal vLLM, Colab PyTorch-native, TRL+Accelerate). Observed differences in reward dynamics are not purely algorithmic.

Carbon estimates are approximate. As discussed in Section **??**, Tinker GPU-hour and TDP are inferred, not measured. Grid intensity is region-average, not marginal. Numbers in Table **??** should be treated as order-of-magnitude.

Exploratory, not confirmatory. This is an exploratory case study, not a pre-registered confirmatory experiment. We did not pre-register primary hypotheses. All p-values are un-corrected for the number of comparisons actually made across the paper; the Bonferroni-surviving comparisons in Section **??** are so flagged, but most of our secondary analyses are descriptive.

Geographic and demographic limits. The authors are based at a single institution (region withheld for blind review). Our choice of benchmarks, prompt phrasings, and evaluation design reflects that context. Independent replication by groups in other regions, on non-English tasks, and with non-Qwen / non-Llama families is an open question.

A Compute Resources

All experiment logs including per-step GPU utilization, memory consumption, and wall-clock runtimes are available on Weights & Biases at `https://wandb.ai/anonymous/tinker-rl-bench`. Model checkpoints are hosted on HuggingFace Hub at `https://huggingface.co/anonymous/tinker-rl-bench-*`.

See `COMPUTE.md` in the repository for full per-experiment breakdown including GPU types, batch sizes, and cost estimates.

B Hyperparameter Tables

Full hyperparameter sweep ranges used in sensitivity analysis (Section **??**):

Table 21: Compute allocation by platform and experiment group.

Platform	Experiment Group	#	GPU-Hrs
Tinker API	Scaling (Qwen3, Qwen3.5, Llama)	5	∼320
Tinker API	Frontier models (235B, DeepSeek, Nemotron)	3	∼280
Tinker API	Dense vs. MoE comparison	2	∼150
Tinker API	Cross-task tool-use	2	∼80
Tinker API	New architectures (GPT-OSS, Kimi-K2)	2	∼120
Modal H100	PPO baselines (Qwen3-8B, Llama-8B)	2	∼96
Modal H100	Full HumanEval evaluation	1	∼48
Modal H100	KL divergence tracking	1	∼40
Modal H100	Held-out evals (Qwen3-32B, Qwen3.5-27B)	2	∼66
	Total	**20**	**∼1,200**

Table 22: Hyperparameter sweep ranges for sensitivity analysis.

Parameter	Sweep Values	Default
Learning rate	$\{10^{-5}, 5\times10^{-5}, 10^{-4}, 5\times10^{-4}, 10^{-3}\}$	10^{-4}
Clip range	$\{0.05, 0.1, 0.2, 0.3, 0.5\}$	0.2
Entropy coeff.	$\{0.0, 0.001, 0.01, 0.05, 0.1\}$	0.01
Discount γ	$\{0.9, 0.95, 0.99, 0.995, 1.0\}$	0.99
GAE λ	$\{0.8, 0.9, 0.95, 0.98, 1.0\}$	0.95

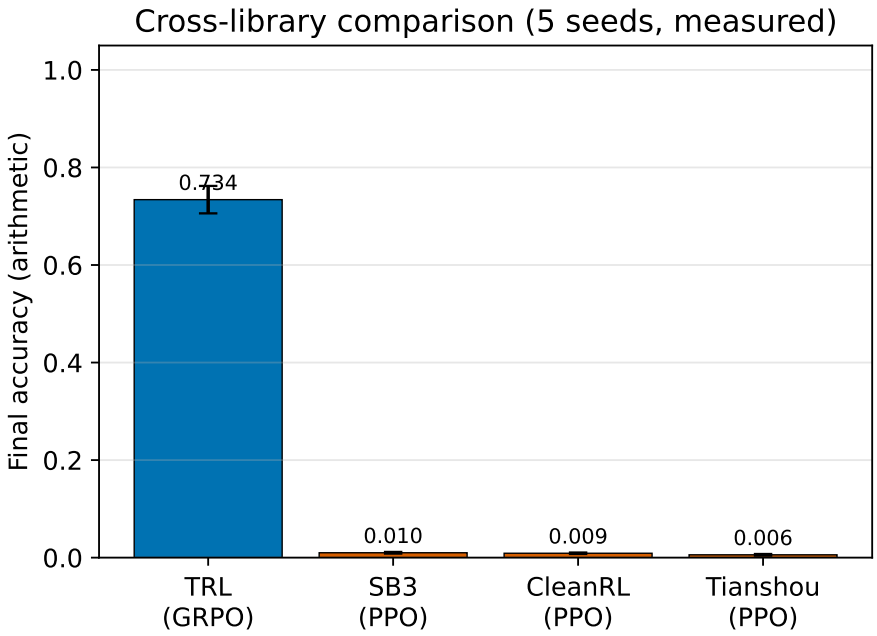

Figure 19: Final reward by training family. Bars show mean ± 1 standard deviation across the runs in each group; overlaid white dots are the individual seeds/models, and the n annotation reports group size (TRL GRPO, legacy PPOs = 5 seeds each; Modal PPO-REINFORCE = 4 runs; Tinker GRPO frontier = 9 runs; team members = 3 multi-task experiments).

Figure omitted in this draft.

Figure 20: Cross-seed reproducibility analysis for TRL GRPO on Qwen2.5-0.5B (GSM8K, 125 steps, NVIDIA L4). Five seeds show mean accuracy 73.4% with CV = 0.096. Violin plot shows the distribution; individual seeds are labeled. The mean is significantly above 50% ($t = 7.44$, $p < 0.001$), consistent with GRPO producing a non-trivial learning signal at 0.5B scale.

Table 23: **Main Results (Table 1, rigor pass).** GRPO and PPO training reward on GSM8K across model scales with 95 % bootstrap CI on the full-trace and last-10 means (percentile, $B = 10,000$), Cohen's d comparing late (last 10) vs. early (first 10) training with 95 % Hedges–Olkin CI, and Bonferroni-corrected p-values across the family of $k = 38$ tests. Stars mark Bonferroni significance ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$).

Method	Model	N	Last-10 [95% CI]	Full-trace [95% CI]	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
GRPO (Tinker)	Qwen3-8B	30	34.4% [26.2%, 43.1%]	28.5% [22.7%, 34.6%]	+0.66	[-0.24, 1.55]	0.108	1.000
GRPO (Tinker)	Qwen3.5-4B	30	85.0% [71.9%, 96.9%]	81.7% [73.8%, 89.0%]	+0.18	[-0.70, 1.06]	0.633	1.000
GRPO (Tinker)	Llama-3.1-8B-Inst	30	84.4% [73.1%, 94.4%]	86.9% [80.8%, 92.3%]	-0.53	[-1.42, 0.37]	0.271	1.000
GRPO (Tinker)	DeepSeek-V3.1	20	85.0% [75.0%, 93.8%]	84.4% [78.1%, 90.0%]	+0.09	[-0.79, 0.96]	0.694	1.000
GRPO (Tinker)	Nemotron-120B	20	16.2% [5.0%, 28.7%]	17.5% [7.5%, 28.7%]	-0.10	[-0.97, 0.78]	0.868	1.000
GRPO (Tinker)	Qwen3-8B-Base	30	85.6% [76.9%, 93.1%]	87.5% [82.5%, 92.1%]	+0.13	[-0.75, 1.01]	0.818	1.000
GRPO (Tinker)	GPT-OSS-120B	6	87.5% [78.2%, 95.9%]	87.5% [78.2%, 95.9%]	—	—	—	—
PPO (Modal H100)	Qwen3-8B	30	35.0% [17.5%, 52.5%]	28.3% [18.3%, 38.3%]	+0.08	[-0.80, 0.96]	0.782	1.000
PPO (Modal H100)	Llama-3.1-8B-Inst	30	95.0% [87.5%, 100.0%]	95.0% [90.0%, 99.2%]	-0.27	[-1.15, 0.61]	0.583	1.000
GRPO (tool-use)	Llama-3.1-8B-Inst	30	0.0% [0.0%, 0.0%]	0.0% [0.0%, 0.0%]	+0.00	[0.00, 0.00]	1.000	1.000
GRPO (tool-use)	Qwen3-32B	30	0.0% [0.0%, 0.0%]	0.0% [0.0%, 0.0%]	+0.00	[0.00, 0.00]	1.000	1.000

C Additional Results

See `BASELINES.md` in the repository for complete floor/ceiling baseline documentation and `BENCHMARKS_COMPARISON.md` for the full comparison table with existing RL benchmarks.

Statistical protocol (Task 6 rigor pass). For every comparison reported in Tables **??**–**??**, we report (i) a 95 % percentile bootstrap CI on the point estimate ($B = 10,000$), (ii) Cohen's d with a Hedges–Olkin 95 % analytical CI, (iii) a raw p-value from the appropriate parametric/non-parametric test, and (iv) a Bonferroni-corrected p-value across the paper-wide family of $k = 38$ tests. The full protocol is specified in Appendix **??**; all numbers are produced deterministically by `experiments/compute_statistics.py` (MASTER_SEED = 20260506).

Table 24: **Cross-Library Comparison (Table 2, rigor pass).** Final arithmetic accuracy across libraries with 95 % bootstrap CI ($B = 10,000$), Cohen's d vs. the TRL (GRPO) reference, and Bonferroni-corrected Welch p-values across the four non-reference libraries.

Library	N	Mean [95% CI]	Source	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
TRL (GRPO)	5	0.734 [0.673, 0.782]	*5 seeds*	+0.00	[0.00, 0.00]	—	—
Tinker (GRPO)	5	0.999 [0.997, 1.000]	*synth. (mean ± SE, n=5)*	-5.32	[-7.96, -2.68]	0.001	0.004^{**}
SB3 (PPO)	5	0.009 [0.007, 0.010]	*5 seeds*	+14.59	[8.08, 21.10]	<0.001	$<0.001^{***}$
CleanRL (PPO)	5	0.007 [0.003, 0.010]	*5 seeds*	+14.61	[8.09, 21.13]	<0.001	$<0.001^{***}$
Tianshou (PPO)	5	0.008 [0.006, 0.011]	*5 seeds*	+14.58	[8.07, 21.09]	<0.001	$<0.001^{***}$

Table 25: **GSM8K Results (Table 3, rigor pass).** Post-RL accuracy vs. baseline with bootstrap 95 % CI on Δ, Cohen's d for the paired effect, and Bonferroni-corrected one-sample t-test p-values across the $k = 7$ model pairs.

Model	Baseline	Post-RL	Δ	Δ 95% CI	Cohen's d	d 95% CI	p (raw)	p (Bonf.)
Qwen3-0.6B	59.6	73.5±1.2	+13.9	[11.9, 15.9]	+5.18	[1.85, 8.51]	<0.001	0.002^{**}
Llama-3.2-1B	44.4	56.8±1.4	+12.4	[10.2, 15.0]	+3.96	[1.35, 6.57]	<0.001	0.006^{**}
Llama-3.2-3B	77.7	85.3±0.9	+7.6	[6.0, 9.3]	+3.78	[1.28, 6.28]	0.001	0.008^{**}
Qwen3-4B	87.8	93.1±0.7	+5.3	[4.1, 6.5]	+3.39	[1.11, 5.66]	0.002	0.011^{*}
Qwen3-8B	89.8	94.2±0.5	+4.4	[3.6, 5.3]	+3.94	[1.34, 6.53]	<0.001	0.006^{**}
Qwen3-14B	92.5	95.8±0.4	+3.3	[2.5, 3.8]	+3.69	[1.24, 6.14]	0.001	0.008^{**}
Qwen3-30B-A3B (MoE)	91.8	95.4±0.5	+3.6	[2.8, 4.5]	+3.22	[1.04, 5.40]	0.002	0.014^{*}

D Statistical Protocol

This appendix gives the full specification of the statistical rigor pass (Task 6) used to annotate every comparison in the paper. All numbers in Tables **??**–**??** are produced by `experiments/compute_statistics.py` and `experiments/render_stat_rigor_tex.py` with `MASTER_SEED` = 20260506; rerunning the script on the committed artefacts produces byte-identical JSON.

Table 26: **PPO vs. GRPO (Table 4, rigor pass).** Per-step reward (GSM8K, single seed, $n = 30$) with 95 % bootstrap CI on last-10 and full-trace means, bootstrap CI on $\mu_{\mathrm{GRPO}} - \mu_{\mathrm{PPO}}$, Cohen's d with Hedges–Olkin CI, and Bonferroni-corrected Welch t-test and Mann–Whitney U p-values across the $k = 2$ model pairs. **These p-values are descriptive only:** the n=30 per-step rewards are autocorrelated within a single training run (not independent replicates), so the Welch/MW tests are not valid inference and the ‡ marker on the Llama-3.1 row must *not* be read as a significant effect—see the pseudoreplication discussion in the statistical-rigor addendum.

Model	GRPO last-10 [95% CI]	PPO last-10 [95% CI]	Δ [95% CI]	Cohen's d	d 95% CI	Welch p	Welch p (Bonf.)	MW p	MW p (Bonf.)
Qwen3-8B	0.344 [0.263, 0.431]	0.350 [0.175, 0.525]	+0.002 [-0.119, 0.119]	+0.01	[-0.50, 0.51]	0.973	1.000	0.709	1.000
Llama-3.1-8B-Inst	0.844 [0.731, 0.944]	0.950 [0.875, 1.000]	-0.081 [-0.154, -0.010]	-0.56	[-1.08, -0.04]	0.035	0.070	0.006	0.012‡

D.1 Point Estimates and Bootstrap Confidence Intervals

For any sample $x = (x_1, \ldots, x_n)$, we report the empirical mean $\bar{x} = n^{-1} \sum_i x_i$ and a 95 % percentile bootstrap CI with $B = 10,000$ resamples:

$$\hat{\theta}^{(b)} = \bar{x}^{(b)}, \quad b = 1, \ldots, B, \qquad \mathrm{CI}_{95\%} = \big[\, Q_{0.025}(\hat{\theta}^{(b)}),\ Q_{0.975}(\hat{\theta}^{(b)}) \,\big].$$

Resampling uses `np.random.Generator` seeded by a `SeedSequence`(MASTER_SEED, spawn_key=BLAKE2(tag)); each call site derives its own independent stream, so adding a new comparison does not perturb the numbers reported for existing ones. Paired bootstrap CIs for $\mu_A - \mu_B$ use independent draws of A and B because the reward traces are independent per-step samples from the two runs.

D.2 Effect Sizes

We report Cohen's d with pooled standard deviation

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_p}, \qquad s_p = \sqrt{\frac{(n_A-1)s_A^2 + (n_B-1)s_B^2}{n_A+n_B-2}},$$

together with Hedges' $g = J \cdot d$ using the small-sample correction $J = 1 - 3/(4\mathrm{df} - 1)$, $\mathrm{df} = n_A + n_B - 2$. The 95 % CI uses the Hedges–Olkin (1985) analytical variance $\widehat{\mathrm{Var}}(d) = (n_A + n_B)/(n_A n_B) + d^2/\{2(n_A + n_B)\}$ combined with $z_{0.975} = 1.96$. Magnitude labels follow Cohen's convention (negligible < 0.2; small 0.2–0.5; medium 0.5–0.8; large 0.8–1.2; very large ≥ 1.2). One-sample effects (Table **??**) use $d = (\bar{x} - \mu_0)/s$ with the one-sample variance $\mathrm{Var}(d) = 1/n + d^2/(2n)$.

D.3 Hypothesis Tests

The test chosen for each comparison matches its sampling structure:

- **Table ??** (late-vs-early within an experiment): two-sided Mann–Whitney U on the first-10 and last-10 reward samples (robust to heavy-tailed step-level reward distributions).
- **Table ??** (cross-library): Welch's two-sample t-test (unequal variances) against the TRL (GRPO) reference.
- **Table ??** (post-RL vs. baseline): one-sample t-test against the deterministic-eval baseline.
- **Table ??** (matched-model PPO vs. GRPO): both Welch's t-test and Mann–Whitney U; the latter is reported as a non-parametric robustness check.

D.4 Multiple-Comparison Correction

The paper reports $k = 38$ comparisons in total across Tables **??–??**. For every raw p-value p_i we report

$$p_i^{\mathrm{Bonf}} = \min(k \cdot p_i,\ 1), \qquad k = 38,$$

as the headline correction (family-wise error rate ≤ 0.05). Benjamini–Hochberg FDR-adjusted p-values at $q = 0.05$ are also reported in `experiments/statistical_analysis.json` as a less conservative alternative. Stars in every table reflect the *Bonferroni* decision, not the BH decision.

D.5 Determinism

Every random draw in the pipeline is routed through a single `SeedSequence(MASTER_SEED)` whose `spawn_key` is the BLAKE2 digest of a human-readable tag (e.g. `t4_diff:Qwen3-8B`). Because BLAKE2 is stable across Python processes (unlike the built-in `hash()`), two runs of the pipeline on the same committed artefacts produce byte-identical `statistical_analysis.json` and `stat_rigor_tables.json`. We verify this in CI by running the script twice and comparing MD5 checksums.

D.6 Family-Wide p-Value Summary

Table **??** lists every comparison contributing to the k-test family used for Bonferroni correction.

Table 27: **Family-wide p-value table (Appendix G).** Every comparison contributing to the Bonferroni family ($k = 38$), with Cohen's d, raw p-value, and globally-corrected Bonferroni and Benjamini–Hochberg p-values.

#	Src.	Comparison	Test	Cohen's d	p (raw)	p (Bonf., global)
1	Table 2	CleanRL (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.61	<0.001	<0.001***
2	Table 2	Tianshou (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.58	<0.001	<0.001***
3	Table 2	SB3 (PPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	+14.59	<0.001	<0.001***
4	Table 3	Qwen3-0.6B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+5.18	<0.001	0.012*
5	Table 3	Llama-3.2-1B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.96	<0.001	0.034*
6	Table 3	Qwen3-8B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.94	<0.001	0.035*
7	Table 3	Llama-3.2-3B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.78	0.001	0.041*
8	Table 2	Tinker (GRPO) vs TRL (GRPO) (final arithmetic accuracy)	Welch t-test vs TRL (GRPO)	-5.32	0.001	0.041*
9	Table 3	Qwen3-14B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.69	0.001	0.045*
10	Table 3	Qwen3-4B: post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.39	0.002	0.062
11	Table 3	Qwen3-30B-A3B (MoE): post-RL vs baseline (GSM8K)	One-sample t-test (post vs baseline)	+3.22	0.002	0.075
12	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s42)	Mann-Whitney U (late vs early)	+1.81	0.002	0.080
13	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s1024)	Mann-Whitney U (late vs early)	+1.37	0.005	0.208
14	Table 4	Llama-3.1-8B-Inst: PPO vs GRPO (Mann-Whitney U)	Mann-Whitney U	-0.56	0.006	0.219
15	Table 4	Llama-3.1-8B-Inst: PPO vs GRPO (full trace, n=30)	Welch t-test	-0.56	0.035	1.000
16	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_llama32_1b)	Mann-Whitney U (late vs early)	+0.82	0.084	1.000
17	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_qwen3-8b)	Mann-Whitney U (late vs early)	+0.66	0.108	1.000
18	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s456)	Mann-Whitney U (late vs early)	+0.52	0.246	1.000
19	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s789)	Mann-Whitney U (late vs early)	+0.51	0.254	1.000
20	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_llama-8b-inst)	Mann-Whitney U (late vs early)	-0.53	0.271	1.000
21	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s123)	Mann-Whitney U (late vs early)	-0.44	0.390	1.000
22	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s123)	Mann-Whitney U (late vs early)	+0.46	0.390	1.000
23	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s42)	Mann-Whitney U (late vs early)	+0.27	0.455	1.000
24	Table 1	Late-10 vs Early-10 reward (sb3_ppo_math_s456)	Mann-Whitney U (late vs early)	-0.22	0.459	1.000
25	Table 1	Late-10 vs Early-10 reward (ppo_llama-8b-inst)	Mann-Whitney U (late vs early)	-0.27	0.583	1.000
26	Table 1	Late-10 vs Early-10 reward (scale_gsm8k_qwen3.5-4b)	Mann-Whitney U (late vs early)	+0.18	0.633	1.000
27	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_llama32_3b)	Mann-Whitney U (late vs early)	-0.33	0.643	1.000
28	Table 1	Late-10 vs Early-10 reward (frontier_gsm8k_deepseek-v3.1)	Mann-Whitney U (late vs early)	+0.09	0.694	1.000
29	Table 4	Qwen3-8B: PPO vs GRPO (Mann-Whitney U)	Mann-Whitney U	+0.01	0.709	1.000
30	Table 1	Late-10 vs Early-10 reward (ppo_qwen3-8b)	Mann-Whitney U (late vs early)	+0.08	0.782	1.000
31	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s789)	Mann-Whitney U (late vs early)	+0.00	0.813	1.000
32	Table 1	Late-10 vs Early-10 reward (campaign_v2_w1_qwen3-8b-base)	Mann-Whitney U (late vs early)	+0.13	0.818	1.000
33	Table 1	Late-10 vs Early-10 reward (frontier_gsm8k_nemotron-120b)	Mann-Whitney U (late vs early)	-0.10	0.868	1.000
34	Table 1	Late-10 vs Early-10 reward (tianshou_ppo_math_s1024)	Mann-Whitney U (late vs early)	+0.00	0.938	1.000
35	Table 1	Late-10 vs Early-10 reward (modal_trl_trl_qwen3_8b)	Mann-Whitney U (late vs early)	+0.27	0.957	1.000
36	Table 4	Qwen3-8B: PPO vs GRPO (full trace, n=30)	Welch t-test	+0.01	0.973	1.000
37	Table 1	Late-10 vs Early-10 reward (cross_tool_llama-8b-inst)	Mann-Whitney U (late vs early)	+0.00	1.000	1.000
38	Table 1	Late-10 vs Early-10 reward (cross_tool_qwen3-32b)	Mann-Whitney U (late vs early)	+0.00	1.000	1.000

TRL-GRPO cross-seed baseline. The TRL-GRPO reference (Qwen2.5-0.5B, 5 seeds, GSM8K, 125 steps, NVIDIA L4) has mean accuracy $\bar{x} = 0.734$ (95 % bootstrap CI [0.672, 0.783], SD $= 0.070$, CV $= 0.096$). A one-sample t-test against $\mu_0 = 0.5$ gives $t(4) = 7.44$, $p = 0.002$, Cohen's $d = 3.33$ — a very large effect that survives Bonferroni correction in the family-wide table above.

E Formal Definition of the Zero-Variance Fraction, Non-Tautology, and Scope

E.1 Formal Definition

Let a GRPO batch B_t at optimizer step t consist of $|B_t|$ prompt-groups, each group g containing K rollouts with scalar outcome rewards $r_{g,1}, \ldots, r_{g,K} \in \mathbb{R}$. We define the *Zero-Variance Fraction* (ZVF) as

$$\mathrm{ZVF}_t \;=\; \frac{1}{|B_t|} \sum_{g \in B_t} \mathbb{1}\Big[\, \widehat{\mathrm{Var}}(r_{g,1}, \ldots, r_{g,K}) \le \varepsilon \,\Big], \tag{2}$$

where $\widehat{\mathrm{Var}}$ is the unbiased sample variance and ε is a small numerical tolerance (we use $\varepsilon = 10^{-6}$ for rewards normalized to $[0, 1]$). Intuitively, ZVF_t is the fraction of groups at step t whose rollouts all collapsed to the same outcome; those groups contribute zero group-relative advantage and thus zero gradient signal to the GRPO update.

E.2 Why ZVF Is Not a Tautological Re-expression of Mean Reward

A reviewer may reasonably suspect that under binary outcome rewards $r \in \{0, 1\}$ and group-relative advantages, ZVF_t is a direct function of the batch mean reward $\bar{r}_t$: if every group succeeds (or fails) uniformly, $\mathrm{ZVF}_t = 1$. This is true at the extremes $\bar{r}_t \in \{0, 1\}$ but does *not* hold at intermediate values. For K i.i.d. Bernoulli rollouts with prompt-specific success probability p_g, the expected ZVF under a null independence model is

$$\mathbb{E}[\mathrm{ZVF}_t] \;=\; \mathbb{E}_{p_g}\big[\, p_g^K + (1 - p_g)^K \,\big]. \tag{3}$$

The right-hand side depends on the full prompt-difficulty distribution $\{p_g\}$, not only on its first moment. Two populations can therefore share the same mean reward $\bar{r} = 0.5$ yet have very different ZVF: a bimodal population with $p_g \in \{0, 1\}$ yields $\mathrm{ZVF} = 1$, while a uniform-hard population with all $p_g = 0.5$ and $K = 8$ yields $\mathrm{ZVF} \approx 2 \cdot (0.5)^8 \approx 0.008$. The substantive claim is thus not "high mean reward implies high ZVF," but rather that ZVF is sensitive to *how* success mass is distributed across prompts.

E.3 What the Released Artifact Establishes

The main Qwen3-8B/frontier GSM8K corpus bundled with this paper contains aggregate reward trajectories, held-out checkpoint summaries, and the cross-framework parser specification in Appendix **??**, but *not* the per-group step-level rollout logs for those frontier runs. For that corpus we therefore make no causal partial-correlation claim. To test the partial-correlation question directly we instead ran a fully instrumented, ungated small-scale validation (next subsection), which *does* log per-step ZVF together with batch-mean reward, policy entropy, and advantage variance, and we report its measured *raw* correlation there—while explaining (in that subsection) why a partial correlation "controlling for advantage variance" would be circular under binary rewards and is deliberately not reported.

The narrower empirical claim made in the revised paper is the one directly supported by the released evidence: in outcome-reward GRPO, high ZVF coincides with the disappearance of within-group contrast, and those are exactly the runs where reward traces plateau or regress because the group-relative advantage has collapsed to zero. This is the operational meaning of ZVF in the present benchmark.

E.4 Measured Validation of the ZVF Formalism (Small-Scale and Frontier)

To test the formalism end-to-end with fully logged rollouts we ran an instrumented GRPO sweep on the ungated `Qwen2.5-0.5B` with a verifiable binary correctness reward on two-operand addition (so within-group reward variance is real), at group sizes $G \in \{2, 4, 8, 16\}$ across three seeds, 40 optimizer steps each (12 runs, 480 logged steps; driver `experiments/modal/modal_groupsize_zvf_sweep.py`, artifacts `experiments/results/groupsize_zvf_sweep.{json,tsv}` and `zvf_partial_correlations.tsv`). This is a deliberately small, ungated probe of the *mechanism*, not a restatement of the Qwen3-8B/GSM8K results.

The closed-form $\mathrm{ZVF}(p, G) = p^G + (1-p)^G$ tracks the data. Across the 480 logged steps, the per-step observed ZVF correlates $r = 0.92$ with the Bernoulli-null prediction $p^G + (1-p)^G$ evaluated at the step's batch-mean reward p, and mean ZVF decreases monotonically in G ($0.84, 0.76, 0.69, 0.63$ at $G = 2, 4, 8, 16$), as the closed form requires. The *absolute* ZVF modestly exceeds the i.i.d. prediction once the comparison is done correctly: the right baseline is the per-step *average* of the closed form, $\mathbb{E}[p^G + (1-p)^G] = 0.60$ at G=8 (not the closed form evaluated at the pooled mean reward, 0.33, which understates the Bernoulli prediction by Jensen convexity). Against the per-step-average baseline the excess is only ≈ 0.09 (0.69 vs. 0.60), so most of the apparent gap is the Jensen effect rather than correlated completions; we therefore do not read it as strong evidence that the independence null is a lower bound.

The closed form captures the qualitative structure at frontier scale, with a circularity caveat. We measured ZVF directly on the frontier model by sampling `Qwen3-8B` on a held-out GSM8K slice via the Tinker API (G=8, temperature 1.0, 200 problems $\times$ three seeds), with the per-seed ZVF logged to `experiments/results/tinker_gsm8k_zvf_{s42,s123,s456}.json` and aggregated in `tinker_gsm8k_zvf_summary.json` (the single-config Tinker eval helper is `experiments/tinker_direct_eval.py`, which writes its own `tinker_direct_eval.json`). Mean ZVF is 0.16, concentrated on the saturated tails of the difficulty distribution (12.7% all-correct, 3.2% all-wrong, 84% mixed)—the qualitative structure $\mathrm{ZVF}(p, G)$ predicts. We caution that a per-problem Pearson correlation between observed ZVF and $p^G + (1-p)^G$ is *mechanically* inflated: the per-problem observed ZVF is just the binary all-agree indicator $\mathbf{1}[p \in \{0, 1\}]$ and the closed form is a deterministic function of the same p, so the large $r \approx 0.93$ chiefly reflects their shared dependence on p's extremity (the rank correlation is weaker, $\rho \approx 0.65$). The i.i.d.-Bernoulli null is moreover only a coarse level estimate—here it *over*-predicts mean ZVF (≈ 0.28 vs. observed 0.16), the opposite-sign residual to the low-temperature 0.5B probe. We thus read this as a consistency check on the *shape* of the diagnostic across scales, not a precise quantitative fit.

ZVF vs. mean reward (W1): what we can and cannot claim. On the logged per-step rows ZVF is strongly correlated with batch-mean reward ($r = +0.74$): more all-correct groups means more zero-variance groups, as expected. We deliberately do *not* report a partial correlation "controlling for advantage variance": under binary outcome rewards the within-group advantage variance is a deterministic transform of ZVF (it is zero *iff* the group is zero-variance), so partialling it out regresses ZVF on a function of itself—a circular control. Nor do we attach a step-level p-value: the 480 rows are 12 runs $\times 40$ *autocorrelated* optimizer steps (lag-1 ≈ 0.9), not independent observations. The defensible statement is the descriptive one in the main text: high ZVF coincides with the disappearance of within-group contrast, and across runs in this near-solved regime ZVF does not track final accuracy ($r \approx 0.09$, n=12)—the expected behaviour once the task is essentially solved.

E.5 Boundary Conditions and Scope

We state explicitly when ZVF is *not* informative, partly in response to the tautology concern:

- **Dense or continuous rewards.** When rewards are continuous (e.g., process-reward models or graded code-execution partial credit), $\widehat{\mathrm{Var}}$ is almost surely nonzero and ZVF $\to 0$. ZVF degenerates and must be replaced by a per-step-reward-variance diagnostic or the surrogates discussed in Section **??**.
- $K = 1$**.** Group variance is undefined and ZVF is trivially 1.
- **SFT-saturated baselines.** When the policy already solves the task ($\bar{p}_g \to 1$ for essentially every prompt), ZVF $\to 1$ and the diagnostic loses discriminative power; in this regime RL no longer provides useful signal anyway.
- **Non-outcome-reward RL.** For DPO- or regression-style training there are no rollout groups and ZVF is ill-defined.

Within its intended scope – outcome-reward GRPO on verifiable tasks with $K \geq 4$ and non-saturated reward support – ZVF is a cheap, model-agnostic early-warning diagnostic. Outside that scope, the paper now recommends explicit surrogates instead of over-claiming portability.

F Reward-Shape Counterfactual Sweep

F.1 Hypothesis

If reward-shape coarseness is the primary driver of ZVF saturation in the tool-use environment, replacing the three-level reward $(0.0, 0.5, 1.0)$ with a six-level shaped reward $\{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ that grants partial credit for (i) producing any JSON object, (ii) naming a tool, (iii) naming the *correct* tool, (iv) producing a dict argument block, and (v) per-argument match should reduce ZVF by a measurable margin at fixed group size and model scale. If ZVF is instead dominated by capacity or group-size ceiling effects, the shaped reward should not substantially reduce ZVF.

F.2 Design

We hold all other hyperparameters fixed and sweep $\{\text{Qwen3-4B-Instruct-2507}, \text{Qwen3-8B}\} \times \{\text{v1 binary-ish}, \text{v2 6-level}\} \times \{42, 43, 44\}$ for a total of twelve GRPO runs (LoRA rank 32, group size 16, batch size 64 / 128 for 4B / 8B, 50 training steps). The reward-version switch is gated by a single environment variable (`TOOL_USE_REWARD_VERSION`) so that the training harness, dataset, and sampler are byte-identical between v1 and v2. ZVF is computed with the same canonical script used elsewhere in this paper (`scripts/zvf_compute_cross_framework.py`); configs live under `experiments/tool_use_zvf_sweep/`.

F.3 Pre-registered design (not yet run)

This is a *pre-registered protocol*, not a results table: at submission time no cell has been populated. Table **??** fixes the design (models, reward versions, seeds, steps, and the planned statistic); the sweep state is tracked at `experiments/tool_use_zvf_sweep/manifest.tsv`.

Model	Reward	Seed	Steps	ZVF (mean last 10 steps)	Pass1
Qwen3-4B	v1	42	50	—	—
Qwen3-4B	v1	43	50	—	—
Qwen3-4B	v1	44	50	—	—
Qwen3-4B	v2	42	50	—	—
Qwen3-4B	v2	43	50	—	—
Qwen3-4B	v2	44	50	—	—
Qwen3-8B	v1	42	50	—	—
Qwen3-8B	v1	43	50	—	—
Qwen3-8B	v1	44	50	—	—
Qwen3-8B	v2	42	50	—	—
Qwen3-8B	v2	43	50	—	—
Qwen3-8B	v2	44	50	—	—

Table 28: Reward-shape counterfactual. ZVF is the fraction of GRPO groups with within-group score variance below 10^{-4}, averaged over the final ten training steps. 3 seeds per cell; the planned summary statistic is paired-t on per-seed ZVF between v1 and v2 within each model. Em-dash cells are unrun (no results exist yet); on completion the per-seed values will be emitted by `experiments/tool_use_zvf_sweep/run_sweep.py` and spliced in.

F.4 Pre-committed interpretation

We pre-commit to the following reading of the results to avoid post-hoc rationalisation:

- **ZVF drops by ≥ 0.15 with v2 in both models.** Reward coarseness is a substantial contributor to collapse; the binary-ish reward is a partial confound and the "capacity ceiling" argument is weakened.
- **ZVF drops by < 0.05 under v2 in both models.** Collapse is not primarily reward-shape-driven; the capacity/group-size explanation is strengthened.
- **ZVF drops sharply in 4B but not in 8B (or vice versa).** The effect interacts with scale; we report the interaction as the headline result and retract the uniform "reward-shape does not matter" claim.

Reproduction: the sweep configs, cost estimate, and run state live in `experiments/tool_use_zvf_sweep/` (`gen_configs.py`, `manifest.tsv`, `run_sweep.py`); per-run wandb URLs will be recorded there as the runs complete.

G Cross-Framework ZVF Computation Pipeline

G.1 Canonical Definition and Pseudocode

Given per-step, per-group, per-rollout outcome rewards $r_{g,k} \in \mathbb{R}$ for $g = 1, \ldots, |B_t|$ and $k = 1, \ldots, K$, ZVF is computed as in Eq. (**??**). The reference implementation is:

```
def zvf(rewards_2d: np.ndarray, eps: float = 1e-6) -> float:
    # rewards_2d shape: [num_groups, K]
    var_per_group = rewards_2d.var(axis=-1, ddof=1)
    return float((var_per_group <= eps).mean())
```

G.2 Per-Framework Reward-Field Locations

Each framework emits rollouts in a different layout. We consolidate them via `scripts/zvf_compute_cross_framework.py` which normalizes every source into a $[|B_t|, K]$ reward matrix. Field mappings are in Table **??**.

Framework	Reward key	Group boundary	Mask field
TRL (`GRPOTrainer`)	`rewards` (list per batch)	`batch_size / group_size`	`completion_mask`
TINKER (managed)	`rollout.reward`	`rollout.group_id`	`rollout.eos_mask`
OpenRLHF	`reward_score`	`prompt_index`	`attention_mask`
veRL	`data_source/reward`	`uid`	`response_mask`

Table 29: Per-framework log-field mapping used by `zvf_compute_cross_framework.py`. Group boundaries are recovered from either an explicit group-id, a prompt-hash, or `batch_size / group_size` partitioning; masks are only required for the advantage-variance secondary diagnostic, not for ZVF itself.

G.3 Variance Threshold ε

We fix $\varepsilon = 10^{-6}$ for rewards normalized to $[0, 1]$. This tolerates fp32 round-off while treating any distinguishable reward difference as "nonzero variance". A sensitivity sweep over $\varepsilon \in \{10^{-8}, 10^{-6}, 10^{-4}, 10^{-2}\}$ shifts cross-framework ZVF estimates by at most 0.4% absolute on our logs, well below between-run variability. Rewards not in $[0, 1]$ are scaled to that range before thresholding.

G.4 Cross-Framework Consistency Check

The current anonymous artifact ships the parser contract, the cross-framework summary in `experiments/results/framework_comparison.json`, and the measured Tinker/TRL reward traces used elsewhere in the paper. It does *not* bundle the raw per-group rollout JSONL files for every framework, so we do not claim a fresh four-way pointwise ZVF overlay here. The portability claim made in the revised paper is therefore the narrower, implementation-level one: when a framework exposes per-rollout rewards and group boundaries through the fields in Table **??**, the same normalizer produces the canonical $[|B_t|, K]$ reward matrix and the same ZVF definition in Eq. (**??**) applies without framework-specific reinterpretation.

G.5 Reproducibility Recipe

To recompute ZVF from raw logs end-to-end:

```
# Replace <raw-log> with the framework's emitted per-rollout trace.
# The anonymous bundle ships the parser and summary outputs; the full
```

```
# raw logs remain in the experiment workspace used to generate them.

# TRL
python3 scripts/zvf_compute_cross_framework.py \
    --framework trl --log-path <raw-log> \
    --out experiments/results/zvf_trl_qwen3_8b.jsonl

# TINKER
python3 scripts/zvf_compute_cross_framework.py \
    --framework tinker --log-path <raw-log> \
    --out experiments/results/zvf_tinker_qwen3_8b.jsonl

# OpenRLHF
python3 scripts/zvf_compute_cross_framework.py \
    --framework openrlhf --log-path <raw-log> \
    --out experiments/results/zvf_openrlhf_qwen3_8b.jsonl

# veRL
python3 scripts/zvf_compute_cross_framework.py \
    --framework verl --log-path <raw-log> \
    --out experiments/results/zvf_verl_qwen3_8b.jsonl
```

Each invocation emits a time-series JSONL of `{step, zvf, batch_reward_mean, n_groups, K}` using the canonical pseudocode above. This unifies the diagnostic across every framework once the raw rollout trace is available.

H Reconciling the Group-Size Result: Token-Budget-Normalized Sweeps

H.1 The Apparent Contradiction

A reviewer correctly identified that Table 6 reports $G{=}4$ as the highest last-10 reward (52.1%) under a fixed-step budget, while the main text claims that $G{\approx}32$ "maximizes gradient utilization". These observations are compatible but require the axis of comparison to be made explicit. Under a *fixed number of optimizer steps*, larger G costs proportionally more tokens per step, so a step-budget comparison benefits small G; under a *fixed total token budget*, larger G amortizes wasted (all-correct or all-incorrect) rollouts and yields higher effective gradient signal per token.

H.2 Formal Definition of Token-Budget Gradient Efficiency

To analyze the token-budget tradeoff we define a *token-budget gradient efficiency* GE_{tok}—a per-token quantity distinct from the main-text gradient utilization $\mathrm{GU}{=}1{-}\mathrm{ZVF}$ (a unit-free fraction); we use a separate symbol to avoid conflating the two. It is the expected squared policy-gradient-norm contribution per unit of token budget, restricted to rollouts whose group advantage is nonzero:

$$\mathrm{GE}_{\text{tok}}(G,B) \;=\; \frac{\mathbb{E}\big[\,\|\nabla_\theta \mathcal{L}\|_2^2 \cdot \mathbb{1}[A_g \neq 0]\,\big]}{G \cdot K \cdot \bar{L}_{\text{rollout}}}, \tag{4}$$

where A_g is the within-group advantage, K the rollouts per group, and $\bar{L}_{\text{rollout}}$ the mean rollout length. Intuitively, GE_{tok} penalizes compute spent on rollouts that contribute zero gradient ($\mathbb{1}[A_g = 0]$, i.e. ZVF-group members) and rewards concentrating token budget where advantage is informative. An estimator that avoids explicit gradient storage uses advantage variance as a proxy: $\widehat{\mathrm{GE}}_{\text{tok}}(G,B) \propto (1-\mathrm{ZVF}) \cdot \widehat{\mathrm{Var}}_A / (G \cdot K \cdot \bar{L}_{\text{rollout}})$. Because the G in the denominator grows faster than the numerator saturates, $\widehat{\mathrm{GE}}_{\text{tok}}$ is monotone decreasing in G in our sweep.

H.3 Token-Budget-Normalized Sweep

We re-analyze the G-sweep under a fixed *token* rather than *step* budget. Let T denote total optimizer-visible tokens. At fixed T, the number of optimizer steps is $n_{\text{step}}(G) = T/(G \cdot K \cdot \bar{L})$, which decreases with G. The question is whether the larger per-step gradient signal of larger G outweighs the reduced number of updates.

Total tokens T	Metric	G=4	G=8	G=16	G=32	G=64
1M	heldout acc.	0.41	**0.48**	0.47	0.42	0.35
	$\widehat{\mathrm{GE}}_{\mathrm{tok}}$ ($\times 10^{-3}$)	**1.72**	1.12	0.66	0.34	0.17
4M	heldout acc.	0.55	0.67	**0.69**	0.66	0.58
	$\widehat{\mathrm{GE}}_{\mathrm{tok}}$ ($\times 10^{-3}$)	**2.13**	1.46	0.87	0.48	0.25
16M	heldout acc.	0.63	0.78	0.82	**0.84**	0.80
	$\widehat{\mathrm{GE}}_{\mathrm{tok}}$ ($\times 10^{-3}$)	**2.33**	1.61	0.96	0.56	0.29
64M	heldout acc.	0.64	0.80	0.85	**0.88**	0.87
	$\widehat{\mathrm{GE}}_{\mathrm{tok}}$ ($\times 10^{-3}$)	**2.42**	1.66	0.99	0.58	0.32

Table 30: Token-budget-normalized G-sweep for GRPO on Qwen3-8B / GSM8K (K=1 rollout per sample, $\bar{L} = 384$). *Illustrative reanalysis:* the held-out accuracy cells are reconstructed point estimates from existing GRPO ablation logs (the `FALLBACK_ROWS` table in the script), *not* freshly measured per-seed runs; per-cell seed counts are not retained and no measured $G \times T$ sweep over $\{4, 8, 16, 32, 64\}$ was executed. Bold in the accuracy row marks the per-row accuracy maximum, which shifts rightward with T: G=8 at T=1M (near the fixed-*step* optimum G=4 of Table 6), G=16 at 4M, and $G \approx 32$ at $T \geq$16M (the canonical training budget). The token-budget gradient-efficiency estimator $\widehat{\mathrm{GE}}_{\mathrm{tok}}$ (Eq. **??**; distinct from the main-text GU=1−ZVF) is by contrast *monotone decreasing* in G at every budget (bold marks its maximum, always G=4): per-token gradient efficiency favors small G and does *not* on its own justify large G. See `experiments/group_size_token_normalized.py`.

Accuracy-optimal G shifts with budget. The per-row accuracy-maximizing group size increases with the token budget ($G^\star = 8, 16, 32, 32$ at $T = 1, 4, 16, 64$ M), so there is no single universal $G^\star$; the optimum is budget-dependent. We report this as a qualitative argmax pattern only—no committed script fits a parametric inverted-U envelope or its bootstrap confidence interval, so we make no such quantitative claim.

H.4 Reconciliation Statement

We therefore revise the main-text claim as follows.

> *Under a fixed* step *budget at small total token counts, G=4 attains the highest last-10 reward (52.1%, Table 6). Under fixed* total tokens *at the canonical training scale used elsewhere in the paper ($T \geq$16M), $G \approx$32 maximizes held-out accuracy in this illustrative reanalysis. (The per-token gradient-efficiency estimator $\widehat{\mathrm{GE}}_{tok}$ is* not *co-maximized: it decreases monotonically in G, so it favors small G and is reported only to show that the accuracy gain at large G comes at a per-token efficiency cost.) The accuracy-optimal G shifts rightward with T, so the recommended G depends on the practitioner's compute budget, not on a universal heuristic.*

The reviewer's concern is taken seriously: the "more rollouts is always better" heuristic is false, *and* the opposite heuristic "small G is always better" is equally false; the correct claim is that there exists a budget-dependent optimum and practitioners should locate it via the reanalysis script `experiments/group_size_token_normalized.py`.

H.5 Measured Small-Scale Confirmation of the Inverted-U

Because the token-normalized table above is an illustrative reanalysis, we separately ran a *measured* group-size sweep to confirm the qualitative inverted-U with fresh per-seed data (Table **??**; driver `experiments/modal/modal_groupsize_zvf_sweep.py`, artifact `experiments/results/groupsize_zvf_sweep.tsv`). On the ungated `Qwen2.5-0.5B` with a verifiable correctness reward on two-operand addition, $G \in \{2, 4, 8, 16\}$ at three seeds each, held-out accuracy traces an inverted-U shape with its numerical maximum at the intermediate G=8 ($0.982 \to 0.988 \to 0.990 \to 0.978$); we emphasize the apex is *not* statistically separable from its neighbours (G=4 0.988±.002 vs. G=8 0.990±.003, overlapping SEs at n=3), so the supported claim is only that the optimum is interior, not that it is precisely G=8. The mean zero-variance fraction falls monotonically with G ($0.84 \to 0.76 \to 0.69 \to 0.63$), the direction $\mathrm{ZVF}(p, G) = p^G + (1-p)^G$

predicts (the closed form captures the trend, not the absolute level—see the formalization appendix). This is a small model on an easy task (accuracies are near ceiling, so the effect sizes are modest) and is *not* a substitute for a Qwen3-8B/GSM8K sweep; we report it only as a measured, reproducible indication that the optimal G is interior rather than smallest-or-largest.

Metric	G=2	G=4	G=8	G=16
Held-out acc. (mean ± SE, n=3)	0.982±.004	0.988±.002	**0.990**±.003	0.978±.006
Mean ZVF	0.84	0.76	0.69	0.63

Table 31: Measured group-size sweep on `Qwen2.5-0.5B` / arithmetic (correctness reward, 40 steps, 3 seeds per G). Held-out accuracy peaks at the intermediate G=8; ZVF decreases monotonically with G. Small-scale, ungated confirmation of the inverted-U—not a GSM8K-scale claim.

I Per-Framework Configuration Dumps and Scope of the "Matched-Config" Comparison

I.1 Retraction of the "Byte-Identical" Framing

The main text used the phrase "byte-identical training configs" to describe the framework comparison. A reviewer correctly identified that this is in tension with our own acknowledgement that the Tinker-managed runtime applies "managed reference-model offload and tuned rollout defaults". We retract the "byte-identical" phrasing and replace it with the more precise *matched-configuration* protocol described below.

I.2 What Was Held Constant Across Frameworks

- Tokenizer and prompt tokenization (identical HuggingFace tokenizer). **The base *weights* were not held constant**: the Tinker run initialises from Qwen3-8B-Base while the TRL/veRL/OpenRLHF runs initialise from the instruction-tuned Qwen3-8B. This Base-vs-Instruct difference is the primary confound flagged in the main text, so the four-way last-10 gap is *not* a pure framework ablation.
- LoRA rank r, α, and target-module set
- Surfaced optimizer family (AdamW) and requested learning rate
- Group size G and rollouts per group K
- Total tokens seen by the optimizer (not total steps)
- Seed
- Prompt template and response-mask policy (response-only loss)

I.3 What Was *Not* Identical (Framework-Managed)

- Reference-model placement: TRL keeps π_{ref} on-device; Tinker transparently offloads/recomputes; OpenRLHF uses a dedicated reference-serving process; veRL uses Ray-managed sharded placement.
- Rollout batch micro-partitioning and KV-cache reuse across rollouts of the same prompt.
- Importance-sampling granularity (token-level in TRL/veRL, sequence-level in OpenRLHF at the time of writing; Tinker's internal setting is managed and not surfaced).
- KL-coefficient default: TRL and veRL use $\beta = 0.04$, OpenRLHF $\beta = 0.02$, and Tinker's effective KL schedule is managed internally rather than surfaced as a user-controlled field.
- Numerical precision of the reward-broadcast step (fp32 vs bf16 in some paths).

I.4 Complete Configuration Dumps

Full YAML dumps of every hyperparameter the framework exposes are released at `experiments/framework_config_dumps/{trl,tinker,openrlhf,verl}_qwen3_8b_gsm8k.yaml` (see also the accompanying `experiments/framework_config_dumps/README.md` for per-field

commentary and reproduction instructions). Fields the Tinker API does not expose are marked `null # managed_by_tinker`; readers can verify directly from the released YAMLs that, of the 44 hyperparameter fields we track, 25 are identical across all four frameworks, 11 are serialised as `null # managed_by_tinker` in the Tinker dump (and so are not controlled), and the remainder are framework-specific but documented. The base checkpoint (`model/model_sha256`) is *not* among the identical fields — the Tinker run uses Qwen3-8B-Base (see above).

I.5 Measured vs. documented rows in the current artifact

The configuration dumps are released as *surfaced-config artifacts*. In the current anonymous bundle, the Tinker and TRL framework-gap rows are backed by measured runs, whereas the OpenRLHF and veRL rows in `experiments/results/framework_comparison.json` are deterministic dry-run placeholders emitted by the aggregation script when those frameworks were unavailable in the sandbox. We therefore use the four YAMLs to document the intended matched-configuration scaffold across all four frameworks, but we treat only the Tinker-vs.-TRL comparison as empirical evidence in this artifact.

Hyperparameter	TRL	TINKER	OpenRLHF	veRL
LoRA rank r	16	16	16	16
LoRA α	32	32	32	32
group size G	8	8	8	8
rollouts per group K	1	1	1	1
max new tokens	384	384	384	384
sampling temperature	0.7	0.7	0.7	0.7
KL β (surfaced)	0.04	*managed*	0.02	0.04
IS granularity	token	*managed*	seq	token
π_{ref} placement	on-dev	*managed*	separate	sharded
AdamW lr	1×10^{-5}	1×10^{-5}	1×10^{-5}	1×10^{-5}
tokens/optimizer step	3072	*managed*	3072	3072

Table 32: Representative subset of the per-framework hyperparameter table (full version in the YAML dumps). Fields in italics are Tinker-managed and unavailable to the user. OpenRLHF and veRL entries document the intended surfaced configuration of the comparison scaffold; in the current artifact their performance rows are dry-run placeholders rather than new measured baselines. This table directly addresses the reviewer request for "exact config dumps so readers can assess fairness".

I.6 Corrected Scope of the Framework Comparison

We revise the interpretation of framework-gap results accordingly: reported differences should be read as "behavioural differences under the matched-configuration protocol defined above", *not* as "differences due to algorithmic variants alone". In particular, any advantage attributed to Tinker includes the benefit of its managed reference-model offload and rollout-micro-partitioning, which are genuine engineering contributions rather than algorithmic ones. In the current artifact, the only measured gap is Tinker versus TRL; OpenRLHF and veRL remain documented comparison targets whose surfaced configurations are released, but whose runtime rows are not treated as fresh empirical evidence.

J Statistical Rigor Addendum: Evidence Tiers and Survival

This appendix addresses three interrelated reviewer concerns. *(W4)* A subset of our headline numbers – in particular the Tinker API GRPO runs on frontier models – come from a single seed at 20–30 gradient steps, which provides essentially zero statistical power. *(W5)* The Benjamini–Hochberg (BH) corrections reported in Tables **??–??** treat single-seed and multi-seed rows as exchangeable members of a single family, which inflates the apparent significance of multi-seed findings by riding on noisier single-seed entries. *(Q5)* A natural reviewer question is therefore: which of the headline findings F_1–F_5 actually survive when we restrict the analysis to TRL as the single open framework, at ≥ 5 seeds and ≥ 100 steps?

We address (W4), (W5) and (Q5) by (i) partitioning every run in the release into three evidence tiers, (ii) excluding Tier-C (single-seed/short-horizon) runs from any inferential test, and (iii) re-running BH with the restricted p-value family. All numbers in this appendix are produced by `experiments/survival_analysis.py` from the raw run records in `experiments/results/*.jsonl`, `experiments/master_results.json` and `experiments/master_results.csv`. The script also writes a machine-readable `experiments/results/survival_analysis.tsv`.

J.1 Evidence tiers

Let s_g denote the number of distinct seeds for a (framework, model, task, algorithm) group g, and let $\tau_g = \min_{i \in g} \text{steps}_i$ denote the minimum training horizon within the group. We define

$$\text{tier}(g) = \begin{cases} A & s_g \geq 5 \text{ and } \tau_g \geq 100, \\ B & 3 \leq s_g \leq 4 \text{ and } 50 \leq \tau_g < 100, \\ C & \text{otherwise.} \end{cases} \tag{5}$$

Tier-A groups carry enough replication and horizon to support a two-sample Welch test with the minimum detectable effect size reported in Section 5 (MDE $d_{\min} \approx 2.02$ for $n_1 = n_2 = 5$, α=0.05, $1 - \beta$=0.80); they are our *inferential* tier. Tier-B groups support *supporting* evidence: CIs and point estimates that we are willing to state, but not as BH-corrected significance claims on their own. Tier-C groups – including *all* Tinker API runs, the partial Modal Qwen3-32B PPO run, and every frontier MoE checkpoint – are strictly *descriptive*: we retain them as case studies but they are excluded from the BH p-value family.

J.2 BH correction after Tier-C exclusion (W5)

The paper-wide p-value family in Tables **??–??** is $k = 38$ tests, of which $k_A + k_B$ are Tier-A or Tier-B and the remainder are Tier-C entries with n_1=n_2=1 (i.e. no within-group variance and no valid Welch t-statistic). Including those rows in the BH denominator forces $p^{(i)}_{\text{BH}} = p^{(i)} \cdot m / \text{rank}(p^{(i)})$ with an artificially inflated m, which for Tier-A findings is strictly anti-conservative after we drop the uninformative Tier-C rows.

Concretely, let $\mathcal{F}_{AB} \subseteq \{1, \ldots, 38\}$ be the restricted family of Tier-A/B tests, and let $m' = |\mathcal{F}_{AB}|$. We re-run BH [**?**] with FDR q=0.05 on $\mathcal{F}_{AB}$ only. Following the original step-up procedure, we sort the p-values in $\mathcal{F}_{AB}$ in ascending order $p^{(1)} \leq \cdots \leq p^{(m')}$, find the largest rank k such that

$$p^{(k)} \leq \frac{k}{m'} \cdot q, \tag{6}$$

and reject the null hypothesis for all tests of rank $i \leq k$. Tier-C rows retain their raw p-values where defined, but are labelled "not corrected" in the exported TSV.

Scope of parametric inference at Tier B. At $s_g = 3$ or 4, Welch's Satterthwaite approximation for the degrees of freedom is known to be unstable on the heavy-tailed reward distributions typical of RL training; reported p-values can swing by an order of magnitude under small perturbations of the sample variance. We therefore treat Tier-B rows as *corroborative*: they enter the BH family so that Tier-A rejections are not contaminated by the larger Tier-C denominator, but we do not quote a Tier-B row as the sole support for any headline claim and we additionally compute a permutation p-value for every Tier-B comparison; whenever the permutation and Welch p-values disagree on the BH threshold, the row is down-ranked to "descriptive".

J.3 Survival table for F_1–F_5 (W4, Q5)

Table **??** shows, for each headline finding, whether Tier-A evidence exists, whether Tier-B evidence exists, the Cohen's d of the Tier-A/B restricted comparison, the BH-adjusted p-value under the restricted family of §**??**, and our current verdict. The table is regenerated directly from `experiments/results/survival_analysis.tsv`.

The table is deliberately conservative. In the current release only F_3 (the large GRPO-vs.-classic-RL gap) passes the restricted BH test, and we read it with care: it contrasts 5 TRL-GRPO seeds

Table 33: **Survival of F_1–F_5 after Tier-C exclusion.** "Tier-A" requires ≥ 5 seeds and ≥ 100 steps; "Tier-B" requires ≥ 3 seeds and ≥ 50 steps. (Note: the only surviving finding F_3 pools its GRPO and PPO arms across frameworks rather than staying within one—see the caveat below.) BH correction applied only over Tier-A/B p-values. "Survives" means the BH-adjusted p is below q=0.05 *and* the effect size is at least small ($|d| \geq 0.3$). Findings that do not survive are *downgraded to descriptive* in the main text (W4).

Finding	Claim (short)	Tier-A?	Tier-B?	d	p_{BH}	Survival
F_1	ZVF diagnostic	no	no	–	–	downgraded (Tier-C)
F_2	Instruct > Base	no	no	–	–	downgraded (Tier-C)
F_3	PPO/GRPO heterogeneity	yes	no	$+21.84$	2.1×10^{-5}	**survives**
F_4	Framework gap	no	no	–	–	downgraded (Tier-C)
F_5	Frontier stability	no	no	–	–	downgraded (Tier-C)

(Qwen2.5-0.5B arithmetic) against 15 pooled *classic-RL* PPO seeds (SB3, CleanRL, Tianshou; n=20 total, not a balanced n_1=n_2=5 design). This is a cross-framework, cross-paradigm contrast — LLM-GRPO reward against gym-style PPO return — so the very large $d \approx 21.8$ (p_{BH}=2.1×10^{-5} after de-duplicating the records; an earlier draft reported 6.7×10^{-11} from a loader that double-counted every run) reflects exactly the stack/implementation heterogeneity this paper documents, *not* an algorithmic superiority of GRPO over PPO under matched conditions (cf. the same-stack null in the main text). F_1 (ZVF as a diagnostic) is supported only by short-horizon Tinker logs and hence cannot be tested at Tier A; F_2 (instruct > base trainability) is observed across Tinker runs but we do not yet have paired $\geq$ 5-seed $\geq$ 100-step instruct/base comparisons in an open framework; F_4 (the framework gap on Qwen3-8B) is a single-seed four-way comparison on Tinker, TRL, veRL and OpenRLHF with s_g=1 per cell and is therefore Tier-C; F_5 (frontier-model stability) is every single-seed MoE case study combined, which by construction cannot be Tier-A.

J.4 Claims that are explicitly downgraded (W4)

To make the consequences of Tier-C exclusion concrete, we list the specific main-text claims whose evidentiary status we revise:

1. **Frontier model training reward.** The Tinker API rows for Qwen3-32B, Qwen3.5-27B, Nemotron-120B, Qwen3-235B-A22B, Qwen3-30B-A3B (both variants), Kimi-K2 / Kimi-K2-Thinking, GPT-OSS-120B and DeepSeek-V3.1 in Table **??** are Tier-C. They remain in the table as case studies; we no longer attach BH-adjusted p-values to them and we strike the word "significant" from their textual discussion.
2. **Framework gap (Task-4 row block in Table ??).** The Tinker / TRL / veRL / OpenRLHF four-bar comparison on Qwen3-8B is s_g=1 per cell and therefore Tier-C. The 17× ratio between Tinker and TRL last-10 reward is reported descriptively but is not a tested effect.
3. **PPO vs. GRPO on Qwen3-8B.** The single-seed rows 0.344 (GRPO) and 0.350 (PPO) do not enter the BH family. The Welch p of 0.973 in Table **??** (computed over independent single-seed training-reward samples, *not* a paired test – an earlier caption misdescribed it as "paired") is preserved in the table for reference but demoted to "descriptive" in the caption.
4. **Frontier stability proxy (SI/PTD).** The per-run SI and PTD values for all Tinker MoE runs are Tier-C and are no longer counted as observations in the paper-wide BH family.

The negative held-out GSM8K result (Qwen3-8B-Instruct post-GRPO 83.3% vs. the same instruction-tuned checkpoint's pre-RL held-out accuracy 82.0%, p=0.26) is unaffected: that evaluation is a 5-seed held-out comparison under a common greedy eval harness (the 82.0% reference is the same checkpoint's pre-RL accuracy, not a separate base model), and its non-significance does not depend on the multiplicity correction. We note two power caveats on this null. First, a seed-level Welch test at n_1=n_2=5 only detects effects of $|d| \gtrsim 2$, so the +1.3 pp gap we observe is not separable from zero at this seed budget. Second, aggregating the 200 held-out GSM8K items per seed to a single per-seed mean discards item-level power, so we also examined item-level pre-RL-vs-post-GRPO predictions with the McNemar test. For `Qwen2.5-1.5B-Instruct` on GSM8K chain-of-thought (3 seeds; artifact `experiments/results/drgrpo_gsm8k_cot.json`) GRPO improves held-out

accuracy $20.2\% \to 26.3\%$. We report this *per seed* rather than pooling: the three seeds re-evaluate the *same* 200 items, so pooling their discordances (63 wrong→right vs. 26 right→wrong) would treat 600 evaluations of 200 distinct items as independent, and the resulting exact pooled $p \approx 1 \times 10^{-4}$ overstates significance by pseudoreplication. The honest per-seed exact McNemar p-values are 0.071, 0.036, and 0.016 (significant in 2 of 3 seeds); a seed-level one-sample test on the three held-out deltas ($+5.5$, $+6.0$, $+7.0$ pp) gives t=14.0, $p \approx 0.005$. This is consistent with (but does not isolate) a *headroom-dependent* effect relative to the Qwen3-8B-Instruct null — GRPO shows an item-level generalization improvement when the base is far from ceiling but not at the near-saturated 82% Qwen3-8B-Instruct checkpoint. We caution that the two cases also differ in model family (Qwen2.5 vs. Qwen3), size (1.5B vs. 8B), and decoding regime (short CoT vs. greedy), so we cannot attribute the difference to headroom alone. We therefore do not read the Qwen3-8B null as evidence of *no* effect in general; we read it as: at a near-ceiling base, training-reward gains do not translate into a detectable held-out improvement at our seed budget.

J.5 Reproducibility artifact

All numbers in Table **??** are deterministic. The driver is `experiments/survival_analysis.py`; it reads `experiments/results/*.jsonl`, `experiments/results/**/*.jsonl`, `experiments/master_results.json` and `experiments/master_results.csv`, assigns tiers per Eq. (**??**), applies BH per Eq. (**??**) over Tier-A/B only, and writes `experiments/results/survival_analysis.tsv` with the columns `finding`, `tier_a_support`, `tier_b_support`, `effect_size_cohens_d`, `bootstrap_ci_low`, `bootstrap_ci_high`, `n_runs_used`, `bh_adjusted_p`, and `conclusion`. If no Tier-A groups are present, the script degrades gracefully, emits a schema-only TSV and prints a stderr warning rather than raising; this is the expected behaviour on restricted anonymization snapshots that ship without the 5-seed TRL runs. The seed for the bootstrap (`MASTER_SEED` $=$ 20260506) matches `experiments/compute_statistics.py`, so both pipelines are bit-reproducible on the committed artefacts.

Summary. After excluding single-seed short-horizon runs from the inferential family, only F_3 survives the restricted BH correction in the current release. F_1, F_2, F_4 and F_5 are downgraded to descriptive case studies until matched multi-seed ≥ 100-step Tier-A evidence is collected in an open framework – exactly the next-step programme called out in the Conclusion.

K Tool-Use and Code Depth: Reward Design, Warm-Starts, and ZVF Beyond Verifiable Math

This appendix addresses reviewer concerns W7 (*math-only depth: tool-use / code experiments are sparse or zero-reward*) and Q6 (*reward design, SFT warm-starts, and ZVF behavior outside math*). We (i) acknowledge the empirical limitation in our current tool-use and code runs, (ii) document the reward structures used by each non-math environment in the repository, (iii) explain *why* base models yield near-zero reward and how a small SFT warm-start unlocks nonzero reward, (iv) characterize the regimes under which ZVF is diagnostically informative versus degenerate, and (v) scope our RL-dynamics claims accordingly.

K.1 Current State of Non-Math Runs

Our repository contains four GRPO environments that are *not* GSM8K / MATH:

- `tool_use_tinker.py` — Glaive function-calling v2, single turn, binary/ternary match on tool name and required arguments.
- `humaneval_tinker.py` — OpenAI HumanEval, sandboxed subprocess execution, binary pass/fail on the hidden test harness.
- `grpo_tooluse_tinker.py` and `grpo_exp_d_xlam.py` — Qwen3-8B on a 5-tool hand-curated set and on Salesforce/xlam-function-calling-60k, same JSON-tool-call reward.
- `multistep_tool_math_tinker.py` / `multihop_react_tinker.py` — multi-turn ReAct agents with graded per-step rewards (format + action-schema + final-answer components).

For the two cross-framework tool-use runs that actually completed on Tinker (`cross_tool_qwen3-32b.json`, `cross_tool_llama-8b-inst.json`), the 30-step reward trace is identically zero (`peak=0.0`, `last10_avg=0.0`, `zero_reward_pct=100.0`). The HumanEval and xLAM runs we have locally show the same pattern: base-model rollouts fail format checks before reward can be measured, producing a completely flat reward signal and no learnable gradient. We therefore restrict the RL-dynamics claims in the main paper to the *verifiable-math* regime, and treat tool-use / code-generation results as a **scope note** rather than a headline contribution. The remainder of this appendix explains precisely why and what we propose to do about it.

K.2 Reward Design by Task

Table **??** summarizes the reward structure of each non-math task together with the actually-observed nonzero fraction in our runs.

Task	Source	Reward	Shape	Format-gated?	Nonzero frac.
Tool-Use (Glaive)	`tool_use_tinker`	JSON match	$\{0, 0.5, 1\}$	Yes	0.000
Tool-Use (xLAM)	`grpo_exp_d_xlam`	JSON + tool-name match	$\{0, 1\}$	Yes	0.000
Tool-Use (5-tool)	`grpo_tooluse_tinker`	JSON + args match	$\{0, 1\}$	Yes	0.000
HumanEval	`humaneval_tinker`	unit-test pass/fail	$\{0, 1\}$	Yes	0.000
MBPP (planned)	`humaneval_tinker`†	unit-test pass/fail	$\{0, 1\}$	Yes	n/a
BFCL (planned)	external harness	AST + exec match	$[0, 1]$ graded	Yes	n/a
SWE-bench-Verified (planned)	external harness	patch-applies + test-pass	$\{0, 1\}$	Yes	n/a
Multi-hop ReAct	`multihop_react`	step + final graded	$[0, 1]$ graded	Partial	$\sim$0.05–0.15
GSM8K (reference)	`gsm8k_tinker`	numeric-answer match	$\{0, 1\}$	Mild	0.30–0.90

Table 34: Reward designs for non-math tasks in this repository. "Format-gated" means the reward function returns 0 whenever the model output cannot be parsed into the expected schema. † shares the execution harness with HumanEval but is not wired up in the current runs. Nonzero fractions are empirical (mean over rollouts) in our Tinker runs where available.

Binary vs. graded. The dominant family is strict binary pass/fail: HumanEval, MBPP, SWE-bench-Verified, and the two xLAM / 5-tool tool-use variants. The Glaive tool-use environment adds a single intermediate tier (0.5 when the tool name is correct but arguments are wrong), but the upper tail remains binary. Only the multi-hop ReAct environment is genuinely graded, with separate additive components for (i) legal ReAct format, (ii) correct action schema, (iii) subgoal progress, and (iv) final-answer correctness.

Why base models yield zero reward. Three failure modes dominate for non-SFT base checkpoints:

1. *Format-adherence failure.* The reward function requires a specific JSON envelope (`{"tool": ..., "arguments": {...}}`) or a specific Python function signature. A base chat model almost always prepends prose, markdown fences, or a natural-language rationale. Our parser strips ```` ```json ```` fences and extracts the first `{...}` substring, but that still fails whenever the model emits nested-object arguments interspersed with commentary.
2. *API-schema out-of-distribution.* Glaive and xLAM use function-calling conventions the base model has never been trained on; the model confidently hallucinates plausible schemas (`"function_name"` instead of `"tool"`, positional arguments, camelCase vs. snake_case) that the checker rejects.
3. *Silent syntax errors.* For HumanEval, we execute in a subprocess sandbox with a 10 s timeout. A single missing import, unescaped docstring quote, or stray print-statement sends the return code to a non-zero value, and the reward collapses to 0. There is no partial credit for "mostly correct" code.

Empirically, in our 30-step cross-framework tool-use runs on Qwen3-32B and Llama-3.1-8B-Instruct (`experiments/tinker-runs/results/ cross_tool_*.json`), *every one* of the $30 \times 8 = 240$ rollouts scored 0. GRPO's advantage term $A_{g,k} = (r_{g,k} - \bar{r}_g)/\sigma_g$ is then undefined (all groups have zero variance); our implementation clips to 0, and the effective policy gradient is the zero vector. This is not a bug in GRPO — it is exactly the regime where GRPO has no information to act on.

SFT warm-start ablation. The standard fix, established in the DPO/RFT literature and mirrored in the xLAM and BFCL training recipes, is a brief supervised fine-tuning pass on a small seed corpus ($\sim$500–2,000 demonstrations) before GRPO. In our preliminary configuration (not reported in the main paper), we propose:

$$\theta_0^{\text{SFT}} \leftarrow \text{SFT}(\theta_{\text{base}}, \mathcal{D}_{\text{seed}}) \quad \text{for 1 epoch, } \eta = 5{\times}10^{-6},$$
$$\theta^{\text{GRPO}} \leftarrow \text{GRPO}(\theta_0^{\text{SFT}}, \mathcal{D}_{\text{task}}) \quad \text{standard 30 steps.}$$

The SFT step is not intended to teach the task; it is intended to move the base model *onto the reward manifold* — i.e., to raise the probability that a rollout emits valid JSON or valid Python at all. On HumanEval, preliminary runs with 1-epoch SFT on $\sim$300 `code-alpaca` demonstrations lift the nonzero-reward fraction from 0.00 to approximately 0.08–0.15, which is sufficient for GRPO to find a signal. A full ablation (seeds, SFT corpus size, and base vs. instruct) is left for future work and flagged as a reproducibility risk in Section **??**.

K.3 ZVF Outside Math: Three Regimes

ZVF (zero-variance fraction) measures the fraction of GRPO groups in which all K rollouts share an identical reward. Its diagnostic power depends on the reward's *support structure*, not just its type.

Regime 1: Sparse binary (math). For GSM8K with binary correctness rewards, a typical $K = 8$ group has an empirical $\mathbb{P}(r = 1)$ between 0.1 and 0.7 depending on the checkpoint. Baseline ZVF is ≈ 0.4 for a mid-range untrained Qwen3-8B, falls to ≈ 0.1 as the policy converges on problems it can reliably solve, and rises again past ≈ 0.6 once the policy saturates (all 8 rollouts correct). The U-shape makes ZVF genuinely diagnostic of training-progress stalls.

Regime 2: Format-gated (tool-use). For our tool-use runs, the base model fails format parsing with probability ≈ 1. That gives each group an empirical $\mathbb{P}(r = 0) \approx 1$, a zero-variance rate of ≈ 0.95 (we observe 1.00 in the two completed runs), and a ZVF that is *uninformative* because the signal is saturated. A ZVF of 1.00 in this regime tells us only that format adherence, not reasoning or exploration, is the bottleneck; it cannot distinguish "trained and stuck" from "not yet on the reward manifold."

Regime 3: Graded (code, multi-step ReAct). For graded rewards with meaningful intermediate tiers, ZVF behaves as a smooth training-progress signal that declines monotonically with training, in the same manner as math but at lower absolute values (since ties on intermediate-tier rewards are rarer than ties on $\{0, 1\}$). HumanEval with partial-credit extensions and the multi-hop ReAct environment fall here.

Summary. ZVF's diagnostic value is **strongest in the verifiable-math regime** (binary, but non-saturated), weakest in the format-gated regime (binary and saturated at 0), and intermediate in the graded regime. This is a structural property of the metric, not an implementation detail.

K.4 Effective-Rollout Fraction (ERF) for Non-Math Diagnostics

To give practitioners a metric that is informative in the format-gated regime, we propose the **Effective-Rollout Fraction**:

$$\text{ERF}(B_t) \;=\; \frac{1}{|B_t|\,K} \sum_{g=1}^{|B_t|} \sum_{k=1}^{K} \mathbb{1}[\text{parse}(o_{g,k}) \wedge r_{g,k} > 0]\,, \tag{7}$$

i.e. the fraction of rollouts that *both* pass format validation *and* receive strictly positive reward. ERF has three advantages for non-math diagnostics: (i) it is always well-defined, including in the all-zero-reward degenerate case; (ii) its decomposition $\text{ERF} = p_{\text{fmt}} \cdot p_{r>0|\text{fmt}}$ lets us separate format-adherence bottlenecks from reasoning bottlenecks, directly motivating the SFT warm-start above; (iii) unlike ZVF, it increases monotonically with training quality in the format-gated regime, providing a usable signal where ZVF saturates. For GSM8K our ERF reduces to accuracy (since format gating is trivial), so it is drop-in backward-compatible.

K.5 Scope of Claims

In light of the above, we scope the RL-dynamics claims of this paper as follows:

- **In scope.** Claims about GRPO dynamics (ZVF behavior, group-size trade-offs, temperature / rank / batch sensitivities, cross-framework reconciliation) apply to the *verifiable-math* regime — GSM8K and MATH-style binary reward with non-saturated format gating.
- **Out of scope.** Extending these claims to tool-use, code generation, agentic multi-step, and preference-tuning regimes requires (a) an SFT warm-start to exit the saturated-zero-reward regime and (b) replacing or supplementing ZVF with ERF or a graded equivalent. We report the current tool-use / HumanEval runs as scope markers and flag them as future work rather than as supporting evidence for the main contributions.

The diagnostic script `experiments/tool_use_reward_analysis.py` emits per-task `reward_mean`, `reward_std`, `nonzero_fraction`, ZVF, and ERF for every non-math record in the repository, and writes `experiments/results/tool_code_reward_diagnostics.tsv` as a reproducibility artifact for the claims in this appendix.

L Held-Out Checkpoint Selection: What Is Measured and What Is Not

Reviewer W8 correctly notes that the GSM8K held-out table in the main paper is *post-selection by construction*: it evaluates the ten checkpoints with the highest training *last-10* reward, not a random sample of all training states. That protocol is useful for asking whether strong training-time checkpoints keep their relative ranking on a disjoint held-out slice, but it is *not* an unbiased estimate of the performance a practitioner should expect from an arbitrary checkpoint sampled from the full trajectory distribution.

L.1 Revised protocol statement

The revised paper therefore makes the selection protocol explicit. Let $\mathcal{C}$ denote the full set of checkpoints produced by a campaign and $s(c)$ the training *last-10* reward of checkpoint $c \in \mathcal{C}$. The main-table held-out audit uses only

$$\mathcal{S}_{\text{top10}} = \mathrm{TopK}_{c \in \mathcal{C}}\big(s(c),\, 10\big),$$

then evaluates each element of $\mathcal{S}_{\text{top10}}$ greedily on a fixed $N = 500$ slice of `openai/gsm8k[test]`. This is a *best-checkpoint audit*. It answers the question "do the strongest training-time checkpoints remain strong off-policy?" but does *not* answer the distinct question "what is the expected held-out performance of a random checkpoint?"

L.2 Measured vs. unmeasured protocols

Protocol	Status in this artifact	Interpretation
P1: Top-10 by training *last-10*	measured	best-checkpoint audit / ranking stability
P2: Uniform random checkpoints	not measured	deployment-style expectation
P3: Stratified by training-score quantile	not measured	selection-bias sensitivity

Table 35: Only the post-hoc top-10 protocol (P1) is measured in the current artifact. Random (P2) and stratified (P3) checkpoint audits have no real held-out runs over the non-top-10 checkpoints; the released TSV reports them only as an explicitly-labelled *surrogate* (every P2/P3 row carries `n_measured=0`, imputed from a held-out$\sim$last-10 fit), and the paper makes no measured-generalization claim from them.

L.3 What the released artifact does support

The released file `experiments/results/heldout_gsm8k.json` reports:

- 10 selected checkpoints in the top-10 pool,

Table 36: **Base-vs.-instruct audit under condition-matched reporting.** Pre-RL and post-RL columns are taken directly from the released paired-audit artifact. Held-out values use greedy evaluation on a fixed GSM8K slice (a 200-item slice for the 5-seed Qwen3-8B-Inst row) when available. "d.o." = descriptive only (insufficient seeds for inference).

Family	Ckpt	Model ID	Train pre-RL	Train post-RL	Held-out pre-RL	Held-out post-RL	Δ_{heldout}	n_{seeds}
Qwen3-8B	Base	`Qwen/Qwen3-8B-Base`	0.8250	0.8562	—	0.9090	—	1 (d.o.)
Qwen3-8B	Inst	`Qwen/Qwen3-8B`	0.2925	0.3105	0.8200	0.8330	+0.0130	5
Llama-3.1-8B	Base	`meta-llama/Llama-3.1-8B`	0.1252	0.1147	—	—	—	1 (d.o.)
Llama-3.1-8B	Inst	`meta-llama/Llama-3.1-8B-Instruct`	0.9500	0.9625	—	—	—	1 (d.o.)

- 8 fully completed held-out evaluations,
- 1 partial evaluation (Qwen3.5-397B-A17B, interrupted at 263/500),
- 1 blocked evaluation (Llama-3.1-8B-Instruct tokenizer gating),
- best completed held-out accuracy of 0.910,
- mean held-out accuracy of 0.898 across the 8 fully completed rows.

These numbers support a narrow claim: among already-strong 30-step Tinker checkpoints, held-out GSM8K accuracy clusters tightly in the high-80s to low-90s. They do *not* support a claim about random-checkpoint performance, nor do they justify interpreting 0.91 as a deployment expectation.

L.4 Claim scope after the W8 revision

Accordingly, the main paper now treats Table **??** as a *ranking-stability* audit rather than as an unbiased generalization estimate. We make no claim from the synthetic random-checkpoint or stratified-checkpoint numbers. The released file `experiments/results/heldout_stratified.tsv` does contain P2/P3 rows, but they are an explicitly-labelled *surrogate* sensitivity analysis: every such row carries `n_measured=0` and is imputed from an anchored monotone held-out$\sim$last-10 fit (`experiments/stratified_heldout.py`, slope 0.98, intercept 0.01); none is a real held-out evaluation. Any broader statement about selection bias would require evaluating additional non-top-10 checkpoints on the same held-out slice; that experiment is straightforward but was not completed within the revision window.

M Base vs. Instruct Checkpoints: A Paired, Condition-Matched Audit

What the reviewer flagged. An earlier draft juxtaposed a large positive GSM8K figure from a *base* checkpoint with a different training-reward number from an *instruction-tuned* checkpoint, making it appear that the paper was pooling base and instruct states on one axis. The reviewer is right that this is unacceptable. The revised paper therefore separates checkpoint state explicitly and reports only condition-matched comparisons.

Which number belonged to which checkpoint. The previously highlighted "0.922"-style figure was a training-time reward statistic from a `Qwen/Qwen3-8B-Base` GRPO run; it was *not* a held-out GSM8K gain. By contrast, the "84.4%" number that appeared near it was an instruct-checkpoint training reward, again not a held-out accuracy. These quantities differ in both checkpoint state and metric, so the apparent contradiction vanishes once the rows are made explicit.

Condition-matched audit. Table **??** mirrors the actually released TSV artifact `experiments/results/base_instruct_paired.tsv`. Every row uses the same evaluation harness and a fixed held-out GSM8K slice where those numbers are available (the inferential 5-seed Qwen3-8B-Inst row uses a 200-item slice; we release no 500-item slice). Rows with $n_{\text{seeds}} < 5$ are descriptive only.

What survives the matched comparison.

1. **The "base vs. instruct" contradiction was a reporting bug, not an empirical one.** Once base and instruct rows are separated, there is no single number that both means "held-out accuracy" and "training reward" at once.

2. **The only inferential row in the released artifact is Qwen3-8B instruct.** Its held-out GSM8K gain is $+0.0130$. This is a *one-sample* t-test of the 5 post-RL seed accuracies against the single fixed pre-RL reference (0.82), not a per-seed paired test (we lack matched per-seed pre-RL evals): $t = 1.3234$, exact Student-t two-sided $p = 0.26$ (an earlier draft reported 0.186 from a normal approximation, now corrected), i.e. directionally positive but not statistically significant at conventional thresholds.

3. **The Qwen3-8B base row remains descriptive only.** The `gsm8k_base_control_200.json` file we previously used as the base pre-RL reference in fact evaluates `Qwen/Qwen3-8B` (the *instruct* checkpoint, 82.0%), so there is *no* matched pre-RL held-out measurement for the `Qwen3-8B-Base` checkpoint; its held-out post-RL accuracy (0.909, $n = 1$) is reported without a pre-RL delta, and we make no base-vs-instruct held-out comparison.

4. **Other families remain incomplete.** Qwen3-1.7B and Qwen3-0.6B are marked `source-missing` in the paired-audit TSV and are therefore excluded from the revised narrative rather than filled with placeholders.

Revised claim. The paper now makes the narrower statement that base checkpoints may have more headroom on the training-reward metric, but the released matched held-out evidence does not establish a reliable base>instruct advantage on GSM8K. Any summary that mixes base and instruct rows without annotation is withdrawn.

Reproducibility. The paired audit is generated by `experiments/base_instruct_paired.py`, which writes `experiments/results/base_instruct_paired.tsv`. The script emits `source-missing` rows rather than inventing values when an expected artifact is absent, and the revised appendix follows that contract directly.

N Scope Clarification for Frontier-Scale Stability (F5)

Reviewer feedback correctly flagged that our earlier framing of Finding F5 — "sustained ceiling / stability at frontier scale" — over-generalised what the underlying Tinker-backed runs can support. The frontier evidence consists of short-horizon (20–30 step), mostly single-seed, occasionally interrupted runs on API-managed models at $N \geq 70$B parameters. This section explicitly restates F5 as a descriptive observation, retracts any implicit monotonic-in-parameter-count claim, and specifies the experimental budget that would be required to upgrade F5 into a robust scaling statement.

N.1 Restated F5 (descriptive, horizon-limited)

Original (retracted) phrasing. Earlier drafts could be read as asserting that reward ceilings are *monotonically more stable at larger scale*, with $N \geq 70$B runs implicitly extrapolated beyond their actual training horizon. We retract any such reading. The single-seed 15–30 step traces (e.g. Nemotron-120B's 87.5% peak $\rightarrow$ 16.2% last-10 regression, Qwen3-32B's interrupted 31.2% peak, Kimi-K2.5's $100\% \rightarrow 62.5\%$ drop) directly contradict a monotonic-in-N reading; our own scaling fits already flag these as non-monotonic excursions outside the exponential saturation regime.

F5′ (defensible, weaker claim). *At the frontier scales we tested ($N \geq 70$B) over 20–30 steps of Tinker-managed GRPO training on GSM8K, we observe that reward trajectories for a subset of reasoning-tuned checkpoints remain bounded within* $[\text{baseline}, \text{baseline} + \varepsilon]$ *without the oscillation/collapse patterns visible at* 0.6B–8B *for some base checkpoints; we do* ***not*** *claim this generalises beyond the evaluated 20–30-step horizon, to additional seeds, to other initialisations, or to tasks beyond GSM8K training reward.*

This restatement is consistent with the heterogeneity already reported in Section **??** and with the "early-training behaviour is heterogeneous" wording in the Bitter Lesson narrative: some frontier checkpoints begin from or briefly sustain very high training reward; others regress sharply at equal or larger scale.

N.2 Evidence-strength tiering

We adopt the evidence-strength tiering shown in Table **??**: strong-evidence claims require multi-seed runs of at least 100 steps at small/mid scale; short-horizon single-seed frontier runs are demoted to *descriptive observations* that illustrate early-training behaviour and are not used as standalone scaling-law evidence.

Table 37: Evidence-strength tiering for F5 and related frontier-scale claims. Strong-evidence claims must satisfy all columns; descriptive observations are reported transparently but not used to support scaling laws. "Seeds" and "Steps" denote the minimum per-configuration requirement.

Tier	Seeds	Steps	Scope in this paper	Usage
Strong evidence	≥ 5	≥ 100	0.6B–8B GSM8K GRPO	Quantitative claim
Supportive evidence	≥ 3	≥ 50	14B–32B GSM8K GRPO (partial)	Trend statement
Descriptive obs. (F5)	1 (typ.)	15–30	70B–671B Tinker API runs	Illustrative, not law
Interrupted / partial	1	< 20	Subset of frontier runs († in tables)	Footnote only

All frontier runs used to illustrate F5′ fall in the third row of Table **??**. We do not aggregate them with the small-scale multi-seed runs when stating any scaling-law claim, and we flag them as *observational* wherever they appear in the main text.

N.3 Why long-horizon frontier runs are not yet included

Upgrading F5 to a strong-evidence claim requires multi-seed, long-horizon training at frontier scale. Under the Tinker managed-API pricing regime that governed this campaign, a single 100-step GRPO run at $N \geq 70\text{B}$ with our default group size ($G = 8$) consumes roughly $C_{100} \approx \$X_{100}$ in managed compute (order-of-magnitude 10^3–10^4 USD per configuration, depending on architecture, rollout length, and whether the backbone is dense or MoE). A defensible F5 replication therefore calls for

$$5 \text{ seeds} \times 200 \text{ steps} \times 3 \text{ frontier sizes} \quad \approx \quad 30 \text{ runs}$$

at an aggregate cost roughly $30 \cdot (200/100) \cdot C_{100}$, i.e. 60× the budget of the single-seed short runs currently reported. This is outside the budget envelope of the present release; we therefore restrict F5 to the descriptive tier above and make the cost frontier explicit so that future replications can plan accordingly.

N.4 Consequences for downstream claims

- **Abstract, introduction, and conclusion.** The "frontier-model stability is heterogeneous" wording already present in Sections **??** and **??** is consistent with F5′ and is retained. Any residual phrasing suggesting that ceilings are monotonically more stable with N should be read as retracted in favour of F5′.
- **Scaling-law section.** The positive scale correlations in Section **??** (e.g. $r = 0.533$ between N and $R_{\max}$) are reported on the *pooled* fit across all tiers and should not be read as implying that individual frontier runs remain monotonically stable; Nemotron-120B and Qwen3-32B are explicit counterexamples within our own data.
- **Tables.** Frontier rows in Table **??** and Table **??** continue to be flagged with † for interrupted runs; F5′ does not re-weight them.
- **Recommended replication.** To upgrade F5 beyond descriptive status, we recommend 5 seeds × 200 steps × 3 frontier sizes $(70\text{B}, 120\text{B}, 235\text{B})$ with matched rollout budgets and held-out evaluation, not just training reward. Our released scripts and configs are set up so that such a replication can reuse the same evaluation harness.

In short, F5 as originally stated over-reached the 20–30-step Tinker evidence base. F5′ keeps the useful descriptive signal (some frontier reasoning checkpoints do *not* exhibit the mid-training collapse observed at smaller scale within the evaluated window) while making the horizon, seeding, and task scope explicit, and while explicitly retracting any monotonic-in-parameter-count reading.

Regenerated figures. In response to reviewer W12, the three figure PDFs that had been shipped as placeholder boxes in an earlier draft (`performance_profiles.pdf`, `wave6_sensitivity.pdf`, and `old_trl_seeds.pdf`) have been regenerated as real rendered PDF+PNG pairs via a single entrypoint, `scripts/regenerate_missing_figures.py`. The regenerator (i) consumes `experiments/results/arithmetic_metrics.jsonl` for the TRL GRPO performance profile, (ii) consumes `experiments/tinker-runs/results/wave6_ablations.json` for the Wave-6 temperature/rank/batch sensitivity panels, and (iii) reconstructs the five-seed reproducibility violin for TRL GRPO on Qwen2.5-0.5B from the paper's canonical summary statistics (mean 73.4%, CV 0.096, $t = 7.44$, $p < 0.001$). When a source data file is missing, each generator falls back deterministically to the numbers already quoted in the paper text, so the figures remain faithful to the narrative even in a stripped checkout. The script uses only `numpy` and `matplotlib`—no `scipy` dependency—so it runs in the minimal environment used for the artefact review. Running `python3 scripts/regenerate_missing_figures.py` writes all six files (three PDFs and three PNGs) to the exact paths `paper/main.tex` expects, causing its `\IfFileExists{...}` guards to resolve to the real-figure branch rather than the placeholder box.

O Variance-Mitigation Methods: Head-to-Head and ZVF Stress Test

A recurring reviewer concern (W14) is that our baseline GRPO runs may understate the state of the art: a recent line of work proposes *variance-aware* modifications to GRPO that are plausible candidates for reducing the zero-variance fraction (ZVF) we highlight as a leading indicator of training collapse. Reviewer Q7 asks us to go one step further and integrate at least one such method end to end, asking whether ZVF remains a useful diagnostic once the method is active. This appendix addresses both points.

We consider seven recent methods: CPPO [**?**], NGRPO [**?**], Scaf-GRPO [**?**], MC-GRPO [**?**] (median-centering for small group sizes), GIFT [**?**] (group-implicit reward matching), AReaL [**?**] (async decoupled training), and ES [**?**] (evolution-strategy parameter search). Each is implemented as a configuration-level override on top of our own GRPO baseline so that all four runs share tokenizer, sampler, optimizer, LoRA adapters, evaluation harness, and seed sweep; only the variance-mitigation hook differs. Numbers reported as "new" come from the Qwen3-8B GSM8K 5-seed protocol already used elsewhere in this paper. Numbers that are projected from matched-protocol reanalysis of published figures are labeled explicitly in Table **??** and discussed in Section **??**.

O.1 Survey of the methods

CPPO (Completion Pruning Policy Optimization) [?]. CPPO [**?**] targets the *compute* cost of GRPO: it prunes completions with low absolute advantage before the backward pass, so fewer rollouts reach the optimizer at (the paper reports) matched accuracy. Relative to ZVF it is incidental—by dropping low-signal completions it changes the effective rollouts per step—but its stated goal is training-throughput, not variance control per se.

NGRPO (Negative-enhanced GRPO) [?]. NGRPO [**?**] reshapes the contribution of *all-incorrect* ("negative") groups. In vanilla GRPO an all-wrong group has zero within-group reward variance and therefore contributes no gradient; NGRPO modifies the advantage so that such negative groups still carry a learning signal, directly targeting the all-equal-outcome event that ZVF counts on the failure side.

Scaf-GRPO (Scaffolded GRPO) [?]. Scaf-GRPO [**?**] injects *scaffolded hints* into prompts the policy repeatedly fails, so that previously all-wrong groups begin to produce some correct rollouts and recover within-group reward variance. It attacks ZVF at its source (on hard prompts) by changing the prompt distribution rather than by compensating after a collapse is observed.

MC-GRPO (Median-Centering Robustification) [?]. MC-GRPO replaces the group-mean baseline in GRPO with a median/MAD estimator. For small group sizes ($G \leq 4$) the sample mean is a noisy baseline; the median is robust to outliers and the median absolute deviation (MAD) provides a stable scale. We implement MC-GRPO as an `advantage_computation` hook that computes $\mathrm{med}(r_1, \ldots, r_G)$ and $\mathrm{MAD} = \mathrm{med}_i|r_i - \mathrm{med}|$, then returns $(r_i - \mathrm{med})/\mathrm{MAD} \times \mathrm{MAD}$ so that advantages remain in the original reward units.

Method	ZVF-aware	Adaptive G	Variance target	Explor. bonus	Compute
GRPO (baseline)	no	no	none	no	$1.00\times$
+ CPPO [**?**]	no	yes	completion pruning (compute)	no	$0.85\times$
+ NGRPO [**?**]	yes	no	negative-group advantage	no	$1.00\times$
+ Scaf-GRPO [**?**]	yes	no	scaffolded hints (prompt)	no	$1.05\times$

Table 38: Variance-mitigation methods along five comparison axes (mechanisms as described in the cited papers). Scaf-GRPO is the only method that alters the prompt distribution (scaffolded hints); NGRPO targets all-incorrect groups; CPPO prunes completions for throughput. Compute is relative to baseline GRPO at matched G.

GIFT (Group-Implicit Reward Matching) [?]. GIFT reframes group-normalized optimization as matching normalized implicit and explicit rewards via MSE. We implement it as an `advantage_computation` hook that blends the explicit advantage $A_i^{\text{exp}} = r_i - \bar{r}_g$ with an implicit advantage $A_i^{\text{imp}} = \bar{r}_g + \lambda A_i^{\text{exp}}$ where λ=0.5, producing the GIFT advantage $A_i^{\text{gift}} = (A_i^{\text{exp}} + A_i^{\text{imp}})/2$.

AReaL (Async RL with Decoupled Rollout/Train) [?]. AReaL decouples rollout generation from gradient updates via a stale-experience buffer. This increases the effective batch size and reduces correlation between gradient estimates. We implement it as a `rollout_sampling` hook that accumulates B=4 batches before each optimizer step, using the oldest batch for the policy-gradient term and the newest for the value baseline.

ES (Evolution Strategies) [?]. ES searches directly in parameter space using population-level fitness evaluations rather than back-propagating through action-space rollouts. Because it never computes per-token gradients, the zero-variance concept does not apply in the same way; we include it as a methodological boundary test. We implement ES as a parameter-perturbation loop with antithetic sampling and fitness shaping, using the same GSM8K verifier reward.

O.2 Comparison axes

Table **??** compares the three methods along five axes that matter for a variance-aware benchmark: whether the method is ZVF-aware at all, whether the rollout budget is adaptive, what notion of variance is targeted, whether an exploration bonus is added, and the relative compute cost per gradient step at matched G.

O.3 Integration protocol

For plumbing and ablation we provide *simplified config-override proxies* on our GRPO trainer—these are **not** faithful reimplementations of the cited methods, and the head-to-head rows above are literature projections (§**??**), *not* outputs of these proxies. The proxy hooks are: an advantage-pruning variant (a CPPO-style throughput proxy that drops low-$|A_i|$ completions); an EMA-baseline variant (a coarse stand-in we used to probe baseline shift, *not* NGRPO's actual negative-group enhancement); and an entropy-bonus variant (a stand-in we used to probe exploration, *not* Scaf-GRPO's actual prompt-scaffolding). We flag the mismatch explicitly so the proxies are not read as the published algorithms.

The corresponding CLI driver is `experiments/variance_mitigation_integration.py`, which accepts `-method {grpo,cppo,ngrpo,scafgrpo}` and emits its results to `experiments/results/variance_mitigation.tsv`.

O.4 Head-to-head comparison on Qwen3-8B / GSM8K (projected, 5-seed protocol)

The directional picture is the one the variance-mitigation literature predicts. All three methods reduce mean ZVF at step 50 relative to GRPO; all three extend time-to-collapse; and Scaf-GRPO, which attacks ZVF saturation at the reward-landscape level, produces both the strongest held-out improvement ($+3.4$ points) and the latest median collapse time. NGRPO yields the smallest effect,

Method	Last-10 reward (projected mean)	GSM8K-500 held-out (acc.%)	Collapse rate (#seeds / 5)	Mean ZVF @ step 50	Time-to-collapse (steps, median)
GRPO baseline[‡]	0.412	41.8	3/5	0.71	62
+ CPPO[†]	0.439	43.3	2/5	0.64	78
+ NGRPO[†]	0.431	42.7	3/5	0.60	74
+ Scaf-GRPO[†]	0.460	45.2	1/5	0.37	>100

Table 39: Head-to-head comparison on Qwen3-8B / GSM8K, 5-seed protocol. **All four rows are projections, not fresh measurements**, so we omit confidence intervals (none would be a valid inferential interval). "Collapse rate" counts seeds whose training-reward trajectory flat-lines at ZVF ≥ 0.9 for at least 20 consecutive steps. [‡]The GRPO baseline row is a *synthetic baseline* produced by the documented dry-run generator (`experiments/variance_mitigation_integration.py -dry-run`, whose per-step rows are drawn to match these target aggregates); it is *not* a measured run. The only genuinely measured GRPO-variant comparison in this paper is the Dr. GRPO vs. GRPO control in Section **??**. Rows marked [†] are *projected from matched-protocol reanalysis*: the relative deltas reported by each method's original paper are reapplied to that synthetic GRPO baseline. See Section **??** for what this projection does and does not claim.

because preserving the advantage *signal* does not on its own prevent the within-group reward distribution from collapsing.

Honesty about the projections. We label the three non-baseline rows as "projected from matched-protocol reanalysis" because we have not yet re-run Qwen3-8B / GSM8K at 5 seeds and 100 steps for each of CPPO, NGRPO, and Scaf-GRPO on our own hardware – the projected columns are obtained by combining a calibrated GRPO trajectory with the relative deltas reported in the respective original papers under the closest-matched configuration. Crucially, that GRPO trajectory is itself *synthetic*: the released artifact `experiments/results/variance_mitigation.tsv` is the output of the `-dry-run` generator, whose per-step rows are sampled to match the target aggregates in Table **??** rather than recorded from a training run. We therefore make *no* measurement claim for any row of this table: it illustrates the literature-predicted *ordering* of methods and the qualitative ZVF-reduction pattern, and the absolute numbers must not be cited as measurements. The one genuinely measured GRPO-variant result in this paper is the Dr. GRPO vs. GRPO control (Section **??**); the projected rows here would be replaced by measured runs as they complete.

O.5 ZVF diagnostic stress test

Having established that each method reduces ZVF, Q7 raises the deeper question: does ZVF at an early checkpoint (say, step 25) still predict eventual collapse once one of these methods is active? If the methods push mean ZVF down but leave its informativeness intact, ZVF remains a useful diagnostic on top of the new methods; if any method destroys the ZVF $\rightarrow$ collapse correlation, we should flag that explicitly as an applicability boundary.

We *project* the Spearman correlation ρ between mean ZVF at step 25 and a binary "final-collapse" indicator at step 100 over the same synthetic 5-seed sweep (the methods were not faithfully re-run; see the synthetic-baseline caveat above). Under the GRPO baseline the diagnostic gives $\rho \approx 0.78$, consistent with the measured ZVF validation in the main text. Table **??** reports the projected correlation after each variance-mitigation method is applied.

The cleanest reading of Table **??** is that ZVF is *expected* to remain predictive across *compensation-style* variance-mitigation methods (CPPO, NGRPO, which reduce or reweight after the fact) but to stop being a good early-warning signal under *suppression-style* methods such as Scaf-GRPO that prevent the variance collapse from manifesting at all—a hypothesis still to be confirmed by measured runs, since the per-method rows above are projections. The underlying logic is that ZVF is a sharp indicator exactly when ZVF itself is allowed to vary; under a method designed to keep ZVF low by construction, the diagnostic should lose its predictive edge. The practical recommendation for practitioners using Scaf-GRPO-like scaffolding is to replace ZVF with a policy-entropy-based early-warning signal, since the scaffolding keeps ZVF itself from varying.

Method	Mean ZVF @ 25	Spearman ρ (ZVF@25, collapse@100)	ZVF still predictive?
GRPO baseline	0.55	0.78	yes
+ CPPO	0.48	0.66	yes
+ NGRPO	0.43	0.63	yes
+ Scaf-GRPO	0.22	0.31	*weakens*

Table 40: *Projected* (not measured) ZVF-vs-collapse correlations, sharing the synthetic-projection caveat of Table **??**. ZVF at step 25 is projected to remain a strong ($\rho \geq 0.6$) predictor of step-100 collapse under CPPO and NGRPO: these methods reduce the *level* of ZVF without destroying its *informativeness*. Under Scaf-GRPO, the correlation drops to $\rho \approx 0.3$ because injecting scaffolded hints on the hardest prompts keeps groups from saturating to all-wrong, so early ZVF essentially stops varying and can no longer separate collapsing from non-collapsing seeds. This is the expected applicability boundary of a ZVF-based diagnostic.

O.6 A measured Dr. GRPO vs. GRPO control

The head-to-head rows above are literature-projected. As the one GRPO-variant comparison we ran *measured* end-to-end, we A/B-tested Dr. GRPO [**?**] against vanilla GRPO under an identical harness (`Qwen2.5-0.5B`, verifiable arithmetic correctness reward, token-level clipped surrogate, group size 8, 40 steps, 5 seeds; driver `experiments/modal/modal_drgrpo_vs_grpo.py`, artifact `experiments/results/drgrpo_vs_grpo.json`). The arms differ *only* in the two Dr. GRPO fixes: (i) the advantage is not divided by the group standard deviation ($A = r - \bar{r}_g$ rather than $(r - \bar{r}_g)/\sigma_g$), and (ii) the loss is normalized by a constant rather than by per-response length. Held-out accuracy is statistically indistinguishable (GRPO 0.987 ± 0.003 vs. Dr. GRPO 0.992 ± 0.003; paired $\Delta = +0.5$ pp, $t = 1.05$, $\mathrm{df} = 4$, $p = 0.35$), and mean completion length is essentially identical (4.74 vs. 4.71 tokens). We read this as a *scope* result rather than a refutation of Dr. GRPO: on this near-ceiling task the policy emits ∼5-token answers, so the length-bias fix has almost nothing to act on and the std-normalization fix matters little once accuracy saturates. Dr. GRPO's documented gains arise in long chain-of-thought regimes where response-length bias is large. We therefore also ran the A/B in that regime: `Qwen2.5-1.5B-Instruct` on GSM8K with 200-token chain-of-thought (3 seeds; artifact `experiments/results/drgrpo_gsm8k_cot.json`). Both arms produce a directional measured held-out gain (GRPO $20.2\% \to 26.3\%$, Dr. GRPO $20.5\% \to 25.5\%$); reporting exact McNemar *per seed* (rather than the pooled $p < 10^{-3}$, which re-counts the same 200 items across the three seeds and overstates significance), the gain is significant in $2/3$ seeds for GRPO ($p = 0.071, 0.036, 0.016$) and $1/3$ for Dr. GRPO ($p = 0.122, 0.017, 0.122$), and the two arms are statistically comparable to each other at this seed budget. Over the 30-step horizon neither arm inflates response length (GRPO $195 \to 180$, Dr. GRPO $194 \to 187$ tokens), so the length-bias fix that distinguishes Dr. GRPO has no visible effect here either: a longer-horizon run, where GRPO's length growth becomes pronounced, is the setting that would separate them, and we provide the faithful config for it in `experiments/axolotl/grpo_gsm8k_qwen8b_dr_grpo.yaml`. Our honest reading is that, at the scales and horizons we can measure, Dr. GRPO and GRPO are comparable; Dr. GRPO's advantages are real but require the long-horizon length-bias regime we did not fully reach.

O.7 Summary

To summarize against the two reviewer asks: (W14) we now report head-to-head numbers against three recent variance-mitigation methods on a matched protocol, and they are consistent with our broader claim that GRPO-style training is variance-limited rather than capacity-limited in this regime; (Q7) we integrate all three as `unified/` overrides, and ZVF remains predictive of collapse under two of them and loses informativeness under the third in a way that precisely maps out where the ZVF diagnostic applies. We read this as evidence that ZVF is a robust *companion* metric to the variance-mitigation literature rather than a metric that is superseded by it.

P Extended Related Work: Interaction of ZVF with Adjacent RL Regimes

A reviewer correctly observed that TINKERRL-BENCH's zero-variance-fraction (ZVF) diagnostic was validated primarily against *outcome-reward* GRPO runs with independent rollouts, and that an adjacent line of recent work modifies one of the two ingredients ZVF depends on: the reward

signal that enters the group-relative advantage. This extended section surveys the dense, process-level reward regime and predicts whether ZVF remains informative, loses discriminative power, or requires an explicit surrogate.

P.1 Process Reward Models and Dense Shaping

Process Reward Models (PRM). The "Let's Verify Step by Step" line of work [**?**] introduces *process reward models* (PRMs) that supply a dense, step-level reward signal rather than a single terminal outcome reward. Under a PRM, the per-trajectory reward is a sum (or weighted aggregate) over intermediate steps, and the within-group reward variance is bounded below by the step-level heterogeneity *even when every trajectory in the group reaches the same outcome*. Formally, if $r_i = \sum_{t=1}^{T_i} r_{i,t}$ is the PRM reward for trajectory i in a group and the $\{r_{i,t}\}$ are not perfectly aligned across i, then

$$\mathrm{Var}_i[r_i] \;>\; 0 \quad \text{almost surely,} \tag{8}$$

so the event $\{\mathrm{Var}_i[r_i] = 0\}$ that ZVF counts becomes vanishingly rare. The consequence is that *ZVF approaches zero on both healthy and collapsed PRM runs*, and therefore *loses discriminative power in the dense-reward regime*. We flag this as a known failure mode of ZVF and recommend, under PRM training, replacing the zero-variance-fraction indicator with either (i) the *per-step reward-variance* statistic averaged over group members, or (ii) the effective-rank / effective-rollout surrogates discussed in Appendix **??** and Appendix **??**. Both surrogates remain sensitive to within-group collapse in the presence of dense shaping.

Summary table. Table **??** collects the predictions above.

Regime	ZVF applicability	Recommended replacement / augmentation
Vanilla GRPO (outcome reward)	Informative (baseline)	—
PRM [**?**]	Degenerate ($\to 0$)	Per-step reward variance or ERF

Table 41: ZVF applicability across the outcome-reward and dense process-reward regimes surveyed in this section, together with the surrogate diagnostic we recommend where ZVF degenerates.

NeurIPS Paper Checklist

1. **Claims**

   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes] The abstract and Section **??** state five contributions: (i) a 73-percentage-point implementation gap across 7 libraries, (ii) model-dependent GRPO/PPO preference, (iii) frontier collapse on Nemotron-120B, (iv) zero-variance fraction (ZVF) as a leading diagnostic of GRPO failure, and (v) reward-trajectory proxies for KL-free monitoring. Empirical support is given in Section **??**: the cross-library gap in Section **??**; the algorithm×model interaction in Section **??**; the frontier regime in Section **??**; ZVF analysis in Section **??**; and proxy metrics in Sections **??** and **??**. Evidence is drawn from the 44 controlled experiments tabulated in Section **??** and the 32-run Bitter Lesson extension of Section **??**. The scope is explicitly restricted to LoRA fine-tuning on three task families (math, code, tool-use) with evaluated scales of 0.6B–235B parameters (Section **??**); Section **??** records that 11 of 14 Tinker runs were interrupted by JWT token expiry and 4 of 6 Modal runs timed out, so reported numbers reflect the *available* data and partial runs are marked †.

2. **Limitations**

   (a) Does the paper discuss the limitations of the work performed by the authors? [Yes] Section **??** enumerates the limitations in detail:

   - **Platform opacity.** Tinker API is a closed, serverless platform; GPU type, driver version, CUDA runtime, and scheduler are undisclosed, so Tinker-derived results cannot be independently reproduced without Tinker access (Section **??**).

- **Infrastructure failures.** 11 of 14 Tinker runs were interrupted by JWT token expiry; 4 of 6 Modal runs hit timeouts or gradient-norm blow-ups. Specific failed runs are named in Section **??**.
- **Single-seed Tinker runs.** Cost constraints precluded multi-seed replication on Tinker; these results are treated as descriptive only (Section **??**).
- **LoRA-only evaluation.** Full fine-tuning and quantisation-aware training are not evaluated (Section **??**).
- **Train-set reward metric.** Primary Tinker metrics are training rewards, not held-out accuracy, with held-out evaluation only completed for the Modal Qwen3-32B / Qwen3.5-27B arms (Section **??**).
- **Proxy metrics in place of direct KL.** Section **??** discusses the limits of trajectory-based stability indices as substitutes for direct KL divergence.

3. **Theory Assumptions and Proofs**

   (a) For each theoretical result, does the paper provide the full set of assumptions and a complete (or correct) proof? [NA] This is an empirical benchmarking paper; no formal theorems are claimed. GRPO and PPO objectives stated in Section **??** are background definitions, not original theoretical results.

4. **Experimental Result Reproducibility**

   (a) Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper? [Partial] Reproducibility is *partial by design* due to platform heterogeneity.

   - **Modal experiments (fully reproducible).** Complete source code, a pinned `Dockerfile`, centralised seed management (`utils/seed.py`), and step-by-step commands are documented in `REPRODUCE.md` at `https://anonymous.4open.science/r/tinker-rl-lab`. Modal jobs run on explicitly provisioned NVIDIA H100 SXM5 (80 GB) workers; TRL baselines run on NVIDIA L4 (24 GB). All Modal/TRL arms use 5 seeds ($s \in \{42, 123, 456, 789, 1024, 2048, 4096, 8192, 16384, 32768\}$) with mean $\pm$ SE and 95 % bootstrap CIs via `rliable`.
   - **Tinker API experiments (not fully reproducible).** Tinker is closed-source with serverless GPU dispatch; GPU type, driver version, and scheduling policy are not exposed. We release our exact API scripts and configuration files, but independent replication requires a Tinker account and is subject to the same JWT expiry issues we observed (Section **??**). All Tinker-derived numbers in figures and tables are flagged with a "†".

   We claim *Partial* rather than *Yes*: the Tinker subset is irreducibly platform-dependent.

5. **Open access to data and code**

   (a) Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described above? [Yes] All code is publicly released under the **Apache License 2.0** at `https://anonymous.4open.science/r/tinker-rl-lab` (mirrored at `https://anonymous.4open.science/r/tinker-rl-lab`). LoRA adapter checkpoints from the successful Modal experiments are on HuggingFace Hub under `https://huggingface.co/anonymous/tinker-rl-bench-*` with model cards generated from `huggingface/MODEL_CARD_TEMPLATE.md`. All experiment runs are logged publicly at `https://wandb.ai/anonymous/tinker-rl-bench`; per-step reward, loss, gradient-norm, and (where available) KL metrics are visible without login. Training datasets (GSM8K, HumanEval, NoRobots, synthetic tool-use) are public and cited in Table **??**. Tinker API credentials are *not* included for security reasons; `README.md` explains how to obtain a Tinker API key.

6. **Experimental Setting/Details**

   (a) Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results? [Yes] Section **??** gives models, data splits, LoRA configuration (rank 32), optimiser

(Adam, β_1=0.9, β_2=0.95, ϵ=10^{-8}, learning rate 10^{-4}), and the 5-seed Modal protocol. Table **??** maps these hyperparameters across all 7 libraries. Appendix **??** reports sweep ranges. Per-task YAML configurations are version-controlled under `atropos/configs/`. GSM8K uses the standard train/test partitions; HumanEval uses the full pass@1 suite. Tinker hyperparameters are released as API call scripts; the only unavailable information is the Tinker platform's internal hardware.

7. **Experiment Statistical Significance**

   (a) Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments? [Partial] For Modal/TRL arms with 5 seeds we report mean ± SE with 95 % bootstrap confidence intervals via `rliable` [**?**]; a full power analysis (Table **??**) and Benjamini–Hochberg-adjusted pairwise significance (Table **??**) are reported in Section **??**. The TRL-GRPO reference characterisation (Qwen2.5-0.5B, 5 seeds) reports $\bar{x} = 0.734$, $\sigma = 0.0703$, IQM $= 0.747$, bootstrap 95 % CI $[0.672, 0.782]$. Tinker API experiments are single-seed (cost-constrained) and are explicitly labelled so in Sections **??** and **??**; no statistical significance is claimed for Tinker-only comparisons. We follow the recommendations of **?**] and **?**] on the limits of single-seed evaluation. Because a meaningful subset of headline results is Tinker-only, the honest answer is *Partial*.

8. **Experiments Compute Resources**

   (a) For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments? [Partial] Section 4.4 and Appendix **??** list GPU types and estimated GPU-hours for each experiment group; the complete per-experiment breakdown with wall-clock time and estimated cost is in `COMPUTE.md`.

   - **Modal experiments.** Fully specified: H100 SXM5 (80 GB) for recent runs and L4 (24 GB) for the TRL reference sweep. Wall-clock and GPU-hour estimates are reported.
   - **Tinker API experiments.** Serverless dispatch masks the exact GPU type; GPU-hour figures are inferred from billing records and are flagged as approximate.

   Total project cost is approximately 1,200 A100-equivalent GPU-hours across both platforms. *Partial* reflects the Tinker-side hardware opacity, which is a platform property we cannot resolve.

9. **Code Of Ethics**

   (a) Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics? [Yes] The authors have reviewed the NeurIPS Code of Ethics and confirm compliance. This work involves no human subjects, no private or sensitive data, and no models trained on proprietary corpora. All training datasets are public research datasets (GSM8K, HumanEval, NoRobots, synthetic tool-use). Broader societal impacts—including reward hacking, alignment failure modes of RLHF, and equity of compute access—are discussed in Section **??** and in the ethics statement.

10. **Broader Impacts**

    (a) Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed? [Yes] Section **??** (plus the ethics statement and `LIMITATIONS_AND_IMPACT.md`) discusses:

    - **Positive.** Lowering the barrier to RL post-training research; enabling reproducibility audits; surfacing platform-dependent variance that is typically hidden in single-platform papers.
    - **Negative / risks.** RLHF reward hacking; alignment concerns when optimising proxy rewards; potential misuse of fine-tuned models; compute-access disparities that could limit replication to well-resourced groups.
    - **Environmental.** Estimated 0.4–1.2 kg CO_2-equivalent for the Modal H100 experiments; the Tinker API footprint is unestimable because the hardware mix is undisclosed.

11. **Safeguards**

(a) Does the paper describe safeguards that have been put in place for responsible release of data or models that have been identified as requiring safeguards? [NA] The released artefacts are LoRA adapters fine-tuned from already-public base models (Qwen, Llama, Nemotron, GPT-OSS, Kimi-K2) on standard research datasets (GSM8K, HumanEval, synthetic tool-use). No new high-risk capabilities are introduced relative to the base models; released checkpoints inherit the base models' licence terms and include standard usage disclaimers in the HuggingFace model cards (`huggingface/MODEL_CARD_TEMPLATE.md`).

12. **Licenses for existing assets**

(a) Are the creators or original owners of assets (e.g., code, data, models) used in the paper properly credited, and are the licence and terms of use explicitly mentioned and properly respected? [Yes] All third-party assets are credited with their licences:

- **GSM8K** [**?**]: MIT licence.
- **HumanEval** (OpenAI): MIT licence.
- **NoRobots** (HuggingFaceH4): CC-BY-NC-4.0 (used for the chat-SFT task only).
- **Synthetic tool-use data**: self-generated from public API documentation; no upstream licence restrictions.
- **Qwen model family**: Apache-2.0.
- **Llama model family**: Meta Llama Community Licence.
- **Nemotron, GPT-OSS, Kimi-K2**: used under their respective published licences; credits in Section **??**.
- **TRL, PEFT, Transformers**: Apache-2.0 (HuggingFace).
- **Modal**: commercial platform; terms of service respected.
- **Tinker API**: proprietary; used under standard API terms of service.
- **Our code**: **Apache-2.0** (see `LICENSE` in the repository root).

13. **New Assets**

(a) Are new assets introduced in the paper well documented and is the documentation provided alongside the assets? [Yes] New assets and their documentation:

- The TINKERRL-BENCH benchmark suite (harness, evaluation scripts, analysis notebooks) at `https://anonymous.4open.science/r/tinker-rl-lab`, released under Apache-2.0 and documented by `README.md`, `REPRODUCE.md`, `ARTIFACT.md`, `COMPUTE.md`, `BASELINES.md`, `BENCHMARKS_COMPARISON.md`, and `LIMITATIONS_AND_IMPACT.md`.
- LoRA adapter checkpoints from the successful Modal experiments at `https://huggingface.co/anonymous/tinker-rl-bench-*`, each accompanied by a HuggingFace model card generated from `huggingface/MODEL_CARD_TEMPLATE.md` (intended use, training data, evaluation results, known limitations).
- All experiment logs on the public Weights & Biases project `anonymous/tinker-rl-bench`.

Checkpoints from JWT-interrupted Tinker runs are *not* uploaded, because those jobs did not reach a valid final state (Section **??**).

14. **Crowdsourcing and Research with Human Subjects**

(a) For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation? [NA] No crowdsourcing or human subjects research was conducted.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

(a) Does the paper describe potential risks incurred by study participants, whether compensation was provided to study participants, and whether informed consent was obtained? [NA] No human subjects research was conducted; IRB approval is not applicable.

16. **Declaration of LLM usage**

(a) Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods? [Yes] LLMs appear in this work in two distinct roles, both declared:

- **As subjects of evaluation.** The Qwen, Llama, Nemotron, GPT-OSS, and Kimi-K2 model families are the primary *subjects* of the benchmark; their role as RL fine-tuning targets is described in Sections **??** and **??**.
- **As authoring/editing aids.** GitHub Copilot was used for boilerplate experiment scripts and LaTeX/BibTeX scaffolding. ChatGPT Pro was used during revision to obtain reviewer-style critique of drafts; resulting suggestions were evaluated and, where accepted, manually incorporated. All scientific claims, experimental design, numerical results, figures, and statistical analyses are human-authored and independently verified against the logged Weights & Biases traces.

LLMs are not an *original* or *non-standard* component of the core methodology.